Fall 2026 Lecture Notes

STAT 8670

Computational Methods in Statistics

Chi-Kuang Yeh

Assistant Professor in Statistics

Department of Mathematics and Statistics

Georgia State University

Personal website

chikuang.github.io

Course website

chikuang.github.io/teaching/2026F_stat8670/

Updated 2026-10-21

Table of contents

Preface **9**
Description . . . 9
Prerequisites . . . 9
Instructor . . . 9
Office Hour . . . 9
Grade Distribution . . . 9
Assignment . . . 9
Midterm . . . 10
Project . . . 10
Topics and Corresponding Lectures . . . 10
Recommended Textbooks . . . 10
Side Readings . . . 11

1 Introduction to R **12**
1.1 Why R? . . . 12
1.2 The working environment . . . 13
1.2.1 RStudio . . . 13
1.2.2 R scripts and Quarto documents . . . 14
1.2.3 A project-oriented workflow . . . 16
1.3 R as a computing environment . . . 17
1.4 Objects, data, and functions . . . 17
1.5 Expressions, objects, and names . . . 18
1.6 Data types . . . 19
1.6.1 Changing data types . . . 19
1.7 Operators . . . 20
1.8 Vectorization . . . 21
1.9 Recycling . . . 22
1.10 Coercion . . . 22
1.11 Missing and special values . . . 23
1.12 Indexing and subsetting . . . 24
1.13 Naming elements . . . 25
1.14 Arrays and matrices . . . 26
1.15 Lists and key-value pairs . . . 28
1.16 Factors . . . 29
1.17 Data frames . . . 30
1.17.1 Inspecting a data frame . . . 31
1.17.2 Subsetting a data frame . . . 32
1.17.3 Adding and modifying variables . . . 33
1.17.4 Importing rectangular data . . . 33
1.18 Functions . . . 34
1.18.1 Argument matching . . . 35
1.18.2 Lazy evaluation . . . 35
1.19 Environments and lexical scoping . . . 36
1.20 Memory and copy-on-modify . . . 36

1.21 Iteration: vectorization, loops, and the apply family . . . 37
1.21.1 `apply()` . . . 38
1.21.2 Why `apply()` can be dangerous for data frames . . . 39
1.21.3 `lapply()` . . . 40
1.21.4 `sapply()` and `vapply()` . . . 41
1.21.5 `tapply()` . . . 41
1.21.6 `Map()` and `mapply()` . . . 42
1.21.7 Choosing an iteration tool . . . 42
1.22 Packages and help . . . 42
1.23 The tidyverse . . . 43
1.23.1 Data frames and tibbles . . . 44
1.23.2 Pipe operators . . . 44
1.23.3 Core `dplyr` verbs . . . 45
1.23.4 Applying functions across several columns . . . 46
1.24 Base R and tidyverse are complementary . . . 46
1.25 Numerical computation and finite precision . . . 47
1.26 Randomness and reproducibility . . . 48
1.27 Debugging and error messages . . . 49
1.28 A computational workflow for this course . . . 50
1.29 In-class exercises . . . 50
1.30 Takeaways . . . 52
1.31 Optional challenge . . . 53

2 Intro to Computational Statistics **55**
2.1 Theory versus Computation . . . 56
2.2 Matrix Inversion . . . 57
2.2.1 Multi-collinearity . . . 59
2.2.2 Trade-off . . . 60

3 Optimization **64**
3.1 Why Numerical Optimization? . . . 64
3.2 When There Is No Closed-Form Solution . . . 66
3.3 Two Families of Optimization Methods . . . 68
3.3.1 Deterministic Methods . . . 69
3.3.2 Metaheuristic Methods . . . 69
3.4 Local and Global Optima . . . 70
3.5 Deterministic Numerical Methods . . . 73
3.5.1 Root Finding . . . 73
3.5.2 Bisection Method . . . 75
3.5.3 Newton-Raphson Method . . . 78
3.5.4 Visualizing Newton's Method . . . 80
3.5.5 Secant Method . . . 82
3.6 Convergence . . . 84
3.6.1 Comparing Convergence in R . . . 85
3.7 From One Dimension to Multiple Dimensions . . . 87
3.8 Gradient Descent . . . 90
3.8.1 Visualizing the Optimization Path . . . 91
3.8.2 Step Size Matters . . . 93
3.9 Multivariate Newton's Method . . . 94
3.10 Quasi-Newton Methods . . . 95
3.11 Scaling and Conditioning . . . 97
3.12 Metaheuristic Methods . . . 99
3.13 Simulated Annealing . . . 100
3.13.1 Simulated Annealing in R . . . 102

3.14 Genetic Algorithm . . . 104
3.14.1 Selection . . . 104
3.14.2 Crossover . . . 105
3.14.3 Mutation . . . 105
3.15 Particle Swarm Optimization . . . 105
3.15.1 Advantages . . . 106
3.15.2 Limitations . . . 106
3.16 Deterministic versus Metaheuristic Methods . . . 106
3.17 Computational Perspective . . . 107
3.18 Error Analysis and Convergence Rates . . . 108
3.19 Questions to Think at Home . . . 111
3.20 Takeaways . . . 112
3.21 Optimal topic: Real-World Applications . . . 112

4 Generating Random Variables **122**
4.1 Overview . . . 122
4.2 Inverse Transformation Method . . . 124
4.2.1 Continuous Distributions . . . 126
4.2.2 Discrete Distributions . . . 131
4.3 Acceptance-Rejection Method . . . 133
4.4 Using Probability Distribution Theory . . . 140
4.5 Sums and Mixtures . . . 145
4.5.1 Sums and Convolution . . . 145
4.5.2 Mixture Distributions . . . 146
4.6 Generating Multivariate Random Variables . . . 149
4.6.1 Multivariate Normal Distribution . . . 149
4.6.2 Multivariate Normal Mixtures . . . 153
4.6.3 Uniform Directions and Points in a Ball . . . 154
4.7 Checking a Generator . . . 157
4.8 Concept Checks and Practice . . . 160
4.9 Chapter Summary . . . 162
4.10 Supplementary . . . 162
4.10.1 Real-World Applications . . . 163
References and Further Reading . . . 167

5 Monte Carlo Simulation and Variance Reduction **168**
5.1 The Main Idea: Simulate, Calculate, Average . . . 168
5.1.1 Why use simulation? . . . 168
5.1.2 What happens in one run? . . . 168
5.1.3 A Practical Example: Are 12 Muffins Enough? . . . 169
5.1.4 A Small Mathematical Example . . . 171
5.2 Two Useful Translations . . . 172
5.2.1 A probability is a proportion . . . 172
5.2.2 An integral is width times average height . . . 173
5.3 Where Does the Variation Come From? . . . 174
5.3.1 SD describes outputs; MCSE describes the estimate . . . 175
5.3.2 The rule of four . . . 176
5.4 Variance Reduction: A More Stable Average . . . 176
5.5 Antithetic Variables: Pair Opposites . . . 177
5.6 Control Variates: Use a Known Mean . . . 178
5.6.1 Which direction should the correction go? . . . 179
5.7 Compare the Three Methods . . . 180
5.8 Quick Check . . . 181
5.9 What to Remember . . . 182

5.10 Supplementary . . . 183
5.10.1 Why the average works . . . 183
5.10.2 Planning a sample size . . . 183
5.10.3 Reporting variance reduction . . . 184
5.10.4 Why antithetic pairing helps . . . 184
5.10.5 Deriving the control coefficient . . . 185
5.10.6 Estimating the coefficient in a pilot run . . . 185
5.10.7 Application: Estimate a Normal Probability . . . 186
5.10.8 Two more variance-reduction ideas . . . 187
References and Further Reading . . . 188

I Assignment **189**

Assignment 1 **190**
Question 1: R Programming and Statistical Computing [10] . . . 190
1a) . . . 190
1b) . . . 191
1c) . . . 191
1d) . . . 191
1e) . . . 191
1f) . . . 192
1g) . . . 192
1h) . . . 192
1i) . . . 192
1j) . . . 192
Question 2: Computational Results Using the Iris Data [10] . . . 193
2a) [2] . . . 193
2b) [2] . . . 193
2c) [2] . . . 193
2d) [2] . . . 193
2e) [2] . . . 194
Question 3: R Coding and Visualization [6] . . . 194
3a) [3] . . . 194
3b) [3] . . . 194
Question 4: Numerical Linear Algebra [4] . . . 195
4a) [2] . . . 195
4b) [2] . . . 195

Assignment 2 **196**
Question 1: Computational Approaches [8] . . . 197
1a) . . . 197
1b) . . . 197
1c) . . . 197
1d) . . . 197
1e) . . . 197
1f) . . . 198
1g) . . . 198
1h) . . . 198
Question 2: Optimization [8] . . . 199
2a) . . . 199
2b) . . . 199
2c) . . . 199
2d) . . . 199

2e) . . . 199
2f) . . . 200
2g) . . . 200
2h) . . . 200
Question 3: True or False [6] . . . 201
3a) . . . 201
3b) . . . 201
3c) . . . 201
3d) . . . 201
3e) . . . 201
3f) . . . 201
Question 4: Explain What the Results Mean [8] . . . 202
4a) Rank deficiency is not missing data [4] . . . 202
4b) A converged result still needs interpretation [4] . . . 203

Assignment 3 **204**
Question 1: Generating Random Variables [8] . . . 205
1a) . . . 205
1b) . . . 205
1c) . . . 205
1d) . . . 205
1e) . . . 206
1f) . . . 206
1g) . . . 206
1h) . . . 206
Question 2: Simulation and Variance Reduction [8] . . . 207
2a) . . . 207
2b) . . . 207
2c) . . . 207
2d) . . . 207
2e) . . . 208
2f) . . . 208
2g) . . . 208
2h) . . . 208
Question 3: Interpreting Simulation [6] . . . 209
3a) . . . 209
3b) . . . 209
3c) . . . 209
3d) . . . 209
3e) . . . 209
3f) . . . 209
Question 4: Simulating Daily Demand [8] . . . 210
4a) Estimating the shortage probability [4] . . . 210
4b) Variation between runs [4] . . . 210

II Exam **211**

Exam 1 **212**
Question 1: Multiple Choice [54] . . . 213
1a) . . . 213
1b) . . . 213
1c) . . . 213
1d) . . . 214

1e) . . . 214
1f) . . . 214
1g) . . . 215
1h) . . . 215
1i) . . . 215
Question 2: True or False [30] . . . 216
2a) . . . 216
2b) . . . 216
2c) . . . 216
2d) . . . 216
2e) . . . 217
2f) . . . 217
Question 3: Conceptual [16] . . . 218
3a) R and numerical computation [10] . . . 218
3b) Optimization [6] . . . 219
Answer Key and Grading Rubric . . . 220
Question 1: Multiple Choice [54] . . . 220
Question 2: True or False [30] . . . 220
Question 3: Conceptual [16] . . . 220

Past Exam 1 **221**
Exam Instructions . . . 221
Question 1: Multiple Choice [18] . . . 221
1a) [2] . . . 222
1b) [2] . . . 222
1c) [2] . . . 223
1d) [2] . . . 223
1e) [2] . . . 224
1f) [2] . . . 224
1g) [2] . . . 224
1h) [2] . . . 225
1i) [2] . . . 225
Question 2: Conceptual [20] . . . 226
2a) [4] . . . 226
2b) [6] . . . 226
2c) [6] . . . 226
2d) [4] . . . 226
Question 3: Calculation [30] . . . 227
3a) [6] . . . 227
3b) [8] . . . 228
3c) [6] . . . 228
3d) [10] . . . 229
Question 4: Algorithms [16] . . . 229
4a) [8] . . . 230
4b) [8] . . . 230
Question 5: Convergence [16] . . . 231
5a) [6] . . . 231
5b) [10] . . . 231

III Project **232**

Instruction **233**
Group Members . . . 233

Project Overview . . . 233
Report writing model . . . 233

Report **235**
What is the project? . . . 235
Groups and topic approval . . . 236
Examples of a manageable project . . . 236
Report requirements . . . 237
Writing model: The R Journal . . . 237
Suggested report structure . . . 237
Download and adapt the template . . . 238
Report grading [10] . . . 238

Package **240**
What your group should implement . . . 240
Required package contents . . . 240
R and Python requirements . . . 241
Package grading [10] . . . 242
Development resources . . . 243

Presentation **244**
What to present . . . 244
Oral presentation format . . . 245
Presentation grading [10] . . . 245

Preface

Description

Topics included are optimization, numerical integration, bootstrapping, cross-validation and Jackknife, density estimation, smoothing, and use of the statistical computer package of S-plus/R.

Prerequisites

MATH 4752/6752 – Mathematical Statistics II, and the ability to program in a high-level language.

Instructor

Chi-Kuang Yeh, I am an Assistant Professor in the Department of Mathematics and Statistics, Georgia State University.

- Office: Suite 1407, 25 Park Place.
- Email: cyeh@gsu.edu.

Office Hour

Wednesday 9:00 - 12:00 or by appointment

Grade Distribution

- Assignments: 40%
- Exam: 30%
- Project: 30%
- Attendance: *7%

Assignment

- ☒ Assignment 1: September 11, 2026
- ☒ Assignment 2: October 2, 2026
- ☐ Assignment 3: Due date TBA. PDF | Response template (.qmd)

Midterm

- ☒ Exam 1: October 7, 2026
- ☐ Exam 2: November 16, 2026

Project

Start with Instruction for a quick overview and a few topic ideas.

Work in groups of **1-3 students** on a statistical method implemented in **R or Python**. Select from the instructor's topic list or propose a topic for instructor approval.

Package option: You may submit either an **R package or a Python package**. Python users should follow the Python package requirements; both languages use the same grading rubric.

- ☐ Report: Due on December 13, 2026 (10%)
- ☐ Package: Due on December 13, 2026 (10%)
- ☐ Presentation: TBA (10%)

Topics and Corresponding Lectures

Those chapters are based on the lecture notes. This part will be updated frequently.

Status	Chapter	Topic	Lecture
Done	Ch. 1	R Programming	1–3
Done	Ch. 2	Intro to Computational Statistics	4–5
Done	Ch. 3	Optimization	6–10
In progress	Ch. 4	Generating Random Variable	11–
Ready	Ch. 5	Monte Carlo Simulation and Variance Reduction	TBA
Done		Exam 1	13
Exam		Exam 2	24

Recommended Textbooks

- Givens, G.H. and Hoeting, J.A. (2012). *Computational Statistics*. Wiley, New York.
- Rizzo, M.L. (2007) *Statistical Computing with R*. CRC Press, Roca Baton.
- Hothorn, T. and Everitt, B.S. (2006). *A Handbook of Statistical Analyses Using R*. CRC Press, Boca Raton.

Side Readings

- Wickham, H., Çetinkaya-Rundel, M. and Grolemund, G. (2023). *R for Data Science*. O'Reilly.

Section 1

Introduction to R

Learning objectives

By the end of this chapter, you should be able to:

1. Describe how R reads, evaluates, and prints an expression.
2. Distinguish an object from the name bound to that object.
3. Identify the major data types and data structures in R.
4. Predict how vectorization, coercion, recycling, and missing values affect a computation.
5. Create, inspect, index, and modify vectors, matrices, arrays, lists, and data frames.
6. Write and use simple R functions.
7. Choose appropriately among vectorization, loops, and the `apply()` family.
8. Use the tidyverse pipe and core `dplyr` verbs to express a data-analysis pipeline.
9. Write computations that are reproducible, readable, and numerically responsible.

Section 1.1 Why R?

Modern statistical analyses routinely involve hundreds, thousands, or millions of observations. The calculations are too numerous, and often too complex, to perform reliably by hand. We therefore need software that can manage data, implement statistical methods, automate repeated computations, and document the complete analysis.

R is both a high-level programming language and an environment for statistical computing and graphics. It is especially useful for statistics because it is:

- **free and open source**, so students and researchers can use the same tools;
- **designed for data analysis**, with vectors, missing values, statistical models, and graphics built into the language;
- **extensible**, with a large ecosystem of packages developed by researchers and software developers;
- **reproducible**, because an analysis can be saved as code and rerun from beginning to end;
- **interoperable**, with interfaces to databases and languages such as C++, Python, and Julia.

R is not a menu-driven statistics program. Learning R means learning to express an analysis as a sequence of precise and reproducible computations.

> **i** R, RStudio, and Quarto are different tools
>
> **R** performs the computation. **RStudio** is an integrated development environment that helps us write and run code. **Quarto** combines text, code, results, figures, equations, and references into reproducible HTML, PDF, Word, or presentation output.

Section 1.2 The working environment

There are different integrated development environment to use R, some of them are Rstudio, VS Code and Positron.

Rstudio

RStudio is an integrated development environment (IDE) designed specifically for working with the R programming language. It provides a user-friendly interface that includes a source editor, console, environment pane, and tools for plotting, debugging, version control, and package management. RStudio supports both R and Python and is widely used for data analysis, statistical modeling, and reproducible research. It also integrates seamlessly with tools like R Markdown, Shiny, and Quarto, making it popular among data scientists, statisticians, and educators.

Visual Studio Code (VS Code)

VS Code is a versatile code editor that supports multiple programming languages, including R. With the R extension for VS Code, users can write and execute R code, access R's console, and utilize features like syntax highlighting, code completion, and debugging. While not as specialized as RStudio for R development, VS Code offers a lightweight alternative with extensive customization options and support for various programming tasks.

Positron

Positron IDE is the next-generation integrated development environment developed by Posit, the company behind RStudio. Designed to be a modern, extensible, and language-agnostic IDE, Positron builds on the strengths of RStudio while supporting a broader range of languages and workflows, including R, Python, and Quarto.

1.2.1 RStudio

RStudio is an integrated development environment (IDE) for R, Python, and reproducible publishing. Its default layout contains four panes:

Pane	Main purpose
Source	Write and edit scripts and Quarto documents.
Console	Send expressions directly to R and view immediate results.
Environment/History	Inspect objects and review previous commands.

Files/Plots/Packages/Help/Viewer	Navigate files, inspect graphics, manage packages, read documentation, and preview output.

Use the console for short experiments. Any computation that matters should be placed in an R script or Quarto document so that it can be reproduced.

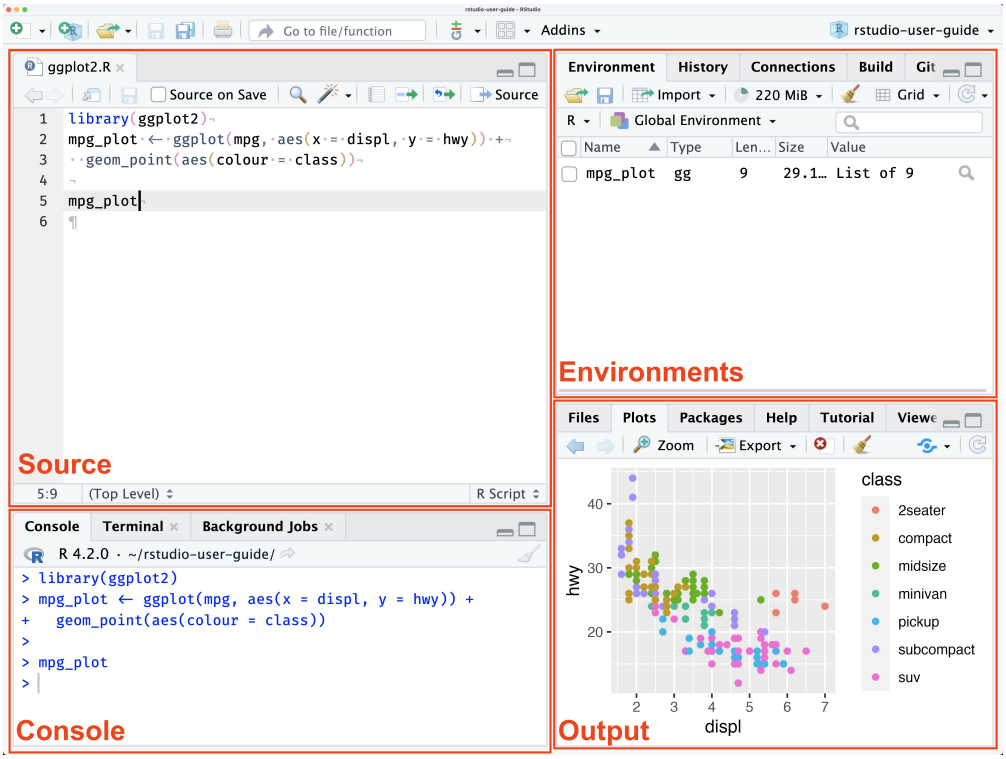


Figure 1.1: RStudio layout borrow from the Posit website.

1.2.2 R scripts and Quarto documents

An R script is a plain-text file with the extension `.R`. It contains R expressions that can be executed one line, one selection, or one file at a time.

A Quarto document has the extension `.qmd`. It combines:

- a YAML header describing the document;
- prose written in Markdown;
- executable code chunks;
- tables, equations, figures, citations, and cross-references.

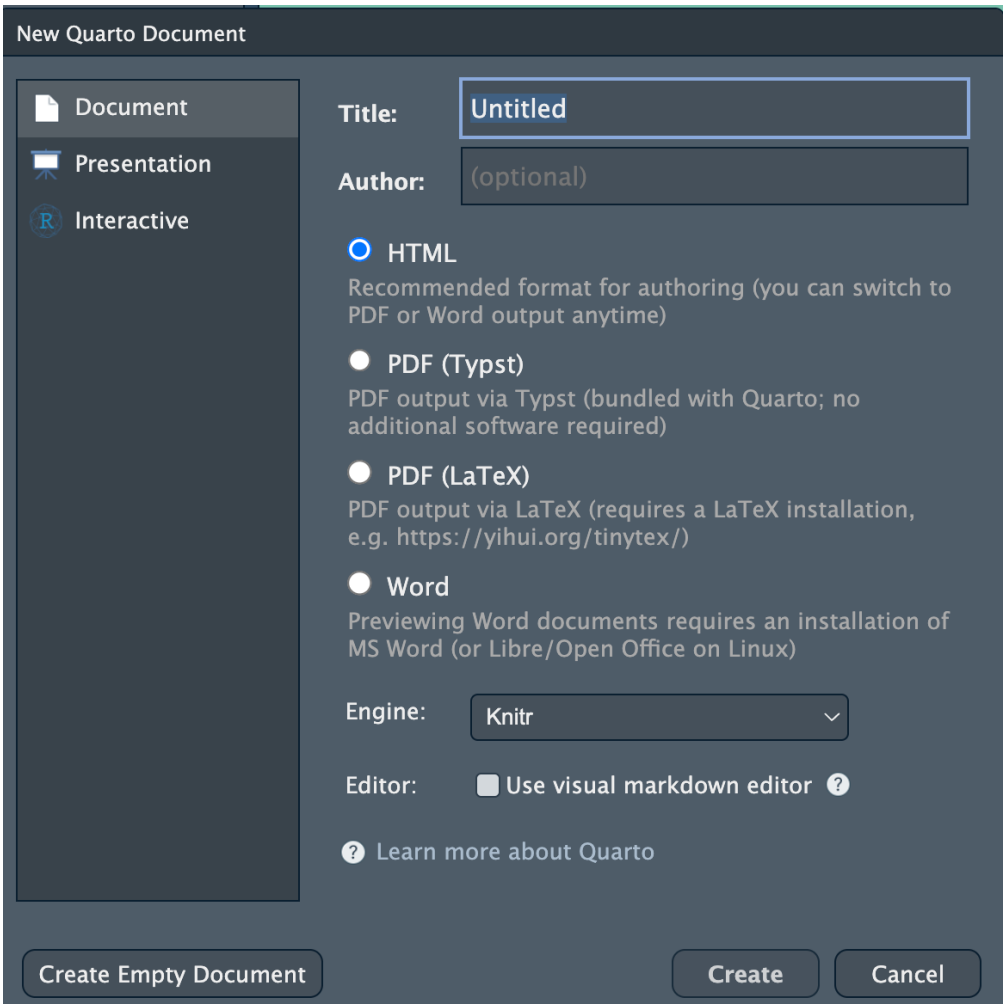


Figure 1.2: Quarto.

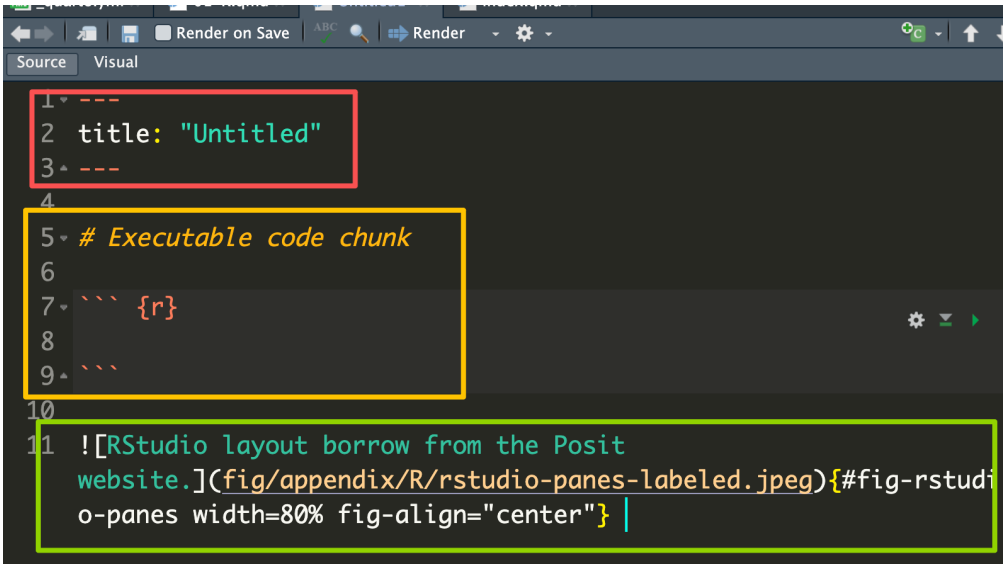


Figure 1.3: Quarto 2.

In Figure 1.3, the red box is the yaml, the yellow box is the executable code chunk and the green box is a figure.

For example, an executable R chunk is written as

````
```{r}
x <- rnorm(100)
mean(x)
```
````

```
[1] -0.1001276
```

Rendering the document executes the enabled chunks in order and places their results into the final output.

1.2.3 A project-oriented workflow

Create one RStudio Project for the course or for each substantial analysis. Use project-relative paths such as `data/survey.csv` rather than hard-coded paths tied to one computer.

A simple project might contain:

```
stat8670-project/
├── data/          # original and processed data
├── R/             # reusable functions
├── figures/       # saved figures
├── notes/         # Quarto lecture or analysis files
└── stat8670-project.Rproj
```

Avoid using `setwd()` inside a reproducible document. When a project is opened, its root provides a stable reference point for relative paths.

Section 1.3 R as a computing environment

At the console, R follows a **read, evaluate, print loop** (REPL):

1. **Read:** parse the text entered by the user.
2. **Evaluate:** compute the value of the expression.
3. **Print:** display the result when appropriate.
4. **Loop:** wait for the next expression.

```
2 + 3 * 4
```

```
[1] 14
```

R respects operator precedence, so multiplication is performed before addition.

```
(2 + 3) * 4
```

```
[1] 20
```

Assignment usually returns its value invisibly:

```
x <- 2 + 3 * 4
x
```

```
[1] 14
```

> A useful mental model
>
> Most R code can be understood as **functions operating on objects**. Even operators such as `+` are functions.

```
`+`(2, 3)
```

```
[1] 5
```

Section 1.4 Objects, data, and functions

Two fundamental kinds of objects that we will work with are:

1. **Data objects**
2. **Functions**

Data objects include numbers, vectors, matrices, arrays, lists, data frames, and fitted models.

A **function** is a set of instructions that receives input, performs a computation, and returns an output. Functions can be built into R, provided by a package, or written by the user.

```
mean(c(1, 2, 3, 4))
```

```
[1] 2.5
```

Here, `c(1, 2, 3, 4)` is a data object and `mean()` is a function.

Section 1.5 Expressions, objects, and names

R computes with **objects**. A **name** is a symbol bound to an object.

```
sample_size <- 100
another_name <- sample_size

sample_size
```

```
[1] 100
```

```
another_name
```

```
[1] 100
```

The assignment operator `<-` creates or changes a binding.

Names are case-sensitive:

```
n <- 10
N <- 25

c(n = n, N = N)
```

```
 n  N
10 25
```

A useful naming convention is to separate words using `_`:

```
sample_mean <- 10
number_students <- 30
```

Use `ls()` to list names in the current environment and `rm()` to remove a binding.

```
temporary_result <- 42
"temporary_result" %in% ls()
```

```
[1] TRUE
```

```
rm(temporary_result)

"temporary_result" %in% ls()
```

```
[1] FALSE
```

Section 1.6 Data types

The fundamental data structure in R is the **vector**. An atomic vector contains elements of one common type.

Common atomic types include:

Type	Example	Typical use
logical	`TRUE`, `FALSE`	conditions and indicators
integer	`2L`	counts and indices
double	`2`, `3.14`	numerical computation
character	`"R"`	text and labels
complex	`1 + 2i`	complex arithmetic
raw	`charToRaw("R")`	bytes

```
logical_vector <- c(TRUE, FALSE, TRUE)
integer_vector <- c(1L, 2L, 3L)
double_vector <- c(1, 2, 3)
character_vector <- c("one", "two", "three")

typeof(logical_vector)
```

```
[1] "logical"
```

```
typeof(integer_vector)
```

```
[1] "integer"
```

```
typeof(double_vector)
```

```
[1] "double"
```

```
typeof(character_vector)
```

```
[1] "character"
```

1.6.1 Changing data types

Data types can be explicitly converted, but conversion may result in information loss.

```
x <- pi
x
```

```
[1] 3.141593
```

```
x_int <- as.integer(x)
x_int
```

```
[1] 3
```

Here, converting π to an integer removes its decimal component.

Common conversion functions include:

- `as.integer()`
- `as.numeric()`
- `as.character()`
- `as.logical()`
- `as.Date()`
- `as.factor()`
- `as.list()`
- `as.matrix()`
- `as.data.frame()`
- `as.vector()`
- `as.complex()`

> ⚠ Type conversion can lose information
>
> Always inspect the result after converting an object from one type to another.

Section 1.7 Operators

A **unary operator** operates on one object:

```
-x
!x
```

A **binary operator** operates on two objects:

```
x + y
x - y
x * y
x / y
```

Comparison operators

Comparison operators return logical values:

```
x == y
x != y
x < y
x > y
x <= y
x >= y
```

Logical operators

Logical operators combine logical values:

```
x & y
x | y
!x
```

The operators `&` and `|` operate element by element.

```
x <- c(TRUE, FALSE, FALSE)
y <- c(TRUE, TRUE, FALSE)

x | y
```

```
[1]  TRUE  TRUE FALSE
```

```
x & y
```

```
[1]  TRUE FALSE FALSE
```

By contrast, `&&` and `||` are intended for a single logical condition and are commonly used inside `if` statements.

Section 1.8 Vectorization

Many R functions and operators act on entire vectors.

```
x <- c(1, 2, 3, 4)

x^2
```

```
[1]  1  4  9 16
```

```
sqrt(x)
```

```
[1] 1.000000 1.414214 1.732051 2.000000
```

```
sum(x)
```

```
[1] 10
```

```
mean(x)
```

```
[1] 2.5
```

Vectorized code often expresses statistical calculations more directly than element-by-element code.

Section 1.9 Recycling

Arithmetic between vectors is usually element-wise.

```
x <- c(10, 20, 30)
y <- c(1, 2, 3)

x + y
```

```
[1] 11 22 33
```

If one vector is shorter, R may **recycle** its values.

```
c(10, 20, 30, 40) + c(1, 2)
```

```
[1] 11 22 31 42
```

When the shorter length does not divide the longer length exactly, R produces a warning.

```
c(10, 20, 30, 40) + c(1, 2, 3)
```

```
Warning in c(10, 20, 30, 40) + c(1, 2, 3): longer object length is not a
multiple of shorter object length
```

```
[1] 11 22 33 41
```

Section 1.10 Coercion

An atomic vector must have one type. Combining different types may cause R to convert values to a common type.

```
mixed_numeric <- c(TRUE, 2L, 3.5)
mixed_character <- c(TRUE, 2L, 3.5, "four")

mixed_numeric
```

```
[1] 1.0 2.0 3.5
```

```
typeof(mixed_numeric)
```

```
[1] "double"
```

```
mixed_character
```

```
[1] "TRUE" "2"    "3.5"  "four"
```

```
typeof(mixed_character)
```

```
[1] "character"
```

A simplified coercion hierarchy is:

```
logical → integer → double → complex → character
```

Inspect unfamiliar objects using

```
typeof()
class()
length()
str()
```

Section 1.11 Missing and special values

R uses `NA` to represent a missing value.

```
day <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday")
weather <- c("Raining", "Sunny", NA, "Windy", "Snowing")

weather_data <- data.frame(day, weather)
weather_data
```

```
        day weather
1    Monday Raining
2   Tuesday   Sunny
3 Wednesday    <NA>
4  Thursday   Windy
5    Friday Snowing
```

Missing values usually propagate through calculations:

```
scores <- c(88, NA, 91, 84)

mean(scores)
```

```
[1] NA
```

```
mean(scores, na.rm = TRUE)
```

```
[1] 87.66667
```

```
is.na(scores)
```

```
[1] FALSE  TRUE FALSE FALSE
```

Do not use

```
x == NA
```

to detect missing values. Use

```
is.na(x)
```

instead.

Other special numerical values include

```
c(1 / 0, -1 / 0, 0 / 0)
```

```
[1]  Inf -Inf  NaN
```

which produce `Inf`, `-Inf`, and `NaN`.

Section 1.12 Indexing and subsetting

Indexing allows us to access or modify parts of an object.

> **!** R indexing begins at 1
>
> Unlike languages such as Python and C++, R uses **1-based indexing**. The first element is indexed by `1`, not `0`.

For a vector:

```
x <- c(first = 10, second = 20, third = 30, fourth = 40)

x[1]
```

```
first
   10
```

```r
x[c(1, 3)]
```

```
first third
   10    30
```

```r
x[-2]
```

```
 first  third fourth
    10     30     40
```

```r
x[x >= 30]
```

```
 third fourth
    30     40
```

```r
x[c("second", "fourth")]
```

```
second fourth
    20     40
```

Positive integers select positions, negative integers exclude positions, logical values filter positions, and character values select named elements.

> ⚠ A common surprise
>
> Prefer `seq_along(x)` to `1:length(x)` when iterating over the positions of an object.

```r
empty <- numeric(0)

1:length(empty)
```

```
[1] 1 0
```

```r
seq_along(empty)
```

```
integer(0)
```

Section 1.13 Naming elements

Names can be attached to vector elements:

```r
temp <- c(20, 30, 27, 31, 45)

names(temp) <- c("Mon", "Tues", "Wed", "Thurs", "Fri")

temp
```

```
  Mon  Tues   Wed Thurs   Fri
   20    30    27    31    45
```

```
temp["Wed"]
```

```
Wed
 27
```

A vector does not have rows, so this is inappropriate:

```
rownames(temp) <- "Day1"
```

A matrix can have row and column names:

```
temp_mat <- matrix(c(20, 30, 27, 31, 45),
                   nrow = 1,
                   ncol = 5)

colnames(temp_mat) <- c("Mon", "Tues", "Wed", "Thurs", "Fri")
rownames(temp_mat) <- "Day1"

temp_mat
```

```
     Mon Tues Wed Thurs Fri
Day1  20   30  27    31  45
```

Section 1.14 Arrays and matrices

An **array** is a multidimensional data structure.

```
array_1d <- array(1:10, dim = 10)
array_1d
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
array_2d <- array(1:12, dim = c(4, 3))
array_2d
```

```
     [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12
```

```
array_3d <- array(1:24, dim = c(4, 3, 2))
array_3d
```

```
, , 1

     [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12

, , 2

     [,1] [,2] [,3]
[1,]   13   17   21
[2,]   14   18   22
[3,]   15   19   23
[4,]   16   20   24
```

A **matrix** is a two-dimensional array:

```
my_matrix <- matrix(1:12, nrow = 4, ncol = 3)
my_matrix
```

```
     [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12
```

```
is.matrix(array_2d)
```

```
[1] TRUE
```

```
is.matrix(my_matrix)
```

```
[1] TRUE
```

```
is.array(array_2d)
```

```
[1] TRUE
```

```
is.array(my_matrix)
```

```
[1] TRUE
```

A matrix is an atomic vector with a `dim` attribute, so all entries must share one common type.

```
A <- matrix(1:6, nrow = 2, ncol = 3)

attributes(A)
```

```
$dim
[1] 2 3
```

```r
A[2, 3]
```

```
[1] 6
```

Matrix multiplication uses `%*%`:

```r
B <- matrix(c(1, 0, 0, 1, 1, 1), nrow = 3)

A %*% B
```

```
     [,1] [,2]
[1,]    1    9
[2,]    2   12
```

while $*$ performs element-wise multiplication.

Section 1.15 Lists and key-value pairs

A list can contain objects of different types and lengths.

```r
fit_summary <- list(
  method = "least squares",
  coefficients = c(intercept = 1.2, slope = 0.8),
  converged = TRUE
)

str(fit_summary)
```

```
List of 3
 $ method      : chr "least squares"
 $ coefficients: Named num [1:2] 1.2 0.8
  ..- attr(*, "names")= chr [1:2] "intercept" "slope"
 $ converged   : logi TRUE
```

Named list elements naturally form **key-value pairs**.

```r
key1 <- "Tues"
value1 <- 32

key2 <- "Wed"
value2 <- 28

list_temp <- list()

list_temp[[key1]] <- value1
list_temp[[key2]] <- value2

list_temp
```

```
$Tues
[1] 32

$Wed
[1] 28
```

Values can then be obtained from their keys:

```
list_temp[["Tues"]]
```

```
[1] 32
```

```
list_temp$Tues
```

```
[1] 32
```

For lists, note the important distinction:

```
fit_summary["coefficients"]
```

```
$coefficients
intercept     slope
      1.2       0.8
```

```
fit_summary[["coefficients"]]
```

```
intercept     slope
      1.2       0.8
```

`[` returns a sublist, whereas `[[` extracts the element itself.

Section 1.16 Factors

A factor represents categorical data using integer codes and a set of levels.

```
group <- factor(c("control", "treatment", "control"))

group
```

```
[1] control   treatment control
Levels: control treatment
```

```
unclass(group)
```

```
[1] 1 2 1
attr(,"levels")
[1] "control"   "treatment"
```

```
levels(group)
```

```
[1] "control"   "treatment"
```

The order of factor levels matters in statistical models, so define it deliberately when necessary.

Section 1.17 Data frames

A **data frame** is a two-dimensional rectangular data structure.

Conceptually, a data frame is a named list of equal-length vectors:

- each row represents an observational unit;
- each column represents a variable;
- columns may have different data types.

```
students <- data.frame(
  id = 1:6,
  name = c("Alice", "Bob", "Chen", "Divya", "Elena", "Farah"),
  program = c("MS", "PhD", "MS", "PhD", "MS", "PhD"),
  score = c(90, 85, NA, 94, 88, 91),
  completed = c(TRUE, TRUE, FALSE, TRUE, TRUE, TRUE)
)

students
```

```
  id  name program score completed
1  1 Alice      MS    90      TRUE
2  2   Bob     PhD    85      TRUE
3  3  Chen      MS    NA     FALSE
4  4 Divya     PhD    94      TRUE
5  5 Elena      MS    88      TRUE
6  6 Farah     PhD    91      TRUE
```

Another familiar built-in data frame is:

```
iris <- datasets::iris
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

1.17.1 Inspecting a data frame

Always inspect newly imported data before analyzing them.

```
class(students)
```

```
[1] "data.frame"
```

```
dim(students)
```

```
[1] 6 5
```

```
nrow(students)
```

```
[1] 6
```

```
ncol(students)
```

```
[1] 5
```

```
names(students)
```

```
[1] "id"        "name"      "program"   "score"     "completed"
```

```
str(students)
```

```
'data.frame':   6 obs. of  5 variables:
 $ id       : int  1 2 3 4 5 6
 $ name     : chr  "Alice" "Bob" "Chen" "Divya" ...
 $ program  : chr  "MS" "PhD" "MS" "PhD" ...
 $ score    : num  90 85 NA 94 88 91
 $ completed: logi  TRUE TRUE FALSE TRUE TRUE TRUE
```

```
summary(students)
```

```
       id              name        program      score      completed
 Min.   :1.00   Length   :6   Length   :6   Min.   :85.0   Mode :logical
 1st Qu.:2.25   N.unique :6   N.unique :2   1st Qu.:88.0   FALSE:1
 Median :3.50   N.blank  :0   N.blank  :0   Median :90.0   TRUE :5
 Mean   :3.50   Min.nchar:3   Min.nchar:2   Mean   :89.6
 3rd Qu.:4.75   Max.nchar:5   Max.nchar:3   3rd Qu.:91.0
 Max.   :6.00                               Max.   :94.0
                                            NAs    :1
```

```
head(students, 3)
```

```
  id  name program score completed
1  1 Alice      MS    90      TRUE
2  2   Bob     PhD    85      TRUE
3  3  Chen      MS    NA     FALSE
```

`str()` is especially useful because it gives a compact summary of the object's structure and types.

1.17.2 Subsetting a data frame

Use

```
data[rows, columns]
```

for two-dimensional indexing:

```
students[1, ]
```

```
  id  name program score completed
1  1 Alice      MS    90      TRUE
```

```
students[, c("name", "score")]
```

```
   name score
1 Alice    90
2   Bob    85
3  Chen    NA
4 Divya    94
5 Elena    88
6 Farah    91
```

```
students[students$program == "PhD", ]
```

```
  id  name program score completed
2  2   Bob     PhD    85      TRUE
4  4 Divya     PhD    94      TRUE
6  6 Farah     PhD    91      TRUE
```

```
students[1:3, c("name", "score")]
```

```
   name score
1 Alice    90
2   Bob    85
3  Chen    NA
```

```
students[2, "score"]
```

```
[1] 85
```

Three commonly used ways to extract a column are:

```
students["score"]
```

```
  score
1    90
2    85
3    NA
4    94
5    88
6    91
```

```
students[["score"]]
```

```
[1] 90 85 NA 94 88 91
```

```
students$score
```

```
[1] 90 85 NA 94 88 91
```

The first returns a one-column data frame. The latter two return the underlying vector.

1.17.3 Adding and modifying variables

```
students$passed <- students$score >= 70

students$centered_score <-
  students$score - mean(students$score, na.rm = TRUE)

students
```

```
  id  name program score completed passed centered_score
1  1 Alice      MS    90      TRUE   TRUE            0.4
2  2   Bob     PhD    85      TRUE   TRUE           -4.6
3  3  Chen      MS    NA     FALSE     NA             NA
4  4 Divya     PhD    94      TRUE   TRUE            4.4
5  5 Elena      MS    88      TRUE   TRUE           -1.6
6  6 Farah     PhD    91      TRUE   TRUE            1.4
```

Missingness propagates naturally. If a score is unknown, whether the student passed is also unknown.

1.17.4 Importing rectangular data

```
survey <- read.csv(
  "data/survey.csv",
  na.strings = c("", "NA", ".")
)

measurements <- read.table(
  "data/measurements.txt",
  header = TRUE,
```

```
  sep = "\t"
)
```

After importing, check:

```
dim(survey)
names(survey)
str(survey)
head(survey)
colSums(is.na(survey))
```

Data import is part of the analysis

Import settings such as delimiters, missing-value codes, decimal marks, encodings, and inferred variable types can change the data that R sees and therefore affect the final analysis.

Do not automatically treat a data frame as a matrix

`apply()` first converts a data frame to a matrix. A mixed-type data frame can therefore be converted entirely to character values. Use `lapply()`, `vapply()`, or `dplyr::across()` for column-wise operations when types differ.

Section 1.18 Functions

A function packages a computation so that it can be reused.

```
standardize <- function(x, remove_na = FALSE) {
  center <- mean(x, na.rm = remove_na)
  spread <- sd(x, na.rm = remove_na)

  (x - center) / spread
}

standardize(c(2, 4, 6, 8))
```

```
[1] -1.1618950 -0.3872983  0.3872983  1.1618950
```

A function has three main components:

- **formals:** the arguments;
- **body:** the expressions evaluated;
- **environment:** where the function was created.

```
formals(standardize)
```

```
$x


$remove_na
[1] FALSE
```

```
body(standardize)
```

```
{
    center <- mean(x, na.rm = remove_na)
    spread <- sd(x, na.rm = remove_na)
    (x - center)/spread
}
```

```
environment(standardize)
```

```
<environment: R_GlobalEnv>
```

When a function is called, R creates a temporary environment, matches the arguments, evaluates the body, and returns the value of the final expression unless `return()` is called earlier.

1.18.1 Argument matching

Arguments can be matched by name or position.

```
round(x = 3.14159, digits = 3)
```

```
[1] 3.142
```

```
round(3.14159, 3)
```

```
[1] 3.142
```

Named arguments are often clearer in code intended to be read by others.

1.18.2 Lazy evaluation

R evaluates function arguments only when they are needed.

```
choose_one <- function(x, y) {
  x
}

choose_one(
  10,
  stop("This argument is never evaluated")
)
```

```
[1] 10
```

This behavior is called **lazy evaluation**.

Section 1.19 Environments and lexical scoping

An environment stores name-object bindings and has a parent environment.

R uses **lexical scoping**: functions look for nonlocal names in the environment where they were defined.

```
make_power <- function(power) {
  function(x) {
    x^power
  }
}

square <- make_power(2)
cube <- make_power(3)

square(5)
```

```
[1] 25
```

```
cube(5)
```

```
[1] 125
```

A function together with its defining environment is called a **closure**.

Local assignment affects the current function environment:

```
x <- 100

change_locally <- function() {
  x <- 5
  x
}

change_locally()
```

```
[1] 5
```

```
x
```

```
[1] 100
```

Section 1.20 Memory and copy-on-modify

R behaves as if objects are copied when modified.

```r
x <- c(10, 20, 30)
y <- x

y[1] <- 999

x
```

```
[1] 10 20 30
```

```r
y
```

```
[1] 999  20  30
```

Changing `y` does not change `x`.

Repeatedly expanding an object can require many allocations:

```r
result <- numeric(0)

for (i in seq_len(100000)) {
  result <- c(result, i^2)
}
```

Instead, preallocate the required memory:

```r
n <- 100000

result <- numeric(n)

for (i in seq_len(n)) {
  result[i] <- i^2
}

head(result)
```

```
[1]  1  4  9 16 25 36
```

Section 1.21 Iteration: vectorization, loops, and the apply family

An iterative computation can usually be expressed in several ways.

```r
x <- c(2, 4, 6, 8)

# Vectorized
x^2
```

```
[1]  4 16 36 64
```

```r
# Preallocated loop
squares <- numeric(length(x))

for (i in seq_along(x)) {
  squares[i] <- x[i]^2
}

squares
```

```
[1]  4 16 36 64
```

```r
# Apply a function
vapply(
  x,
  function(value) value^2,
  numeric(1)
)
```

```
[1]  4 16 36 64
```

Use vectorization when an existing vectorized function directly expresses the computation. Use a loop when the calculation has evolving state or complex indexing. Use an apply-family function when the same operation is independently applied to several pieces of an object.

> **!** The apply family is not magic vectorization
>
> `apply()` and `lapply()` still perform iteration internally. They are not automatically faster than a well-written preallocated `for` loop. Their main advantage is often clarity and conciseness.

1.21.1 `apply()`

`apply(X, MARGIN, FUN, ...)` applies a function over dimensions of a matrix or array.

- `MARGIN = 1`: rows
- `MARGIN = 2`: columns

```r
X <- matrix(1:12, nrow = 3, byrow = TRUE)

colnames(X) <- c("x1", "x2", "x3", "x4")

X
```

```
     x1 x2 x3 x4
[1,]  1  2  3  4
[2,]  5  6  7  8
[3,]  9 10 11 12
```

```r
apply(X, MARGIN = 1, FUN = mean)
```

```
[1]  2.5  6.5 10.5
```

```
apply(X, MARGIN = 2, FUN = sd)
```

```
x1 x2 x3 x4
 4  4  4  4
```

Additional arguments are passed through `...`:

```
X_missing <- X
X_missing[2, 3] <- NA

apply(X_missing, 2, mean, na.rm = TRUE)
```

```
x1 x2 x3 x4
 5  6  7  8
```

For common operations, specialized functions are preferable:

```
rowMeans(X)
```

```
[1]  2.5  6.5 10.5
```

```
colMeans(X)
```

```
x1 x2 x3 x4
 5  6  7  8
```

```
rowSums(X)
```

```
[1] 10 26 42
```

```
colSums(X)
```

```
x1 x2 x3 x4
15 18 21 24
```

1.21.2 Why `apply()` can be dangerous for data frames

```
mixed_df <- data.frame(
  id = c("A", "B", "C"),
  x = c(10, 20, 30),
  y = c(2, 4, 6)
)

as.matrix(mixed_df)
```

```
     id  x    y
[1,] "A" "10" "2"
[2,] "B" "20" "4"
[3,] "C" "30" "6"
```

```
typeof(as.matrix(mixed_df))
```

```
[1] "character"
```

Because one column is character, the entire matrix becomes character.

A safer approach is:

```
numeric_columns <- vapply(
  mixed_df,
  is.numeric,
  logical(1)
)

vapply(
  mixed_df[numeric_columns],
  mean,
  numeric(1)
)
```

```
 x  y
20  4
```

1.21.3 `lapply()`

`lapply()` applies a function to each element and always returns a list.

```
lapply(students, class)
```

```
$id
[1] "integer"

$name
[1] "character"

$program
[1] "character"

$score
[1] "numeric"

$completed
[1] "logical"

$passed
[1] "logical"
```

```
$centered_score
[1] "numeric"
```

```
lapply(
  students[c("score", "centered_score")],
  mean,
  na.rm = TRUE
)
```

```
$score
[1] 89.6

$centered_score
[1] 5.861978e-15
```

1.21.4 `sapply()` and `vapply()`

`sapply()` attempts to simplify the output.

```
sapply(students, class)
```

```
          id         name      program        score    completed
   "integer"  "character"  "character"    "numeric"    "logical"
      passed centered_score
   "logical"    "numeric"
```

`vapply()` requires the expected output type:

```
vapply(students, typeof, character(1))
```

```
          id         name      program        score    completed
   "integer"  "character"  "character"     "double"    "logical"
      passed centered_score
   "logical"     "double"
```

This makes `vapply()` safer for reusable code.

1.21.5 `tapply()`

`tapply()` splits a vector into groups and applies a function within each group.

```
with(
  students,
  tapply(score, program, mean, na.rm = TRUE)
)
```

```
 MS PhD
 89  90
```

1.21.6 `Map()` and `mapply()`

These functions apply a function to corresponding elements of multiple inputs.

```
Map(
  function(center, spread) rnorm(3, center, spread),
  center = c(0, 10),
  spread = c(1, 2)
)
```

```
[[1]]
[1] -1.745914290 -0.003917581 -1.685942158

[[2]]
[1] 10.58976 11.72398 12.37988
```

1.21.7 Choosing an iteration tool

Goal	Recommended starting point
Element-wise arithmetic	Vectorized function or operator
Standard matrix row/column summaries	`rowSums()`, `colSums()`, `rowMeans()`, `colMeans()`
General matrix row/column operation	`apply()`
Operation on list or data-frame columns	`lapply()`
Known scalar output type	`vapply()`
Grouped operation on one vector	`tapply()`
Data-frame columns in tidyverse	`dplyr::across()`
Complex state-dependent computation	Preallocated `for` loop

Section 1.22 Packages and help

Packages extend R with functions, data, documentation, and statistical methods. The historical package count can be found https://www.datasciencemeta.com/rpackages.

```
# CRAN
install.packages("packageName")

# Bioconductor
BiocManager::install("packageName")

# GitHub
remotes::install_github("username/repository")
```

Load an installed package:

```
library(packageName)
```

or call one function explicitly:

```
packageName::function_name()
```

Useful help tools include:

```
?mean
help("mean")
example(mean)
args(mean)
methods(mean)
```

Section 1.23 The tidyverse

The **tidyverse** is a coordinated collection of R packages for data manipulation, visualization, importing, tidying, and programming.

```
install.packages("tidyverse")
```

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.1     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.3     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.2
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
 ↪  errors
```

Important packages include:

```
library(dplyr)    # data manipulation
library(ggplot2)  # data visualization
library(readr)    # data import
library(tibble)   # tidy data frames
library(tidyr)    # data tidying
library(purrr)    # functional programming
```

Base R and tidyverse tools are complementary rather than competing systems.

1.23.1 Data frames and tibbles

A **tibble** is a modern data frame.

```
students_tbl <- as_tibble(students)

students_tbl
```

```
# A tibble: 6 x 7
     id name  program score completed passed centered_score
  <int> <chr> <chr>   <dbl> <lgl>     <lgl>           <dbl>
1     1 Alice MS         90 TRUE      TRUE            0.400
2     2 Bob   PhD        85 TRUE      TRUE           -4.60
3     3 Chen  MS         NA FALSE     NA             NA
4     4 Divya PhD        94 TRUE      TRUE            4.40
5     5 Elena MS         88 TRUE      TRUE           -1.60
6     6 Farah PhD        91 TRUE      TRUE            1.40
```

```
class(students_tbl)
```

```
[1] "tbl_df"     "tbl"        "data.frame"
```

Some important differences are:

Operation	Base data frame	Tibble
Printing	May print many rows	Compact preview
`x[, "score"]`	May simplify	Remains a tibble
Partial matching	May occur in some contexts	More strict
Row names	Supported	Discouraged

Both are still data frames:

```
is.data.frame(students_tbl)
```

```
[1] TRUE
```

1.23.2 Pipe operators

The native R pipe is

```
|>
```

and the commonly used magrittr/tidyverse pipe is

```
%>%
```

The pipe passes the result of one operation to the next function.

```
set.seed(8670)

x <- rnorm(5)

# Without pipe
print(round(mean(x), 2))
```

```
[1] 0.43
```

```
# Native pipe
x |>
  mean() |>
  round(2) |>
  print()
```

```
[1] 0.43
```

The pipe makes the sequence of operations explicit.

A useful formatting rule is that `|>` should have spaces around it and normally appear at the end of a line.

1.23.3 Core `dplyr` verbs

Verb	Purpose
`select()`	Choose or reorder columns
`filter()`	Retain rows satisfying conditions
`arrange()`	Sort observations
`mutate()`	Create or modify variables
`summarise()`	Compute summaries
`group_by()`	Define groups

Example:

```
students_tbl |>
  filter(completed, !is.na(score)) |>
  select(name, program, score) |>
  arrange(desc(score))
```

```
# A tibble: 5 x 3
  name  program score
  <chr> <chr>   <dbl>
1 Divya PhD        94
2 Farah PhD        91
3 Alice MS         90
4 Elena MS         88
5 Bob   PhD        85
```

Create variables and summarize by group:

```
students_tbl |>
  mutate(
    passed = score >= 70,
    score_z =
      (score - mean(score, na.rm = TRUE)) /
      sd(score, na.rm = TRUE)
  ) |>
  group_by(program) |>
  summarise(
    n = n(),
    n_observed = sum(!is.na(score)),
    mean_score = mean(score, na.rm = TRUE),
    sd_score = sd(score, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 2 x 5
  program     n n_observed mean_score sd_score
  <chr>   <int>      <int>      <dbl>    <dbl>
1 MS          3          2         89     1.41
2 PhD         3          3         90     4.58
```

1.23.4 Applying functions across several columns

Inside `mutate()` or `summarise()`, `across()` applies functions to selected columns.

```
students_tbl |>
  summarise(
    across(
      where(is.numeric),
      list(
        mean = ~ mean(.x, na.rm = TRUE),
        sd = ~ sd(.x, na.rm = TRUE)
      ),
      .names = "{.col}_{.fn}"
    )
  )
```

```
# A tibble: 1 x 6
  id_mean id_sd score_mean score_sd centered_score_mean centered_score_sd
    <dbl> <dbl>      <dbl>    <dbl>               <dbl>             <dbl>
1     3.5  1.87       89.6     3.36            5.86e-15              3.36
```

This provides a tidyverse alternative to column-wise `lapply()` or `vapply()`.

Section 1.24 Base R and tidyverse are complementary

For example, the same grouped mean can be obtained using either style.

```
# Base R
aggregate(
  score ~ program,
  data = students,
  FUN = mean,
  na.rm = TRUE
)
```

```
  program score
1      MS    89
2     PhD    90
```

```
# tidyverse
students_tbl |>
  group_by(program) |>
  summarise(
    mean_score = mean(score, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 2 x 2
  program mean_score
  <chr>        <dbl>
1 MS              89
2 PhD             90
```

The important questions are the same regardless of syntax:

- What object enters the function?
- What object does the function return?
- How are missing values handled?
- Are types and groups preserved?

Section 1.25 Numerical computation and finite precision

Most real numbers cannot be represented exactly using binary floating-point arithmetic.

```
0.1 + 0.2
```

```
[1] 0.3
```

```
(0.1 + 0.2) == 0.3
```

```
[1] FALSE
```

```
all.equal(0.1 + 0.2, 0.3)
```

```
[1] TRUE
```

For computed floating-point quantities, tolerance-based comparisons such as `all.equal()` are usually more appropriate than exact equality.

Numerical algorithms must also consider overflow, underflow, cancellation, conditioning, and convergence.

```
log(dbinom(500, size = 1000, prob = 0.01))
```

```
[1] -Inf
```

```
dbinom(
  500,
  size = 1000,
  prob = 0.01,
  log = TRUE
)
```

```
[1] -1618.143
```

The second computation is numerically safer.

Section 1.26 Randomness and reproducibility

Pseudo-random numbers are generated deterministically from an internal state.

```
set.seed(8670)
rnorm(5)
```

```
[1] -0.4916281  1.1467524  1.5963118 -0.6655463  0.5516107
```

```
set.seed(8670)
rnorm(5)
```

```
[1] -0.4916281  1.1467524  1.5963118 -0.6655463  0.5516107
```

The two results are identical.

Set the seed at a meaningful boundary, usually once near the beginning of a simulation, rather than repeatedly inside the simulation loop.

Record the information necessary to reproduce the analysis:

```
sessionInfo()
```

```
R version 4.6.1 (2026-06-24)
Platform: aarch64-apple-darwin23
Running under: macOS Golden Gate 27.0.1

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib;  LAPACK
 ↪  version 3.12.1

locale:
[1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8

time zone: America/New_York
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
 [1] lubridate_1.9.5 forcats_1.0.1   stringr_1.6.0   dplyr_1.2.1
 [5] purrr_1.2.2     readr_2.2.0     tidyr_1.3.2     tibble_3.3.1
 [9] ggplot2_4.0.3   tidyverse_2.0.0

loaded via a namespace (and not attached):
 [1] gtable_0.3.6       jsonlite_2.0.0     compiler_4.6.1     tidyselect_1.2.1
 [5] scales_1.4.0       yaml_2.3.12        fastmap_1.2.0      R6_2.6.1
 [9] generics_0.1.4     knitr_1.51         pillar_1.11.1      RColorBrewer_1.1-3
[13] tzdb_0.5.0         rlang_1.3.0        utf8_1.2.6         stringi_1.8.9
[17] xfun_0.60          S7_0.2.2           otel_0.2.0         timechange_0.4.0
[21] cli_3.6.6          withr_3.0.3        magrittr_2.0.5     digest_0.6.39
[25] grid_4.6.1         hms_1.1.4          lifecycle_1.0.5    vctrs_0.7.3
[29] evaluate_1.0.5     glue_1.8.1         farver_2.1.2       codetools_0.2-20
[33] rmarkdown_2.32     tools_4.6.1        pkgconfig_2.0.3    htmltools_0.5.9
```

Section 1.27 Debugging and error messages

When an error occurs:

1. Read the entire error message.
2. Reduce the problem to the smallest example that still fails.
3. Inspect objects with `str()`, `typeof()`, `class()`, and `dim()`.
4. Use `traceback()` after an error when necessary.
5. Verify assumptions with functions such as `stopifnot()`.

```r
safe_mean <- function(x) {
  stopifnot(
    is.numeric(x),
    length(x) > 0
  )

  mean(x)
}
```

```
safe_mean(c(2, 4, 6))
```

```
[1] 4
```

Section 1.28 A computational workflow for this course

A reliable statistical computation separates:

1. **Inputs:** data, constants, configuration, and random seeds.
2. **Transformation:** cleaning and constructing analysis variables.
3. **Computation:** algorithms, models, optimization, or simulation.
4. **Validation:** diagnostics, tests, and sensitivity analysis.
5. **Communication:** figures, tables, interpretation, and limitations.

For example:

```
set.seed(8670)

n <- 200

x <- rnorm(n)
y <- 1 + 2 * x + rnorm(n, sd = 0.75)

simulated_data <- data.frame(
  x = x,
  y = y
)

fit <- lm(y ~ x, data = simulated_data)

coef(fit)
```

```
(Intercept)           x
   1.020794    1.991843
```

This script records where the data came from, fixes the random seed, creates an explicit data object, and passes that object to the statistical model.

Section 1.29 In-class exercises

> **Exercise 1: Predict before running**
>
> Predict the value and type of each expression before running the code.

```r
c(TRUE, 2L, 3.5)

c(1, 2, 3, 4) + c(10, 20)

c(1, 2, NA) > 1

TRUE | NA
```

Exercise 2: Subsetting

Using

```r
x <- c(4, NA, 10, 15, 3, NA, 21)
```

write expressions that return:

1. all nonmissing values;
2. values greater than or equal to 10;
3. the positions of values greater than or equal to 10;
4. the mean of the observed values.

Exercise 3: Functions and scope

Without running the code, determine the result of `f(5)` and identify where each occurrence of `a` is found.

```r
a <- 10

f <- function(x) {
  a <- 2

  g <- function(y) {
    a * y
  }

  g(x)
}

f(5)
```

Exercise 4: Preallocation

Write a function that takes a positive integer `n` and returns a vector whose ith element is

$$\frac{(-1)^{i+1}}{i}.$$

Preallocate the result and use `seq_len(n)`.

Exercise 5: Data frames and tibbles

Using `students`, complete each task once with base R and once with `dplyr`:

1. retain students with observed scores;
2. select `name`, `program`, and `score`;
3. create a variable equal to `score - 90`;
4. sort from highest to lowest score;
5. compute the number of observed scores and mean score within each program.

Before running the code, predict whether each result is a vector, data frame, tibble, or grouped tibble.

Exercise 6: Choosing an apply function

For each task, choose among a vectorized function, `apply()`, `lapply()`, `vapply()`, `tapply()`, `dplyr::across()`, or a loop. Justify your choice and then write the code.

1. Compute the mean of every numeric column in `students`.
2. Compute the row sums of a $10,000 \times 50$ numerical matrix.
3. Record the class of every column in a data frame.
4. Compute the mean score within each program.
5. Fit the same model to each of 20 independently generated data sets and retain the fitted-model objects.

Exercise 7: Diagnose coercion

Predict the type and contents of `result`. Explain why `apply()` is inappropriate and rewrite the computation using `vapply()` or `dplyr::across()`.

```
dat <- data.frame(
  subject = c("A", "B", "C"),
  pre = c(10, 12, 9),
  post = c(14, 13, 15)
)

result <- apply(dat, 2, mean)
```

Section 1.30 Takeaways

- R is both a programming language and an environment for statistical computing.
- Objects and functions are the basic building blocks of R programs.
- Atomic vectors are the foundation of R computation.
- R uses **1-based indexing**.
- Vectorization, recycling, coercion, and missing-value propagation are central features of R.
- Matrices and arrays require homogeneous element types, whereas lists can contain heterogeneous objects.

- A data frame is a named list of equal-length columns.
- Tibbles are modern data frames with stricter and more predictable behavior.
- Functions have arguments, a body, and an environment.
- R uses lexical scoping and lazy evaluation.
- The apply family provides convenient abstractions for repeated computation but is not automatically faster than loops.
- `apply()` is designed primarily for matrices and arrays and can cause unwanted coercion when applied to mixed-type data frames.
- Base R and tidyverse tools are complementary.
- The pipe (`%>%` or `|>`) can make a sequence of data transformations easier to read.
- R's copy-on-modify behavior gives objects intuitive semantics, although memory allocation still matters.
- Statistical computing requires attention to floating-point arithmetic, missing values, reproducibility, and numerical stability.
- Understanding R's computational model makes code easier to predict, debug, optimize, and trust.

Section 1.31 Optional challenge

Explain why these two functions may have very different memory and timing behavior even though they return the same values.

```r
grow <- function(n) {
  out <- numeric(0)

  for (i in seq_len(n)) {
    out <- c(out, i^2)
  }

  out
}

preallocate <- function(n) {
  out <- numeric(n)

  for (i in seq_len(n)) {
    out[i] <- i^2
  }

  out
}
```

Design a timing experiment for increasing values of `n`.

Some of the materials are adapted from CMU Stat36-350.

A comprehensive reference for the *tidyverse* tools is R for Data Science.

A comprehensive reference for *ggplot2* is ggplot2: Elegant Graphics for Data Analysis.

Additional introductory material is adapted from Why R?.

Section 2

Intro to Computational Statistics

> "Statistical computing is more than programming. It is the process of using computation to connect statistical theory with practical data analysis. Programming provides the tools, but the main goal is to understand how a statistical method can be implemented, approximated, validated, and interpreted. A good computational solution should therefore be not only correct in code, but also statistically meaningful, numerically stable, and reproducible."

A computational approach is a systematic way to solve a statistical problem when an exact analytical solution is difficult, unavailable, or inefficient. Examples include using Monte Carlo methods to approximate expectations, numerical optimization to estimate parameters, bootstrap methods to approximate sampling distributions, and cross-validation to select tuning parameters. In this course, the emphasis is on understanding the connection

$$\text{Theory} \iff \text{Computation} \iff \text{Practice}.$$

There is a general principle that will be repeated in this chapter that Kenneth Lange calls *optimization transfer* in his 1999 paper. The basic idea applies to the problem of maximizing a function f.

1. Direct optimize the function f.
    - It can be difficult
2. Optimize a surrogate function g that is easier to optimize than f.
3. So here, instead of optimize f, we optimize g.

Note 1: steps 2&3 are repeated until convergence.

Note 2: maximizing f is equivalent to minimizing $-f$.

Note 3: the surrogate function g should be chosen such that it is easier to optimize than f.

For instance, for a linear regression

$$y = X\boldsymbol{\beta} + \varepsilon. \tag{2.1}$$

From regression class, we know that the (ordinary) least-squares estimation (OLE) for $\boldsymbol{\beta}$ is given by $\widehat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top y$. It is convenient as the solution is in the **closed-form**! However, in the most case, the closed-form solutions will not be available.

For GLMs or non-linear regression, we need to do this **iteratively**!

Section 2.1 Theory versus Computation

One confusing aspect of statistical computing is that often there is a disconnect between what is printed in a statistical computing textbook and what should be implemented on the computer.

- In textbooks, simpler to **present solutions as convenient mathematical formulas whenever possible**, in order to communicate basic ideas and to provide some insight.
  - However, directly translating these formulas into computer code is usually not advisable because there are many problematic aspects of computers that are simply not relevant when writing things down on paper.

Some potential issues includ:

1. Memory overflow: The computer has a limited amount of memory, and it is possible to run out of memory when working with large datasets or complex models.
2. Numerical Precision: Sometimes, due to the cut precision of floating-point arithmetic, calculations that are mathematically equivalent can yield different results on a computer.
   - Example 1: round 1/3 to two decimal places, we get 0.33. Then, $3 \cdot (1/3)$ is exactly 1, but $3 \cdot 0.33$ is 0.99.
   - Example 2: $1 - 0.99999999$ is 0.00000001 (=1E-8), but if we round 0.99999999 to two decimal places, we get 1.00, and then $1 - 1.00$ is 0. If we round 0.00000001 to two decimal places, we get 0.00.
   - Example 3: π
3. (Lienar) Dependence: The detection of linear dependence in matrix computations is influenced by machine precision. Since computers operate with finite precision, situations often arise where true linear dependence exists, but the computer cannot distinguish it from independence.
   - Example: Consider the matrix
   $$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$
   The 3rd column is a linear combination of the first two columns (i.e., col3 = col1 + col2). However, due to machine precision limitations, the computer might not recognize this exact linear dependence, leading to numerical instability in computations involving this matrix. With a small distortion, we have
   $$B = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 + 10^{-5} \end{pmatrix}$$

```
A <- matrix(
  c(1, 2, 3,
    4, 5, 6,
    7, 8, 9),
  nrow = 3, ncol = 3, byrow = TRUE)
B <- A
B[3, 3] <- B[3, 3] + 1E-5
```

```
qr(A)$rank
```

```
[1] 2
```

```
qr(B)$rank
```

```
[1] 3
```

Section 2.2 Matrix Inversion

In many statistical analyses, such as linear regression and specify the distribution (such as normal distribution), matrix inversion plays a central role.

Example 2.1 (Normal distribution)
We know that, a normal density with the parameters mean μ and standard deviation σ is

$$f(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left\{-\frac{1}{2\sigma^2}(x-\mu)^2\right\}$$

or we may work on the multivariate normal distribution case which is a bit more involved.

Definition 1: Multivariate normal distribution
$\boldsymbol{X} = (X_1, \dots, X_d)$ is said to be a multivariate normal distribution if and only if it is a linear comibnation of independent and identically distributed standard normals:

$$\boldsymbol{X} = \boldsymbol{C}\boldsymbol{Z} + \mu, \quad \boldsymbol{Z} = (Z_1, \dots, Z_d), \quad Z_i \overset{iid}{\sim} N(0,1).$$

The property of the multivariate normal are:

- mean vector: $E(\boldsymbol{X}) = \mu$
- variance: $Var(\boldsymbol{X}) = \boldsymbol{C}\boldsymbol{Z}\boldsymbol{C}^\top = \boldsymbol{C}var(\boldsymbol{Z})\boldsymbol{C}^\top := \boldsymbol{\Sigma}$

Notation: $\boldsymbol{X} \sim N_d(\mu, \boldsymbol{\Sigma})$.
PDF:

$$f(\boldsymbol{x} \mid \mu, \Sigma) = (2\pi)^{-d/2} \cdot \exp\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})'\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}) - \frac{1}{2}\log|\boldsymbol{\Sigma}|\right\}.$$

Some of the potential ways to do this is to take logarithm of the PDF (Think about why).

Recall the linear regression model . The OLE for $\boldsymbol{\beta}$ is given by $\hat{\boldsymbol{\beta}} = (X^\top X)^{-1}X^\top y$.

We can solve this using the R command

```
beta_hat <- solve(t(X) %*% X) %*% t(X) %*% y
```

where `solve()` is the R function for matrix inversion. However, it is not a desired way (think about why).

A better way is to go back to the formula, and look at

$$X^\top X\beta = X^\top y,$$

and solve this using the R command

```
solve( crossprod(X), crossprod(X, y) )
# this is the same as
# solve(t(X) %*% X, t(X) %*% y)
```

Here, we avoid explicitly calculating the inverse of $X^\top X$. Instead, we use gaussian elimination to solve the system of equations, which is generally more numerically stable and efficient.

2.2.0.1 Speed comparison

```
set.seed(8670)
X <- matrix(rnorm(5000 * 100), 5000, 100)
y <- rnorm(5000)
library(microbenchmark)
microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y)
```

```
Unit: milliseconds
                           expr      min       lq
 solve(t(X) %*% X) %*% t(X) %*% y 28.83505 30.16593
     mean   median       uq      max neval
 31.96782 30.79489 32.63315 111.0151   100
Warning message:
In microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y) :
  less accurate nanosecond times to avoid potential integer overflows
```

```
microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y,
               solve(crossprod(X), crossprod(X, y)))
```

```
Unit: milliseconds
                                expr      min       lq
     solve(t(X) %*% X) %*% t(X) %*% y 28.90135 30.11608
 solve(crossprod(X), crossprod(X, y)) 25.05859 25.27480
     mean   median       uq      max neval
 31.78686 31.38513 32.66482 53.03354   100
 26.15771 25.81678 26.89188 29.12045   100
```

> Take home message 1
>
> The take home here is that the issues arise from the finite precision of computer arithmetic and the limited memory available on computers. When implementing statistical methods on a computer, it is crucial to consider these limitations and choose algorithms and implementations

that are robust to numerical issues.

2.2.1 Multi-collinearity

The above approach may break down when there is any multi-colinearity in the $\boldsymbol{X}$ matrix. For example, we can tack on a column to $\boldsymbol{X}$ that is very similar (but not identical) to the first column of $\boldsymbol{X}$.

```
set.seed(8670)
N <- 3000
K <- 100
y <- rnorm(N)
X <- matrix(rnorm(N * K), N, K)
W <- cbind(X, X[, 1] + rnorm(N, sd = 1E-15))
```

```
solve(crossprod(W), crossprod(W, y))

Error in `solve.default()`:
! system is computationally singular: reciprocal condition number = 1.36748e-32
```

The algorithm does not work because the cross product matrix $W^\top W$ is **singular**. In practice, matrices like these can come up a lot in data analysis and it would be useful to have a way to deal with it automatically.

R takes a different approach to solving for the unknown coefficients in a linear model. R uses the QR decomposition, which is not as fast, but has the added benefit of being able to automatically detect and handle colinear columns in the matrix.

Here, we use the fact that X can be decomposed as $\boldsymbol{X} = QR$, where Q is an orthonormal matrix and R is an upper triangular matrix. Given that, we can rewrite $X^\top X\boldsymbol{\beta} = X^\top y$ as

$$\begin{aligned} R^\top Q^\top QR\boldsymbol{\beta} &= R^\top Q^\top y \\ R^\top IR\boldsymbol{\beta} &= R^\top Q^\top y \\ R^\top R\boldsymbol{\beta} &= R^\top Q^\top y, \end{aligned}$$

this leads to $R\boldsymbol{\beta} = Q^\top y$. Now we can perform the Gaussian elimination to do it. Because R is an upper triangular matrix, the computational speed is much faster. Here, we **avoid to compute the cross product** $X^\top X$, which is numerical unstable if it is not *standardized* properly

We can see in R code that even with our singular matrix W above, the QR decomposition continues without error.

```
Qw <- qr(W)
str(Qw)
```

```
List of 4
 $ qr   : num [1:3000, 1:101] 55.50466 0.00764 -0.00893 -0.00577 0.00469 ...
 $ rank : int 100
 $ qraux: num [1:101] 1.01 1.03 1.02 1.01 1.03 ...
 $ pivot: int [1:101] 1 2 3 4 5 6 7 8 9 10 ...
 - attr(*, "class")= chr "qr"
```

Note that the output of `qr()` computes the rank of W to be 100, not 101 as the last column is collinear to the 1st column. From there, we can get $\hat{\boldsymbol{\beta}}$ if we want using `qr.coef()`,

```
betahat <- qr.coef(Qw, y)
head(betahat, 3)
```

```
[1] -0.001036084 -0.013519395  0.002871807
```

```
tail(betahat, 3)
```

```
[1]  0.025263715 -0.001872629          NA
```

Q: Why there is an `NA`?

2.2.2 Trade-off

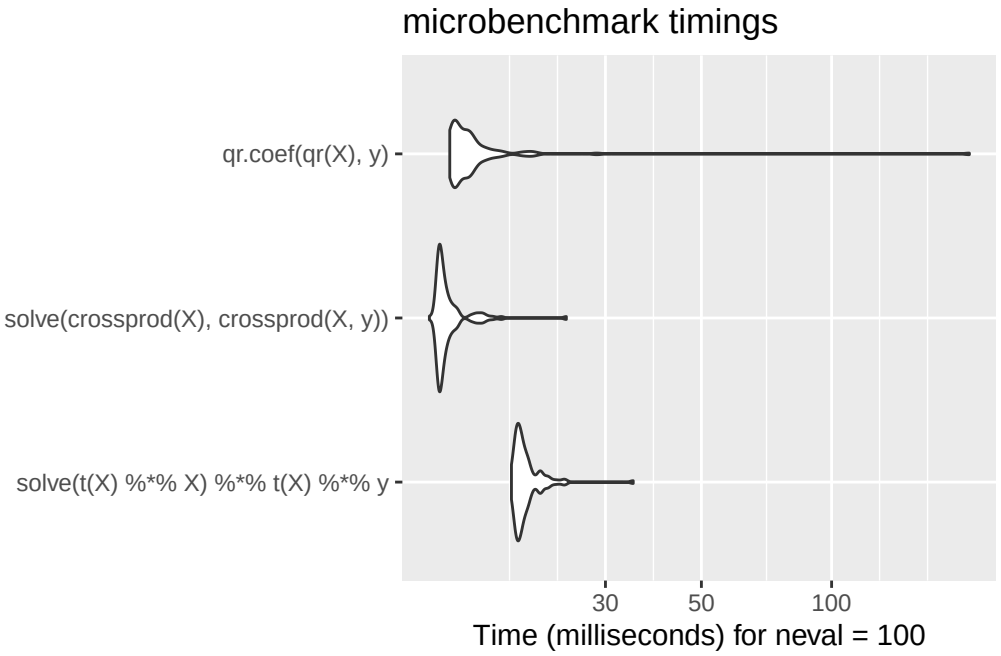
There isn't always elegance and flourish. When we take the robust approach, we accept that it comes at a cost.

```
library(ggplot2)
library(microbenchmark)
m <- microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y,
                    solve(crossprod(X), crossprod(X, y)),
                    qr.coef(qr(X), y))
```

```
Warning in microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y, solve(crossprod(X),
: less accurate nanosecond times to avoid potential integer overflows
```

```
autoplot(m)
```

```
Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
i Please use tidy evaluation idioms with `aes()`.
i See also `vignette("ggplot2-in-packages")` for more information.
i The deprecated feature was likely used in the microbenchmark package.
  Please report the issue at
  <https://github.com/joshuaulrich/microbenchmark/issues/>.
```



Compared with the approaches discussed above, this method performs similarly to the naive approach but is much more stable and reliable.

In practice, we rarely call functions such as `qr()` or `qr.coef()` directly, since higher-level functions like lm() handle these computations automatically. However, in certain specialized and performance-critical settings, it can be advantageous to use alternative matrix decompositions to compute regression coefficients, especially when the computation must be repeated many times in a loop (i.e., *Vectorization*)

Example 2.2 (Multivariate Normal revisit)
Computing the multivariate normal (MVN) density is a common task, for example, when fitting spatial models or Gaussian process models. Because maximum likelihood estimation(MLE) and likelihood ratio tests (LRT) often require evaluating the likelihood many times, efficiency is crucial.
After taking the log of the MVN density, we have

$$\ell(\boldsymbol{x} \mid \boldsymbol{\mu}, \Sigma) := \log\{f(\boldsymbol{x} \mid \boldsymbol{\mu}, \Sigma)\} = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\log|\Sigma| - \frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu}).$$

On the right hand side, the first term is a constant, the second term is linear, and the last term is quadratic, which requires much more computational power.
A Naive Implementation
We first center the data $\boldsymbol{z} := \boldsymbol{x} - \mu$. Then we have $\boldsymbol{z}^\top \Sigma^{-1} \boldsymbol{z}$. This simiplified the question for a bit.
Here, much like the linear regression example above, the key bottleneck is the inversion of the p-dimensional covariance matrix Σ. If we take $\boldsymbol{z}$ to be a $p \times 1$ column vector, then a literal translation of the mathematics into R code might look something like this,

```
t(z) %*% solve(Sigma) %*% z
```

To illustrate, let's simulate some data and compute the quadratic form the naive way:

```
set.seed(8670)

# Generate data
z <- matrix(rnorm(200 * 100), 200, 100)
S <- cov(z)

# Naive quadratic form
quad.naive <- function(z, S) {
  Sinv <- solve(S)
  rowSums((z %*% Sinv) * z)
}

library(dplyr)
quad.naive(z, S) %>% summary()
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  75.28   94.38  101.31  100.55  108.15  120.85
```

A Better Way: Cholesky Decomposition
Because the covariance matrix Σ is symmetric and positive definite, we can exploit its **Cholesky decomposition**. That is, we write $\Sigma = R^\top R$, where R is a upper triangular matrix. Then,

$$\boldsymbol{z}^\top \Sigma^{-1} \boldsymbol{z} = \boldsymbol{z}^\top (R^\top R)^{-1} \boldsymbol{z} = \boldsymbol{z}^\top R^{-1} R^{-\top} \boldsymbol{z} = (R^{-\top} \boldsymbol{z})^\top (R^{-\top} \boldsymbol{z}) := \boldsymbol{v}^\top \boldsymbol{v}.$$

Note that $\boldsymbol{v} \in \mathbb{R}^p$ is the solution to the linear system $R^\top \boldsymbol{v} = \boldsymbol{z}$. Because R is upper triangular, we can solve this system efficiently using back substitution. Also, we can solve this without doing the inversion.
Once we have $\boldsymbol{v}$ we can compute its quadratic form $\boldsymbol{v}^\top \boldsymbol{v}$ by the `crossprod()` function.

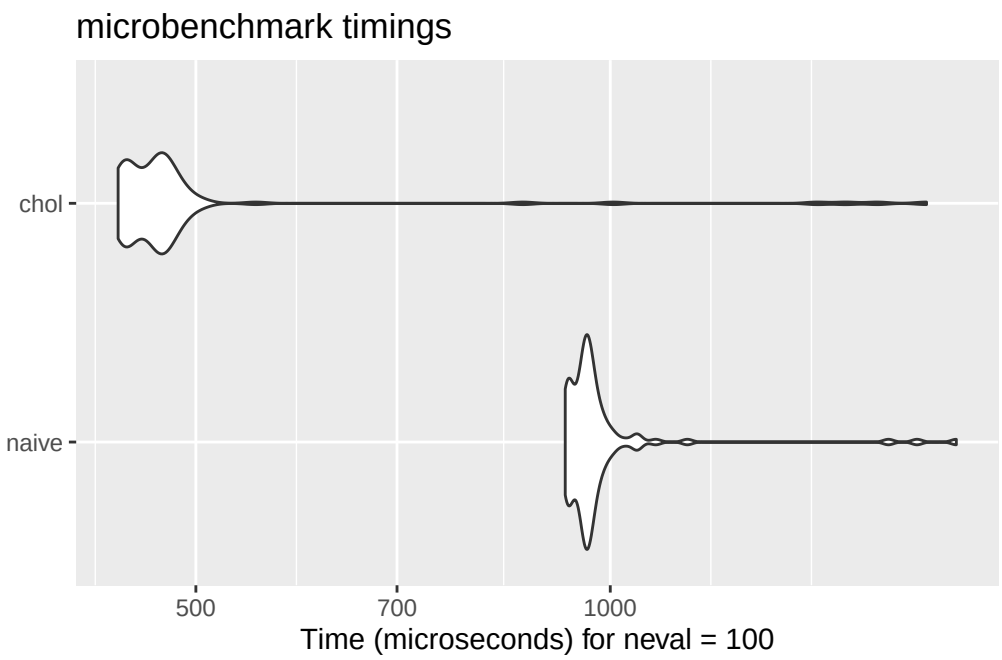
```
quad.chol <- function(z, S) {
  R <- chol(S)
  v <- backsolve(R, t(z), transpose = TRUE)
  colSums(v * v)
}

quad.chol(z, S) %>% summary()
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  75.28   94.38  101.31  100.55  108.15  120.85
```

By product
Another benefit of the Cholesky decomposition is that it gives us a simple way to compute the log-determinant of Σ. The log-determinant of Σ is simply two times the sum of the log of the diagonal elements of R. (Why?)
Performance comparison

```
library(microbenchmark)
library(ggplot2)
m2 <- microbenchmark(
  naive = quad.naive(z, S),
  chol  = quad.chol(z, S)
)
autoplot(m2)
```



Q: Why one is faster than the other?

> Take home message 2
>
> The naive algorithm simply inverts the covariance matrix. The Cholesky-based approach, on the other hand, exploits the fact that covariance matrices are symmetric and positive definite. This results in an implementation that is both faster and numerically more stable—exactly the kind of optimization that makes a difference in real-world statistical computing.

Thus, a knowledge of statistics and numerical analysis can often lead to better algorithms, often invaluable!

Examples are borrowed from the following sources:

- Peng, R.D. Advanced Statistical Computing.

Section 3

Optimization

Optimization plays an important role in statistical computing. Many statistical estimators can be written as

$$\hat{\theta} = \arg\min_{\theta} Q(\theta) \qquad \text{or} \qquad \hat{\theta} = \arg\max_{\theta} \ell(\theta),$$

where $Q(\theta)$ is an objective function and $\ell(\theta)$ may be a likelihood.

For simple problems, a closed-form solution may exist. For many statistical models, however, the solution must be obtained numerically.

Examples include maximum likelihood estimation, nonlinear regression, mixture models, machine learning, smoothing, and regularization.

The central idea is

$$\boxed{\text{Statistical Problem} \iff \text{Optimization Problem} \iff \text{Numerical Algorithm} \iff \text{R Implementation}}.$$

Section 3.1 Why Numerical Optimization?

Consider

$$X_1, \dots, X_n \overset{iid}{\sim} N(\mu, 1).$$

From mathematical statistics,

$$\hat{\mu}_{MLE} = \bar{X}.$$

We already know the closed-form solution. However, we can also obtain the estimator computationally by minimizing the negative log-likelihood.

```r
set.seed(8670)

x <- rnorm(
  n = 50,
  mean = 2,
  sd = 1
)

nll <- function(mu) {
  -sum(
    dnorm(
      x,
      mean = mu,
      sd = 1,
      log = TRUE
    )
  )
}

fit <- optimize(
  nll,
  interval = c(-1, 5)
)

c(
  numerical = fit$minimum,
  theoretical = mean(x)
)
```

```
  numerical theoretical
    1.89689     1.89689
```

Visualizing the Objective Function

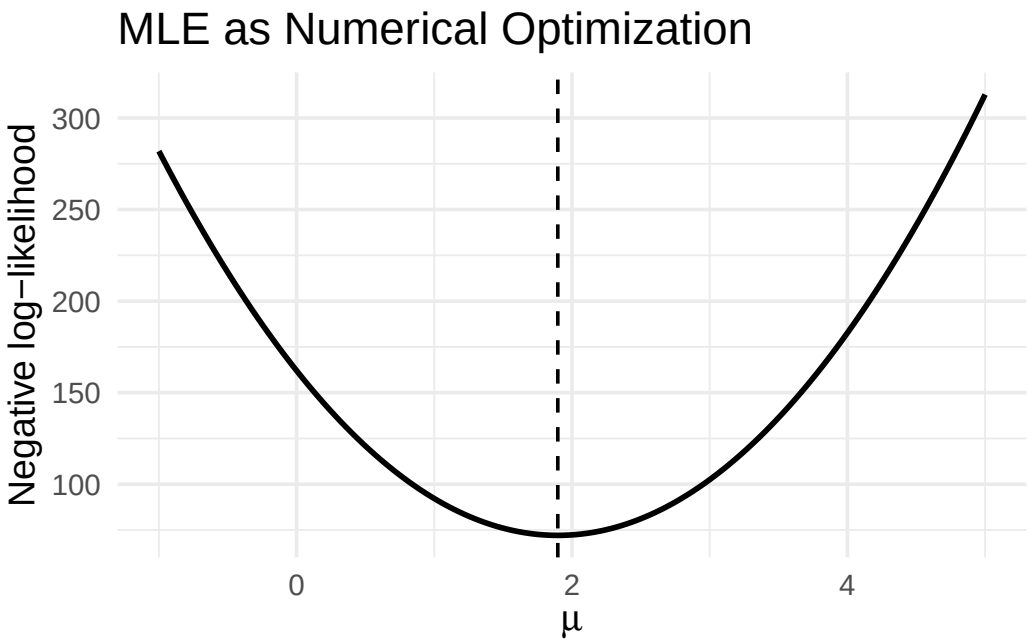
Rather than treating an optimizer as a black box, we can evaluate the objective function over a grid.

```r
mu_grid <- seq(
  -1,
  5,
  length.out = 400
)

nll_data <- data.frame(
  mu = mu_grid,
  nll = vapply(
    mu_grid,
    nll,
    numeric(1)
  )
)

ggplot(
  nll_data,
```

```
  aes(
    x = mu,
    y = nll
  )
) +
  geom_line(linewidth = 1) +
  geom_vline(
    xintercept = mean(x),
    linetype = "dashed"
  ) +
  labs(
    x = expression(mu),
    y = "Negative log-likelihood",
    title = "MLE as Numerical Optimization"
  )
```



Question 1: In-Class Question

Why is numerical optimization useful here if we already know that

$$\hat{\mu}_{MLE} = \bar{X}?$$

The closed-form solution gives us a way to **validate the numerical algorithm**.

When a closed-form solution is unavailable, numerical optimization becomes essential.

Section 3.2 When There Is No Closed-Form Solution

Consider the nonlinear regression model

$$Y_i = \exp(\theta X_i) + \varepsilon_i.$$

Suppose we estimate θ by minimizing

$$Q(\theta) = \sum_{i=1}^{n} \left[Y_i - \exp(\theta X_i) \right]^2 .$$

Unlike ordinary linear regression, there is no convenient closed-form least-squares estimator.

```
set.seed(8670)

x <- seq(
  0,
  1,
  length.out = 50
)

y <- exp(1.5 * x) +
  rnorm(
    50,
    sd = 0.15
  )

sse <- function(theta) {
  sum(
    (y - exp(theta * x))^2
  )
}

fit_nonlinear <- optimize(
  sse,
  interval = c(0, 3)
)

fit_nonlinear$minimum
```

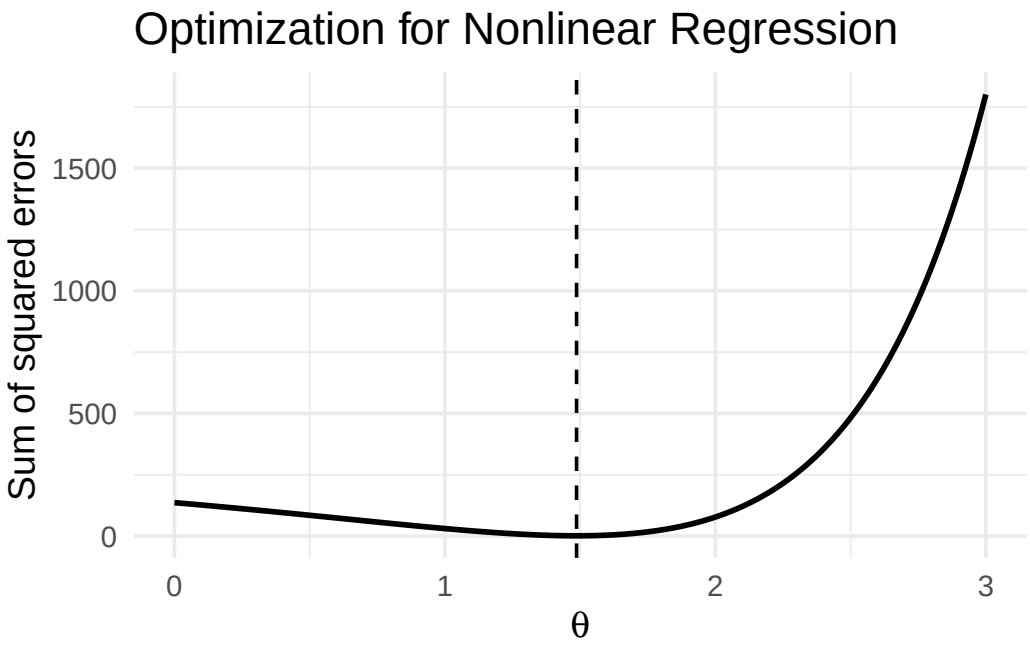
```
[1] 1.486825
```

```
theta_grid <- seq(
  0,
  3,
  length.out = 400
)

sse_data <- data.frame(
  theta = theta_grid,
  SSE = vapply(
    theta_grid,
    sse,
    numeric(1)
  )
```

```
)

ggplot(
  sse_data,
  aes(
    x = theta,
    y = SSE
  )
) +
  geom_line(linewidth = 1) +
  geom_vline(
    xintercept = fit_nonlinear$minimum,
    linetype = "dashed"
  ) +
  labs(
    x = expression(theta),
    y = "Sum of squared errors",
    title = "Optimization for Nonlinear Regression"
  )
```



The important distinction is

Theory tells us what to optimize; computation tells us how to find it.

Section 3.3 Two Families of Optimization Methods

Computational optimization methods can be broadly divided into two families:

Deterministic Methods and Metaheuristic Methods.

3.3.1 Deterministic Methods

A deterministic algorithm follows a predictable sequence of updates once the starting value is fixed.

Examples include:

- Newton's method;
- gradient descent;
- quasi-Newton methods such as BFGS;
- deterministic root-finding methods.

Advantages:

- usually efficient for smooth and well-behaved problems;
- mathematically well understood;
- convergence can often be studied theoretically;
- reproducible for a fixed starting value.

Limitations:

- may converge to a local rather than global optimum;
- can be sensitive to the starting value;
- some methods require derivatives;
- poor scaling can substantially slow convergence.

3.3.2 Metaheuristic Methods

Metaheuristic methods introduce randomness or population-based search to explore the parameter space more broadly.

Examples include:

- simulated annealing;
- genetic algorithms;
- particle swarm optimization.

Advantages:

- can escape local optima;
- usually do not require derivatives;
- useful for multimodal or irregular objectives;
- can be applied to complicated search spaces.

Limitations:

- often computationally expensive;
- results can vary between runs;
- tuning parameters can strongly affect performance;
- global optimality is generally not guaranteed.

The basic trade-off is

Deterministic efficient local search	$\Longleftrightarrow$	Metaheuristic broader stochastic search.

Section 3.4 Local and Global Optima

Consider

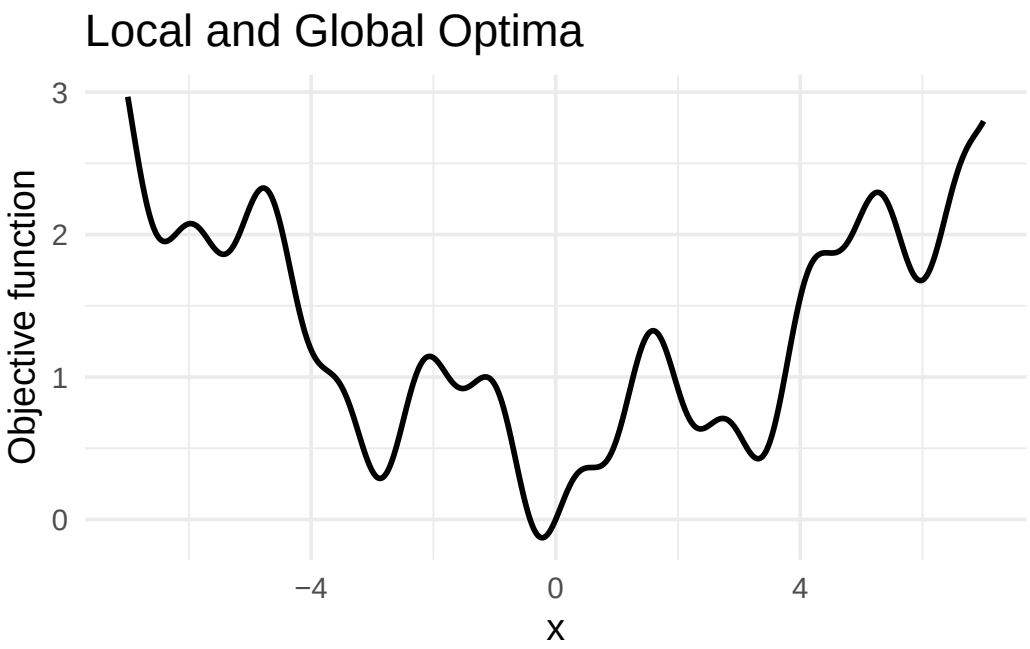
$$Q(x) = 0.05x^2 + \sin^2(x) + 0.2\sin(5x).$$

```
objective <- function(x) {
  0.05 * x^2 +
    sin(x)^2 +
    0.2 * sin(5 * x)
}

x_grid <- seq(
  -7,
  7,
  length.out = 1000
)

objective_data <- data.frame(
  x = x_grid,
  objective = objective(x_grid)
)

ggplot(
  objective_data,
  aes(
    x = x,
    y = objective
  )
) +
  geom_line(linewidth = 1) +
  labs(
    x = "x",
    y = "Objective function",
    title = "Local and Global Optima"
  )
```



The objective contains several local minima.

A deterministic local optimizer may converge to different solutions depending on its starting value.

```
starts <- c(
  -6,
  -2,
  2,
  6
)

solutions <- vapply(
  starts,
  function(x0) {
    optim(
      par = x0,
      fn = objective,
      method = "BFGS"
    )$par
  },
  numeric(1)
)

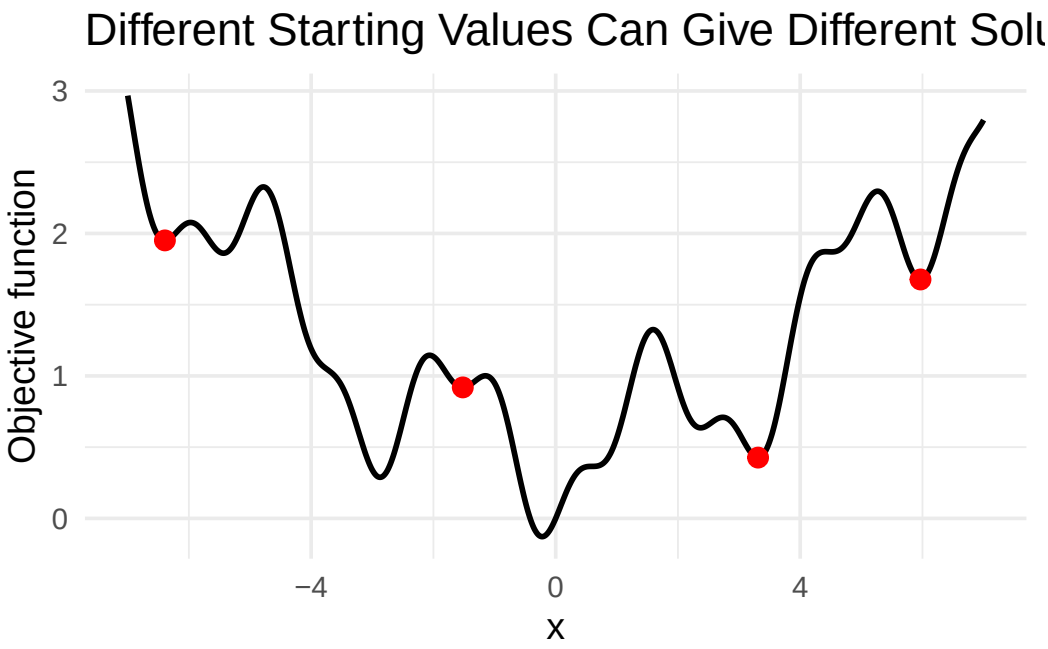
local_results <- data.frame(
  start = starts,
  solution = solutions,
  objective = objective(solutions)
)

local_results
```

```
  start  solution objective
1    -6 -6.392105 1.9511520
2    -2 -1.519268 0.9193572
```

```
3     2  3.310780 0.4266970
4     6  5.967669 1.6769489
```

```
ggplot(
  objective_data,
  aes(
    x = x,
    y = objective
  )
) +
  geom_line(linewidth = 1) +
  geom_point(
    data = local_results,
    col = "red",
    aes(
      x = solution,
      y = objective
    ),
    size = 3
  ) +
  labs(
    x = "x",
    y = "Objective function",
    title = "Different Starting Values Can Give Different Solutions"
  )
```



Question 2: In-Class Question

If the algorithm is deterministic, why can different starting values still produce different answers?

The important distinction is

Local Optimum $\neq$ Global Optimum.

Section 3.5 Deterministic Numerical Methods

We first study several deterministic numerical methods.

3.5.1 Root Finding

Root finding solves

$$f(x) = 0.$$

Why is root finding relevant to optimization?

If $Q(x)$ is differentiable and x^* is an interior optimum, then typically

$$Q'(x^*) = 0.$$

Therefore, optimization can sometimes be converted into

$$f(x) = Q'(x) = 0.$$

A root of $Q'(x)$ is only a candidate optimum. It may correspond to a minimum, maximum, or another stationary point.

Example 1: Normal Quantiles

Suppose we want the 97.5th percentile of a standard normal distribution.
We seek z such that

$$P(Z \leq z) = 0.975.$$

Equivalently,

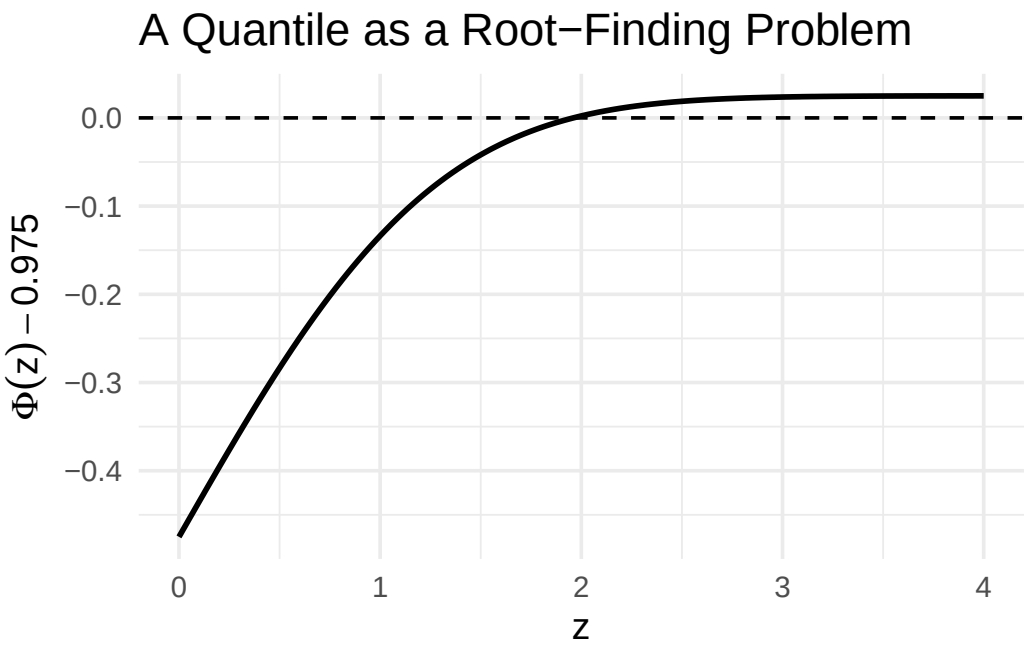
$$\Phi(z) - 0.975 = 0.$$

```r
f_root <- function(z) {
  pnorm(z) - 0.975
}

z_grid <- seq(
  0,
  4,
  length.out = 400
)

root_data <- data.frame(
  z = z_grid,
  f = f_root(z_grid)
)

ggplot(
  root_data,
  aes(
    x = z,
    y = f
  )
) +
  geom_line(linewidth = 1) +
  geom_hline(
    yintercept = 0,
    linetype = "dashed"
  ) +
  labs(
    x = "z",
    y = expression(Phi(z) - 0.975),
    title = "A Quantile as a Root-Finding Problem"
  )
```



R can solve the root directly:

```
root_R <- uniroot(
  f_root,
  interval = c(0, 4)
)$root

c(
  root_finding = root_R,
  qnorm = qnorm(0.975)
)
```

```
root_finding        qnorm
    1.959963     1.959964
```

Thus,

$$\boxed{\text{Quantile Calculation} \iff \text{Root Finding.}}$$

Question 3: In-Class Question

Why do `uniroot()` and `qnorm()` give essentially the same answer?

3.5.2 Bisection Method

The bisection method is similar to a binary search.

Suppose f is continuous and

$$f(a)f(b) < 0.$$

Then there must be at least one root between a and b.

Algorithm 1: Bisection Method

1. Choose a and b such that $f(a)f(b) < 0$.
2. Compute
$$c = \frac{a+b}{2}.$$
3. Keep the half containing the sign change.
4. Repeat until the interval is sufficiently small.

The main intuition is

Each iteration cuts the search interval approximately in half.

```
bisect_trace <- function(
    f,
    a,
    b,
    tol = 1e-6,
    maxit = 100
) {

  if (f(a) * f(b) > 0) {
    stop("f(a) and f(b) must have opposite signs.")
  }

  history <- data.frame()

  for (k in 0:maxit) {

    midpoint <- (a + b) / 2

    history <- rbind(
      history,
      data.frame(
        iteration = k,
        a = a,
        b = b,
        midpoint = midpoint,
        f_midpoint = f(midpoint)
      )
    )
```

```
    if (
      abs(f(midpoint)) < tol ||
      (b - a) / 2 < tol
    ) {
      break
    }

    if (f(a) * f(midpoint) <= 0) {
      b <- midpoint
    } else {
      a <- midpoint
    }
  }

  history
}

bisect_history <- bisect_trace(
  f_root,
  a = 0,
```

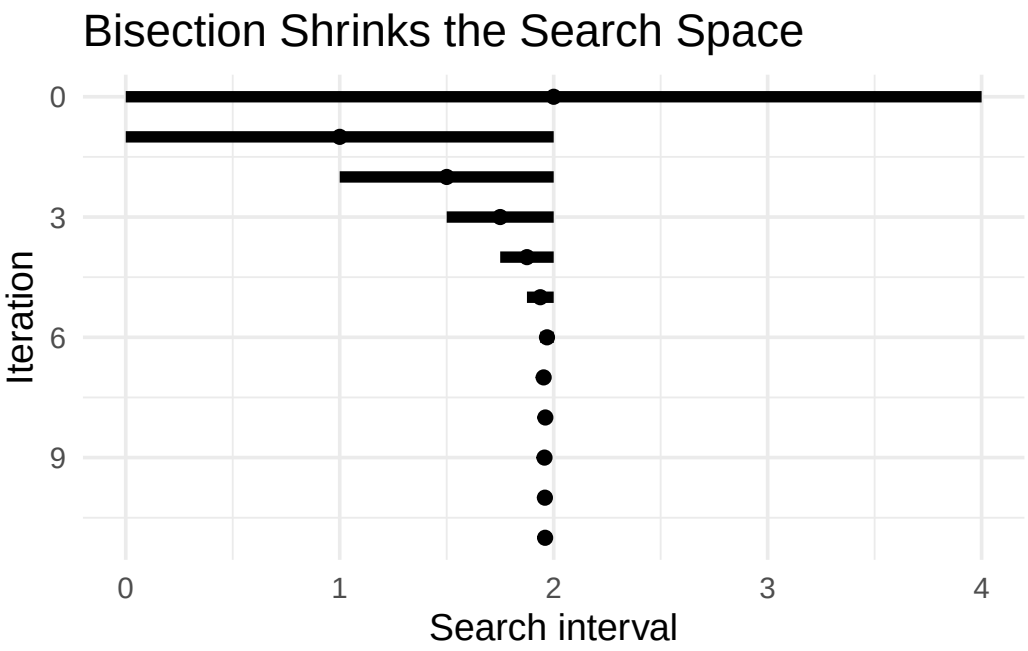
```
  b = 4
)
```

```
tail(bisect_history)
```

```
   iteration        a        b midpoint    f_midpoint
7          6 1.937500 2.000000 1.968750  5.090967e-04
8          7 1.937500 1.968750 1.953125 -4.023926e-04
9          8 1.953125 1.968750 1.960938  5.684292e-05
10         9 1.953125 1.960938 1.957031 -1.718972e-04
11        10 1.957031 1.960938 1.958984 -5.730834e-05
12        11 1.958984 1.960938 1.959961 -1.780850e-07
```

Visualizing

```
ggplot(
  bisect_history,
  aes(y = iteration)
) +
  geom_segment(
    aes(
      x = a,
      xend = b,
      yend = iteration
    ),
    linewidth = 2
  ) +
  geom_point(
    aes(x = midpoint),
    size = 2
  ) +
  scale_y_reverse() +
  labs(
    x = "Search interval",
    y = "Iteration",
    title = "Bisection Shrinks the Search Space"
  )
```



Bisection requires continuity and a sign change, but does not require derivatives.

> **Question 4: In-Class Question**
> What information does bisection use about the function?
> What information does it ignore?

3.5.3 Newton-Raphson Method

Newton's method uses local derivative information.

Around the current estimate x_k,

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k).$$

Set the approximation equal to zero:

$$0 = f(x_k) + f'(x_k)(x_{k+1} - x_k).$$

Therefore,

$$\boxed{x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}}.$$

The intuition is:

> Approximate the function locally by a straight line and use the root of that line as the next guess.

For

$$f(z) = \Phi(z) - 0.975,$$

we have

$$f'(z) = \phi(z).$$

```r
f <- function(z) {
  pnorm(z) - 0.975
}

df <- function(z) {
  dnorm(z)
}
```

```r
newton_trace <- function(
    f,
    df,
    x0,
    tol = 1e-8,
    maxit = 100
) {

  x <- x0

  history <- data.frame(
    iteration = 0,
    x = x,
    fx = f(x)
  )

  for (k in seq_len(maxit)) {

    dfx <- df(x)
```

```r
    if (abs(dfx) < 1e-12) {
      stop("Derivative is too close to zero.")
    }

    x_new <- x - f(x) / dfx

    history <- rbind(
      history,
      data.frame(
        iteration = k,
        x = x_new,
        fx = f(x_new)
      )
    )
```

```
    if (abs(x_new - x) < tol) {
      break
    }

    x <- x_new
  }
```

```
  history
}

newton_history <- newton_trace(
  f,
  df,
  x0 = 1
)

newton_history
```

```
  iteration        x            fx
1         0 1.000000 -1.336553e-01
2         1 1.552361 -3.528790e-02
3         2 1.847484 -7.338509e-03
4         3 1.948843 -6.570601e-04
5         4 1.959844 -7.006182e-06
6         5 1.959964 -8.227717e-10
7         6 1.959964  0.000000e+00
8         7 1.959964  0.000000e+00
```

3.5.4 Visualizing Newton's Method

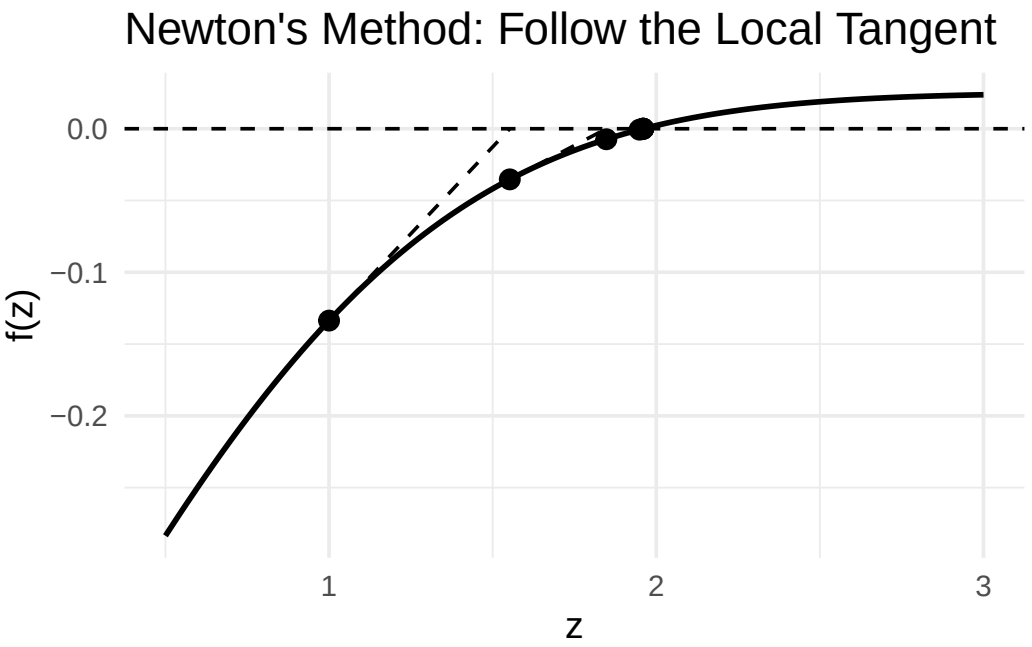
```
tangent_data <- data.frame(
  x = head(newton_history$x, -1),
  y = head(newton_history$fx, -1),
  xend = tail(newton_history$x, -1),
  yend = 0
)

z_grid <- seq(
  0.5,
  3,
  length.out = 400
)

newton_function_data <- data.frame(
  z = z_grid,
  f = f(z_grid)
)
```

```
ggplot(
  newton_function_data,
  aes(
    x = z,
    y = f
  )
) +
  geom_line(linewidth = 1) +
  geom_hline(
    yintercept = 0,
    linetype = "dashed"
  ) +
  geom_segment(
    data = tangent_data,
    aes(
      x = x,
      y = y,
      xend = xend,
      yend = yend
    ),
    inherit.aes = FALSE,
    linetype = "dashed"
```

```
  ) +
  geom_point(
    data = newton_history,
    aes(
      x = x,
      y = fx
    ),
    inherit.aes = FALSE,
    size = 3
  ) +
  labs(
    x = "z",
    y = "f(z)",
    title = "Newton's Method: Follow the Local Tangent"
  )
```



Newton's method can converge very quickly near the root.

However, if

$$f'(x_k) \approx 0,$$

then the update may become very large and unstable.

Question 5: In-Class Question
What happens to Newton's update when $f'(x_k)$ is close to zero?

3.5.5 Secant Method

Newton's method requires the derivative $f'(x)$.

If the derivative is unavailable or expensive, we can approximate it using two previous points:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

Substituting this into Newton's update gives

$$\boxed{x_{k+1} = x_k - f(x_k)\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}}.$$

```r
secant_trace <- function(
    f,
    x0,
    x1,
    tol = 1e-8,
    maxit = 100
) {

  history <- data.frame(
    iteration = c(0, 1),
    x = c(x0, x1),
    fx = c(
      f(x0),
      f(x1)
    )
  )

  for (k in 2:maxit) {

    f0 <- f(x0)
    f1 <- f(x1)
```

```r
    denominator <- f1 - f0

    if (abs(denominator) < 1e-12) {
      stop("Secant denominator is too close to zero.")
    }

    x2 <- x1 -
      f1 * (x1 - x0) /
      denominator

    history <- rbind(
      history,
      data.frame(
        iteration = k,
        x = x2,
        fx = f(x2)
      )
    )

    if (abs(x2 - x1) < tol) {
      break
    }
```

```r
    x0 <- x1
    x1 <- x2
  }

  history
}
```

```
secant_history <- secant_trace(
  f,
  x0 = 1,
  x1 = 2.5
)

secant_history
```

```
   iteration        x            fx
1          0 1.000000 -1.336553e-01
2          1 2.500000  1.879033e-02
3          2 2.315111  1.469657e-02
4          3 1.651361 -2.433241e-02
5          4 2.065173  5.546662e-03
6          5 1.988354  1.613717e-03
7          6 1.956835 -1.834592e-04
8          7 1.960052  5.151979e-06
9          8 1.959964  1.577731e-08
10         9 1.959964 -1.363021e-12
11        10 1.959964  0.000000e+00
```

The three methods illustrate different computational trade-offs.

Method	Derivative?	Starting information	Main advantage	Main limitation
Bisection	No	$[a, b]$	Very reliable	Slower
Newton	Yes	x_0	Very fast near root	Can fail
Secant	No	x_0, x_1	Fast without derivative	Less reliable

Section 3.6 Convergence

Suppose an iterative algorithm produces

$$x_0, x_1, x_2, ...$$

and converges to a solution x^*.

Define

$$e_k = |x_k - x^*|.$$

A common description of convergence is

$$e_{k+1} \approx C e_k^q.$$

The value q describes how quickly the error decreases. Here $C > 0$, and the approximation describes the behavior near the solution.

- **Linear convergence:** $q = 1$ and $0 < C < 1$.
- **Superlinear convergence:** faster than linear.
- **Quadratic convergence:** $q = 2$.

Under suitable conditions:

- bisection has a linearly decreasing error bound: the bracket width halves at each step;
- the secant method converges superlinearly;
- Newton's method converges quadratically near the root.

For a simple worked interpretation, see Section 3.18: Error Analysis and Convergence Rates.

3.6.1 Comparing Convergence in R

```
true_root <- qnorm(0.975)

bisection_error <- data.frame(
  iteration = bisect_history$iteration,
  error = abs(
    bisect_history$midpoint -
      true_root
  ),
  method = "Bisection"
)

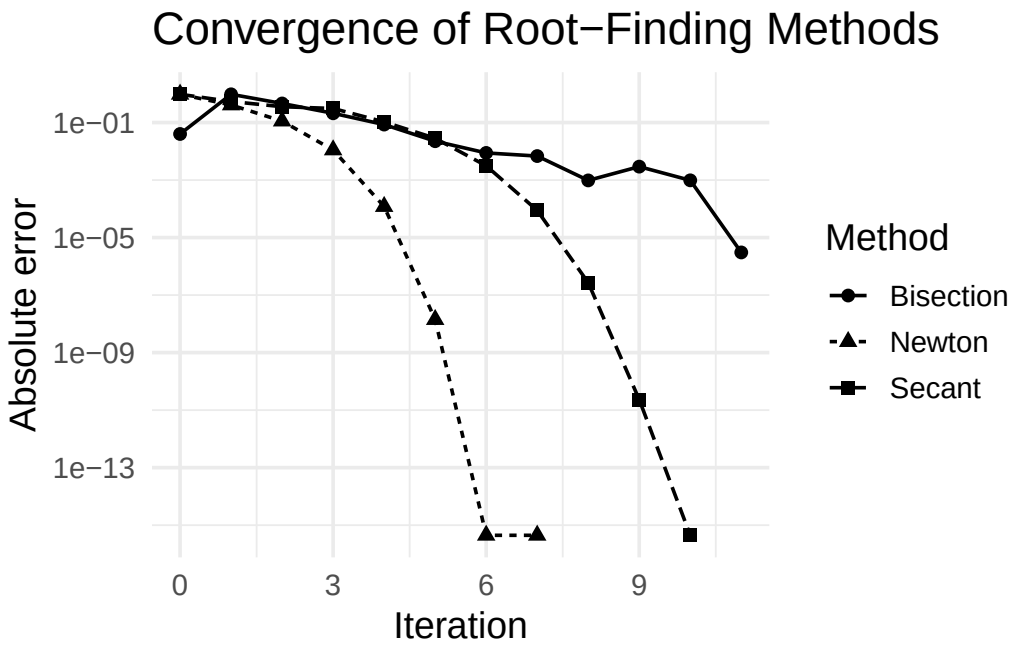
newton_error <- data.frame(
  iteration = newton_history$iteration,
  error = abs(
    newton_history$x -
      true_root
  ),
  method = "Newton"
)

secant_error <- data.frame(
  iteration = secant_history$iteration,
  error = abs(
    secant_history$x -
      true_root
  ),
  method = "Secant"
)

convergence_data <- rbind(
  bisection_error,
  newton_error,
  secant_error
)
```

```
convergence_data <- subset(
  convergence_data,
  error > .Machine$double.eps
)
```

```
ggplot(
  convergence_data,
  aes(
    x = iteration,
    y = error,
    linetype = method,
    shape = method
  )
) +
  geom_line() +
  geom_point(size = 2) +
  scale_y_log10() +
  labs(
    x = "Iteration",
    y = "Absolute error",
    title = "Convergence of Root-Finding Methods",
    linetype = "Method",
    shape = "Method"
  )
```



Question 6: In-Class Question

Which method appears to converge fastest?
Does fewer iterations necessarily mean less computation?

A useful computational perspective is

Total Computational Cost $\approx$ Cost per Iteration $\times$ Number of Iterations.

Section 3.7 From One Dimension to Multiple Dimensions

Most statistical problems involve more than one parameter.

Instead of

$$x \in \mathbb{R},$$

we now consider

$$\boldsymbol{\theta} = (\theta_1, \dots, \theta_p)^\top \in \mathbb{R}^p.$$

The one-dimensional concepts generalize naturally:

$$Q'(x) \quad \Longrightarrow \quad \nabla Q(\boldsymbol{\theta}),$$

and

$$Q''(x) \quad \Longrightarrow \quad H(\boldsymbol{\theta}).$$

The **gradient** is

$$\nabla Q(\boldsymbol{\theta}) = \begin{pmatrix} \partial Q/\partial \theta_1 \\ \vdots \\ \partial Q/\partial \theta_p \end{pmatrix},$$

and the **Hessian** is the matrix of second derivatives,

$$H(\boldsymbol{\theta}) = \left(\frac{\partial^2 Q}{\partial \theta_j \partial \theta_k} \right)_{j,k=1}^{p}.$$

Example 2: Visualizing a Two-Dimensional Objective

Consider

$$Q(\theta_1, \theta_2) = (\theta_1 - 2)^2 + 4(\theta_2 + 1)^2.$$

```r
objective_2d <- function(theta1, theta2) {
  (theta1 - 2)^2 +
    4 * (theta2 + 1)^2
}

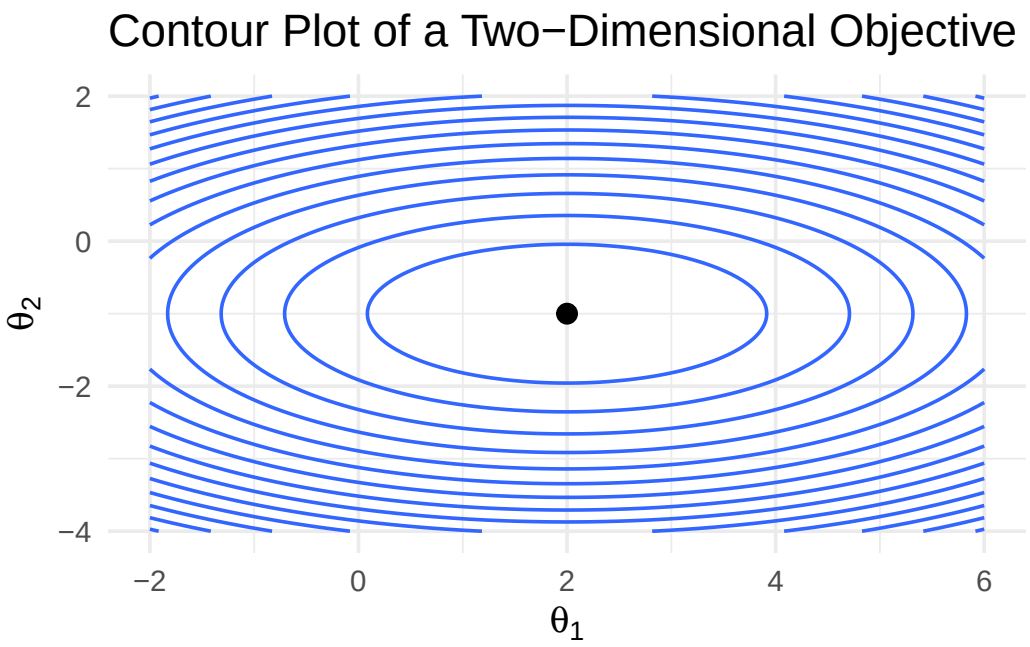
theta1_grid <- seq(
  -2,
  6,
  length.out = 150
)

theta2_grid <- seq(
  -4,
  2,
  length.out = 150
)

objective_grid <- expand.grid(
  theta1 = theta1_grid,
  theta2 = theta2_grid
)
```

```r
objective_grid$value <- with(
  objective_grid,
  objective_2d(theta1, theta2)
)

ggplot(
  objective_grid,
  aes(
    x = theta1,
    y = theta2,
    z = value
  )
) +
  geom_contour(
    bins = 15
  ) +
  geom_point(
    data = data.frame(
      theta1 = 2,
      theta2 = -1
    ),
    aes(
      x = theta1,
      y = theta2
    ),
    inherit.aes = FALSE,
    size = 3
  ) +
  labs(
    x = expression(theta[1]),
    y = expression(theta[2]),
    title = "Contour Plot of a Two-Dimensional Objective"
  )
```



The minimum occurs at

$$\boldsymbol{\theta}^* = \begin{pmatrix} 2 \\ -1 \end{pmatrix}.$$

Section 3.8 Gradient Descent

The gradient points in the direction of greatest increase.

Therefore,

$$-\nabla Q(\boldsymbol{\theta})$$

is a natural direction for decreasing the objective.

Gradient descent updates

$$\boxed{\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha_k \nabla Q(\boldsymbol{\theta}_k),}$$

where $\alpha_k > 0$ is the **step size** or **learning rate**.

For the quadratic example,

$$\nabla Q(\theta_1, \theta_2) = \begin{pmatrix} 2(\theta_1 - 2) \\ 8(\theta_2 + 1) \end{pmatrix}.$$

```r
gradient_2d <- function(theta) {
  c(
    2 * (theta[1] - 2),
    8 * (theta[2] + 1)
  )
}

gradient_descent <- function(
    start,
    alpha = 0.1,
    maxit = 30
) {

  theta <- start

  history <- data.frame(
    iteration = 0,
    theta1 = theta[1],
    theta2 = theta[2]
  )

  for (k in seq_len(maxit)) {

    theta <-
      theta -
      alpha * gradient_2d(theta)

    history <- rbind(
      history,
      data.frame(
        iteration = k,
        theta1 = theta[1],
        theta2 = theta[2]
      )
    )
  }

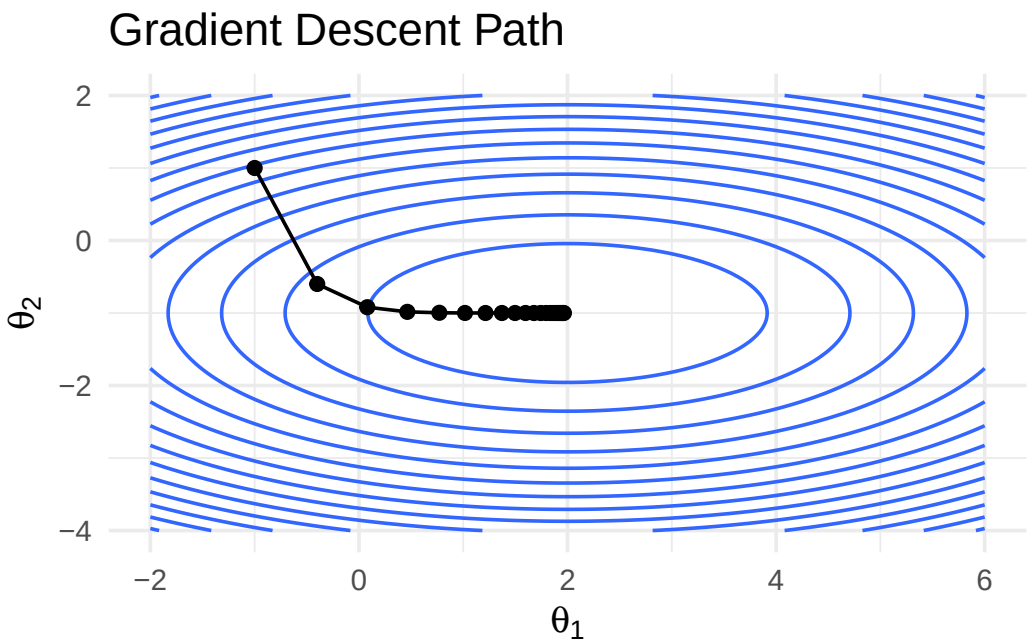
  history
}

gd_path <- gradient_descent(
  start = c(-1, 1),
  alpha = 0.1,
  maxit = 20
)
```

3.8.1 Visualizing the Optimization Path

```r
ggplot(
  objective_grid,
  aes(
```

```
    x = theta1,
    y = theta2,
    z = value
  )
) +
  geom_contour(
    bins = 15
  ) +
  geom_path(
    data = gd_path,
    aes(
      x = theta1,
      y = theta2
    ),
    inherit.aes = FALSE
  ) +
  geom_point(
    data = gd_path,
    aes(
      x = theta1,
      y = theta2
    ),
    inherit.aes = FALSE,
    size = 2
  ) +
  labs(
    x = expression(theta[1]),
    y = expression(theta[2]),
    title = "Gradient Descent Path"
  )
```



> **Question 7: In-Class Question**
>
> What happens if the step size α is too small?
> What happens if it is too large?

3.8.2 Step Size Matters

```
gd_small <- gradient_descent(
  start = c(-1, 1),
  alpha = 0.02,
  maxit = 30
)

gd_medium <- gradient_descent(
  start = c(-1, 1),
  alpha = 0.1,
  maxit = 30
)

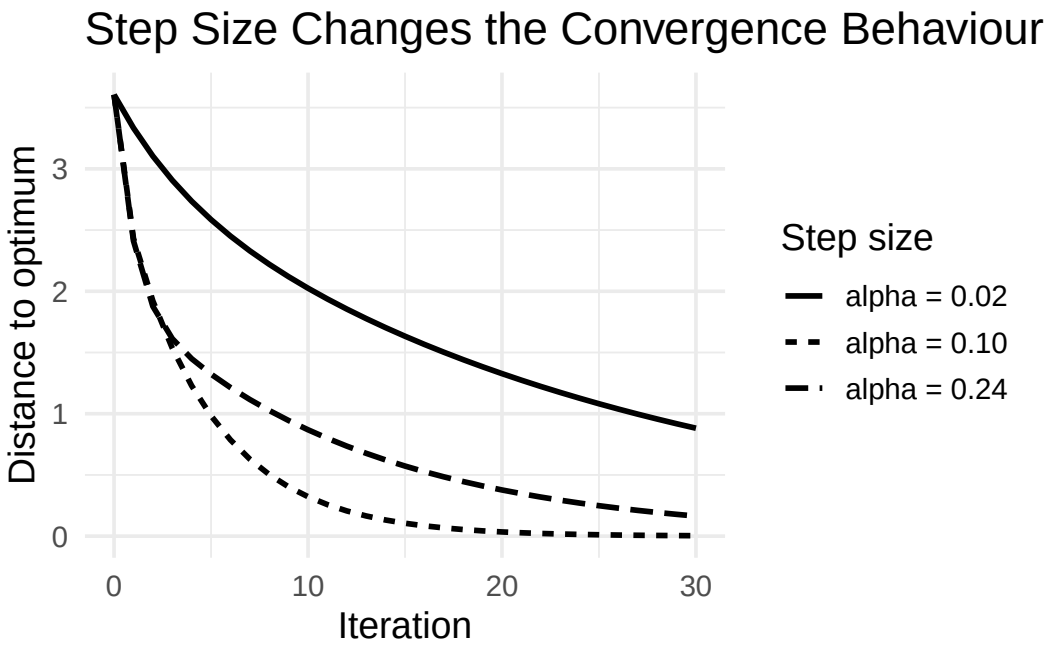
gd_large <- gradient_descent(
  start = c(-1, 1),
  alpha = 0.24,
  maxit = 30
)

gd_compare <- rbind(
  transform(
    gd_small,
    step = "alpha = 0.02"
  ),
  transform(
    gd_medium,
    step = "alpha = 0.10"
  ),
  transform(
    gd_large,
    step = "alpha = 0.24"
  )
)
```

```
gd_compare$error <-
  sqrt(
    (gd_compare$theta1 - 2)^2 +
      (gd_compare$theta2 + 1)^2
  )

ggplot(
  gd_compare,
  aes(
    x = iteration,
```

```
    y = error,
    linetype = step
  )
) +
  geom_line(linewidth = 1) +
  labs(
    x = "Iteration",
    y = "Distance to optimum",
    linetype = "Step size",
    title = "Step Size Changes the Convergence Behaviour"
  )
```



Section 3.9 Multivariate Newton's Method

In one dimension,

$$x_{k+1} = x_k - \frac{Q'(x_k)}{Q''(x_k)}.$$

In multiple dimensions, this becomes

$$\boxed{\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - H(\boldsymbol{\theta}_k)^{-1}\nabla Q(\boldsymbol{\theta}_k).}$$

However, computationally we usually do **not** calculate the inverse explicitly.

Instead, solve

$$H(\boldsymbol{\theta}_k)\boldsymbol{d}_k = -\nabla Q(\boldsymbol{\theta}_k),$$

and update

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \boldsymbol{d}_k.$$

Question 8: In-Class Question

Why do we solve a linear system instead of explicitly computing

$$H^{-1}?$$

This connects directly to the earlier discussion of numerical stability and matrix inversion.

Section 3.10 Quasi-Newton Methods

Newton's method requires the Hessian.

For high-dimensional statistical models, computing the Hessian can be expensive.

Quasi-Newton methods approximate curvature information rather than calculating the Hessian directly.

One of the most widely used methods is **BFGS**.

In R:

```
optim(
  par = start,
  fn = objective,
  method = "BFGS"
)
```

The main computational idea is

Use curvature information without repeatedly computing the full Hessian.

Example 3: Logistic Regression

Consider

$$Y_i \sim \text{Bernoulli}(p_i),$$

with

$$\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 x_i.$$

The parameter is now a vector

$$\boldsymbol{\beta} = (\beta_0, \beta_1)^\top.$$

There is no closed-form MLE, so logistic regression requires numerical optimization.

```
set.seed(8670)

n <- 200

x_logit <- rnorm(n)

beta_true <- c(
  -0.5,
  1.5
)

eta <-
  beta_true[1] +
  beta_true[2] * x_logit

p <- plogis(eta)

y_logit <- rbinom(
  n,
  size = 1,
  prob = p
)
```

Define the negative log-likelihood:

```
logistic_nll <- function(beta) {

  eta <-
    beta[1] +
    beta[2] * x_logit

  p <- plogis(eta)

  -sum(
    dbinom(
      y_logit,
      size = 1,
      prob = p,
      log = TRUE
    )
  )
}
```

Estimate the coefficients using BFGS:

```r
fit_optim <- optim(
  par = c(0, 0),
  fn = logistic_nll,
  method = "BFGS"
)

fit_optim$par
```

```
[1] -0.381730  1.014505
```

Compare with `glm()`:

```r
fit_glm <- glm(
  y_logit ~ x_logit,
  family = binomial()
)

coef(fit_glm)
```

```
(Intercept)     x_logit
 -0.3817297   1.0145052
```

```r
fit_optim$par
```

```
[1] -0.381730  1.014505
```

Question 9: In-Class Question

Why is comparing `optim()` with `glm()` useful?

This illustrates the complete computational-statistics workflow:

$$\boxed{\text{Statistical Model} \to \text{Likelihood} \to \text{Optimization} \to \text{Algorithm} \to \text{Validation.}}$$

Section 3.11 Scaling and Conditioning

Optimization can behave very differently when parameters operate on different scales.

Consider

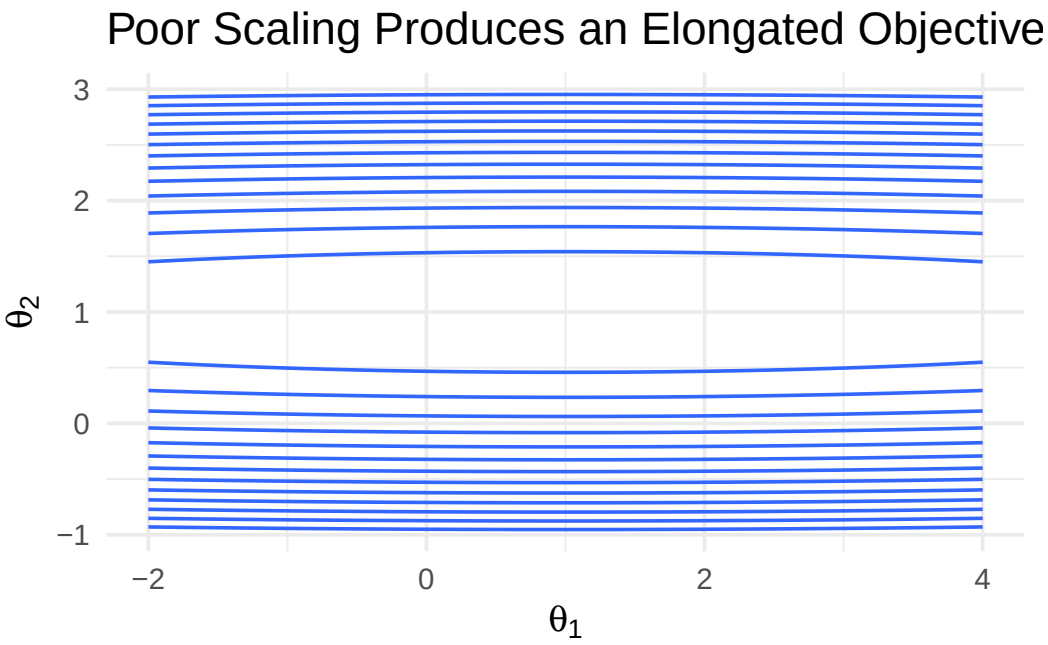
$$Q(\theta_1, \theta_2) = (\theta_1 - 1)^2 + 100(\theta_2 - 1)^2.$$

```r
scaled_objective <- function(theta1, theta2) {
  (theta1 - 1)^2 +
    100 * (theta2 - 1)^2
}
```

```
scale_grid <- expand.grid(
  theta1 = seq(
    -2,
    4,
    length.out = 150
  ),
  theta2 = seq(
    -1,
    3,
    length.out = 150
  )
)

scale_grid$value <- with(
  scale_grid,
  scaled_objective(
    theta1,
    theta2
  )
)

ggplot(
  scale_grid,
  aes(
    x = theta1,
    y = theta2,
    z = value
  )
) +
  geom_contour(
    bins = 15
  ) +
  labs(
    x = expression(theta[1]),
    y = expression(theta[2]),
    title = "Poor Scaling Produces an Elongated Objective"
  )
```



A highly elongated objective can cause gradient-based algorithms to zig-zag and converge slowly.

This illustrates another important principle:

Same mathematical problem + different scaling $\Longrightarrow$ different computational behaviour.

Section 3.12 Metaheuristic Methods

Deterministic optimization methods can work extremely well for smooth and well-behaved objectives.

However, they may struggle when the objective:

- contains many local optima;
- is not differentiable;
- is discontinuous;
- has a complicated search space.

Metaheuristic methods use broader stochastic exploration.

The key idea is

Explore broadly $\longrightarrow$ search more locally later.

Why Accept a Worse Solution?

A local deterministic method usually attempts to improve the objective at every step.

A stochastic method may deliberately accept a worse solution.

> **Question 10: In-Class Question**
> Why would an optimization algorithm ever accept a worse solution?

Temporarily moving to a worse location can allow the algorithm to escape a local optimum and later reach a better region.

Section 3.13 Simulated Annealing

Simulated annealing is a stochastic optimization method inspired by the physical process of heating and slowly cooling a material.

Suppose the current solution is x_k and a candidate solution x' is generated.

For a minimization problem, if

$$Q(x') < Q(x_k),$$

the new solution is accepted.

If

$$Q(x') > Q(x_k),$$

the worse solution may still be accepted with probability

$$\boxed{P(\text{accept}) = \exp\left(-\frac{\Delta Q}{T}\right),}$$

where

$$\Delta Q = Q(x') - Q(x_k) > 0.$$

The parameter T is called the **temperature**.

When

$$T \quad \text{is large,}$$

the algorithm explores more broadly.

When

$$T \quad \text{is small,}$$

the algorithm becomes more selective.

Visualizing the Acceptance Probability

```r
delta_grid <- seq(
  0,
  5,
  length.out = 300
)

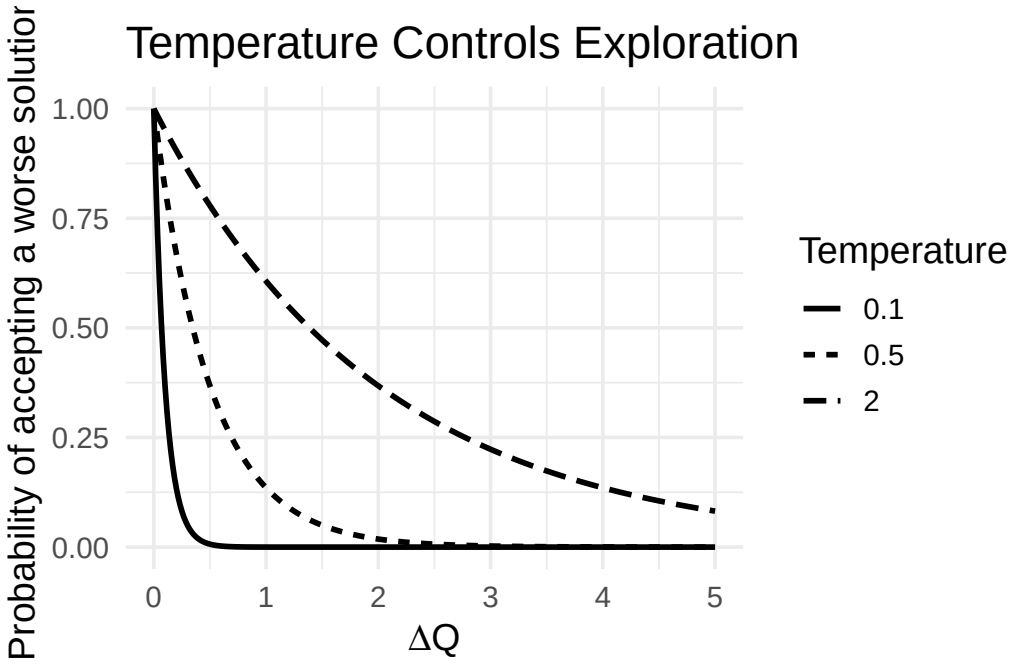
temperature <- c(
  0.1,
  0.5,
  2
)

sa_probability <- expand.grid(
  delta = delta_grid,
  temperature = temperature
)

sa_probability$probability <-
  exp(
    -sa_probability$delta /
      sa_probability$temperature
  )

sa_probability$temperature <-
  factor(
    sa_probability$temperature
  )

ggplot(
  sa_probability,
  aes(
    x = delta,
    y = probability,
    linetype = temperature
  )
) +
  geom_line(linewidth = 1) +
  labs(
    x = expression(Delta * Q),
    y = "Probability of accepting a worse solution",
    linetype = "Temperature",
    title = "Temperature Controls Exploration"
  )
```



Question 11: In-Class Question

At high temperature, why is the algorithm more willing to accept a poor move? What happens as the temperature approaches zero?

3.13.1 Simulated Annealing in R

R provides simulated annealing through

```
optim(
  ...,
  method = "SANN"
)
```

Using the earlier multimodal objective:

```
set.seed(8670)

sa_fit <- optim(
  par = 5,
  fn = objective,
  method = "SANN",
  control = list(
    maxit = 3000
  )
)

sa_fit$par
```

```
[1] -0.1958383
```

```
sa_fit$value
```

```
[1] -0.1262269
```

Because simulated annealing is stochastic, successive runs may produce different results. Set the seed once before the experiment so the complete sequence of runs is reproducible; allow the random-number generator to advance between runs.

```
set.seed(8670)
runs <- 1:20

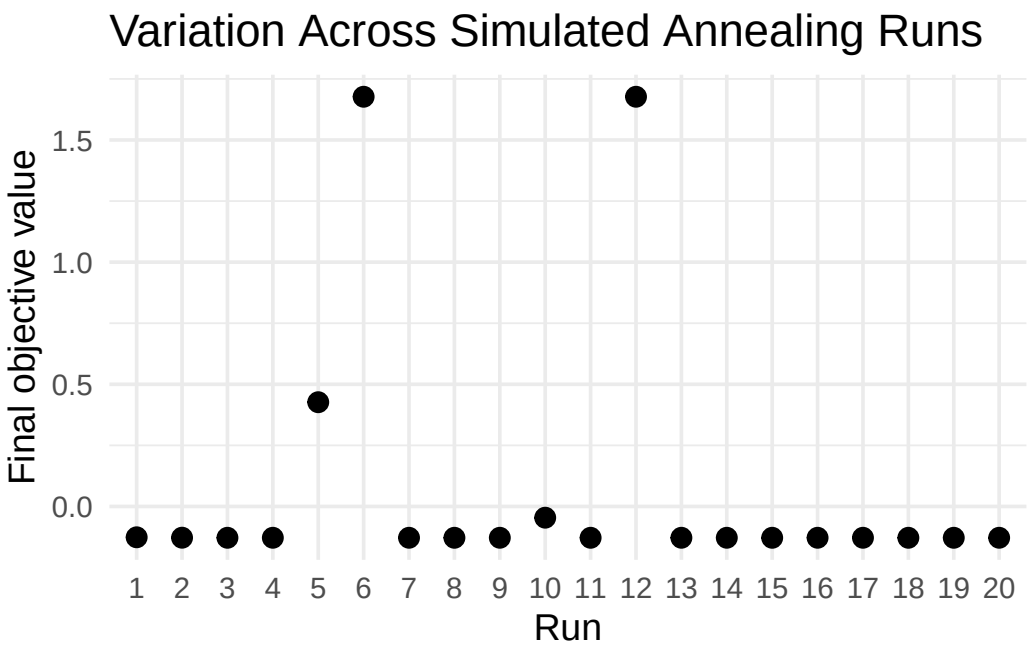
sa_solution <- vapply(
  runs,
  function(run) {

    optim(
      par = 5,
      fn = objective,
      method = "SANN",
      control = list(
        maxit = 3000
      )
    )$par

  },
  numeric(1)
)

sa_data <- data.frame(
  run = runs,
  solution = sa_solution,
  objective = objective(sa_solution)
)

ggplot(
  sa_data,
  aes(
    x = factor(run),
    y = objective
  )
) +
  geom_point(size = 3) +
  labs(
    x = "Run",
    y = "Final objective value",
    title = "Variation Across Simulated Annealing Runs"
  )
```



Advantages of Simulated Annealing

- Can escape local optima.
- Does not require derivatives.
- Easy to adapt to many objective functions.
- Useful for multimodal problems.

Limitations of Simulated Annealing

- Can require many function evaluations.
- Results vary across runs.
- Performance depends on the cooling schedule.
- Does not guarantee the global optimum.

Section 3.14 Genetic Algorithm

A genetic algorithm uses a **population of candidate solutions** rather than one current solution.

The basic cycle is

$$\boxed{\text{Selection} \rightarrow \text{Crossover} \rightarrow \text{Mutation} \rightarrow \text{New Generation.}}$$

3.14.1 Selection

Better candidate solutions are more likely to remain in the population.

3.14.2 Crossover

Information from two candidate solutions is combined to create new candidates.

3.14.3 Mutation

Random changes are introduced to preserve diversity and explore new regions.

The key difference from simulated annealing is:

SA: one evolving solution	GA: a population of solutions.

Advantages

- Explores several regions simultaneously.
- Does not require gradients.
- Flexible for continuous and discrete problems.

Limitations

- Can require many objective evaluations.
- Requires tuning of population size and mutation rate.
- Can converge slowly.
- Does not guarantee a global optimum.

Section 3.15 Particle Swarm Optimization

Particle swarm optimization also maintains a population of candidate solutions called **particles**.

Each particle uses information about:

- its current location;
- its previous velocity;
- its own best location;
- the best location found by the group.

A simplified velocity update is

$$v_{i,k+1} = wv_{i,k} + c_1 r_1 (p_i - x_{i,k}) + c_2 r_2 (g - x_{i,k}),$$

followed by

$$x_{i,k+1} = x_{i,k} + v_{i,k+1}.$$

The intuition is

Momentum + Individual Experience + Group Experience + Randomness.

3.15.1 Advantages

- Does not require derivatives.
- Explores multiple regions simultaneously.
- Relatively intuitive and flexible.

3.15.2 Limitations

- May require many function evaluations.
- Sensitive to tuning parameters.
- Can converge prematurely.
- Global optimality is not guaranteed.

Section 3.16 Deterministic versus Metaheuristic Methods

The two families search very differently.

Method	Type	Derivatives	Search	Main Advantage	Main Limitation
Bisection	Deterministic	No	Root finding	Very reliable	Slow
Newton	Deterministic	Yes	Local	Very fast near solution	Sensitive to derivative and start
Secant	Deterministic	No	Local	Fast without derivative	Less reliable
Gradient Descent	Deterministic	Gradient	Local	Simple and scalable	Step size matters
BFGS	Deterministic	Usually gradient	Local	Efficient for smooth problems	Can reach local optimum
Simulated Annealing	Metaheuristic	No	Global exploration	Can escape local optima	Slow and stochastic
Genetic Algorithm	Metaheuristic	No	Population	Flexible search	Computationally expensive
Particle Swarm	Metaheuristic	No	Population	Cooperative exploration	Requires tuning

The important question is not

> Which algorithm is the best?

Instead, ask

Which algorithm is appropriate for the structure of the statistical problem?

For a smooth and well-behaved objective,

deterministic methods

are usually preferable.

For a highly multimodal, irregular, or derivative-free objective,

metaheuristic methods

may become useful.

> **Question 12: In-Class Question**
> Suppose the objective is smooth, convex, and easy to differentiate.
> Would you choose BFGS or simulated annealing?
> What changes if the objective has many local minima and no useful derivatives?

Section 3.17 Computational Perspective

The goal is not simply to memorize optimization algorithms.

For a statistical problem, ask:

1. What is the statistical quantity we want?
2. Can it be written as an optimization or root-finding problem?
3. Is there a closed-form solution?
4. Is the parameter scalar or multivariate?
5. Is the objective smooth?
6. Are derivatives available?
7. Are there many local optima?
8. How sensitive is the method to the starting value?
9. Is the computation numerically stable?
10. How can we validate the result?

This brings us back to the central idea of statistical computing:

$$\boxed{\text{Theory} \iff \text{Computation} \iff \text{Practice.}}$$

Section 3.18 Error Analysis and Convergence Rates

Chapter 3 - Optimization, Section 3.6: Convergence asks how quickly an algorithm approaches its target x^*. Use two quantities:

$$e_k = |x_k - x^*|, \qquad R_k = \frac{e_{k+1}}{e_k}.$$

The **error** e_k is the distance from the target. The **ratio** R_k is the next error divided by the current error. For example, $R_k = 0.5$ means that half of the error remains after one update. Convergence means $e_k \to 0$; the ratios tell us how quickly this happens.

For **linear convergence** (order $q = 1$), the limiting ratio must satisfy $0 < C < 1$. These are **asymptotic definitions**: they describe what happens as k becomes large and the errors approach zero, not necessarily the first few steps.

Rate	Diagnostic	Interpretation
Linear	$e_{k+1}/e_k \to C \in (0,1)$	Eventually keeps a fixed fraction of the error.
Superlinear	$e_{k+1}/e_k \to 0$	The fraction of error kept tends to zero.
Quadratic	$e_{k+1}/e_k^2 \to C \in (0,\infty)$	The next error is approximately a constant times the square of the current error.

Example 4: Linear or Superlinear? Calculate the Error Ratios

Example: Linear or Superlinear? Calculate the Error Ratios
Our target is $x^* = 2$, and we start at $x_0 = 2.1$. Because the target is known in this example, we can calculate the error directly:

$$e_0 = |2.1 - 2| = 0.1.$$

A. Keep half of the error. Move halfway from the current guess to 2:

$$x_{k+1} = 2 + 0.5(x_k - 2).$$

The first update is $x_1 = 2 + 0.5(2.1 - 2) = 2.05$, giving $e_1 = |2.05 - 2| = 0.05$. Continue the same way:

Step k	Guess x_k	Error $e_k = \lvert x_k - 2\rvert$
0	2.1	0.1
1	2.05	0.05
2	2.025	0.025
3	2.0125	0.0125

Now divide each error by the one before it:

$$R_0 = \frac{0.05}{0.1} = 0.5,$$
$$R_1 = \frac{0.025}{0.05} = 0.5,$$
$$R_2 = \frac{0.0125}{0.025} = 0.5.$$

The errors approach zero, while the ratio stays at 0.5. **This is linear convergence.** The rule $e_{k+1} = 0.5e_k$ confirms that the same fraction remains at every step.

B. Square the error. Start again at $x_0 = 2.1$, but change the update to

$$x_{k+1} = 2 + (x_k - 2)^2.$$

Now $x_1 = 2 + (2.1 - 2)^2 = 2.01$, so $e_1 = |2.01 - 2| = 0.01$. Continue the same way:

Step k	Guess x_k	Error $e_k = \lvert x_k - 2\rvert$
0	2.1	0.1
1	2.01	0.01
2	2.0001	0.0001
3	2.00000001	0.00000001

Calculate the same ratios:

$$R_0 = \frac{0.01}{0.1} = 0.1,$$
$$R_1 = \frac{0.0001}{0.01} = 0.01,$$
$$R_2 = \frac{0.00000001}{0.0001} = 0.0001.$$

Each step keeps a smaller fraction of the error: $0.1,\ 0.01,\ 0.0001, \ldots$. Starting at $e_0 = 0.1$, repeatedly squaring the error makes it tend to zero. Since $e_{k+1} = e_k^2$,

$$R_k = \frac{e_k^2}{e_k} = e_k \longrightarrow 0.$$

This is superlinear convergence. It is also quadratic because the next error is the current error squared. Quadratic convergence is one kind of superlinear convergence.

Example 5: Newton's Method: Superlinear, Quadratic, and Accurate Enough

Example: Newton's Method - Superlinear, Quadratic, and Accurate Enough
Return to Chapter 3 - Optimization, Section 3.5.3: Newton-Raphson Method. To find the standard normal 97.5th percentile, solve $f(x) = \Phi(x) - 0.975 = 0$. Starting at $x_0 = 1$, use

$$x_{k+1} = x_k - \frac{\Phi(x_k) - 0.975}{\phi(x_k)}.$$

For example, $x_1 \approx 1.552361$. Using `qnorm(0.975)` as the reference $x^* \approx 1.959964$ gives $e_0 \approx 0.959964$ and $e_1 \approx 0.407603$. The first five updates give the following errors. Ratios are calculated before rounding; a dash means that no next error is included in the table.

k	Error e_k	$R_k = e_{k+1}/e_k$	e_{k+1}/e_k^2
0	9.600×10^{-1}	0.424602	0.442310
1	4.076×10^{-1}	0.275955	0.677019
2	1.125×10^{-1}	0.098867	0.878971
3	1.112×10^{-2}	0.010778	0.969244
4	1.199×10^{-4}	0.000117	0.979865
5	1.408×10^{-8}	-	-

Why superlinear? The next-error/current-error ratios head toward zero. For example,

$$R_0 \approx \frac{0.407603}{0.959964} \approx 0.424602, \qquad R_4 \approx \frac{1.408 \times 10^{-8}}{1.199 \times 10^{-4}} \approx 0.000117.$$

Why quadratic as well? The last column settles near a positive constant, about 0.98. It suggests

$$e_{k+1} \approx 0.98 e_k^2.$$

Consequently, $R_k \approx 0.98 e_k \to 0$. **Quadratic and superlinear are both correct descriptions.** The table illustrates Newton's local quadratic behavior for this problem; a finite table alone is not a proof.

How do we choose C? We do not freely choose C in $e_{k+1} \approx C e_k^q$: it describes the algorithm's error behavior. For linear convergence, estimate it from e_{k+1}/e_k. For quadratic convergence, use e_{k+1}/e_k^2 once that ratio is stable. Here,

$$C \approx \frac{1.408 \times 10^{-8}}{(1.199 \times 10^{-4})^2} \approx 0.98.$$

This 0.98 is an estimate of the limiting ratio, not a tolerance or a guaranteed error bound. For quadratic convergence, C need not be less than 1.

What precision should we request? We choose a **tolerance**, based on how accurate the answer needs to be. For example, an absolute tolerance of 10^{-6} asks for error below one millionth:

- At step 4, $e_4 \approx 1.2 \times 10^{-4} > 10^{-6}$: not accurate enough.
- At step 5, $e_5 \approx 1.4 \times 10^{-8} < 10^{-6}$: accurate enough.

The quadratic rule predicts this improvement: $0.98(1.2 \times 10^{-4})^2 \approx 1.4 \times 10^{-8}$. Near a solution on this scale, roughly four decimal places of accuracy become roughly eight in one update. Ordinary R arithmetic has about 15–16 significant digits of precision, so this improvement cannot continue indefinitely.

Here we can check the error because we have a trusted reference. When the true solution is unknown, small steps and residuals are practical checks, not automatically certified error bounds.

How to decide: for errors approaching zero,

- Ratios approaching a **fixed constant between** 0 **and** 1 mean **linear convergence**.
- Ratios approaching 0 mean **superlinear convergence**.
- If e_{k+1}/e_k^2 approaches a **positive finite constant**, convergence is **quadratic**, and therefore also superlinear.

Even a fixed ratio of 0.01 is linear: it is small, but it does not tend to zero. A ratio below 1 only tells us that the error decreased in that step. Always divide the **errors**, not the guesses x_k.

In practice: a short table suggests a pattern; the update rule explains whether it continues. If x^* is unknown, monitor the residual $|f(x_k)|$ and step size $|x_{k+1} - x_k|$ together. A small step alone may mean the algorithm is stuck. Ratios computed from errors near floating-point precision are unreliable.

Section 3.19 Questions to Think at Home

1. Why can a mathematically correct optimization formula still lead to a poor computational implementation?
2. Why is a closed-form solution useful even when numerical optimization will eventually be used?
3. Why is bisection more reliable than Newton's method, and what is the computational cost of that reliability?
4. What additional information does Newton's method use compared with bisection?
5. Why can different starting values lead to different solutions for the same objective function?
6. What role does the step size play in gradient descent?
7. Why might poor scaling make optimization difficult?
8. Why is solving

$$H\boldsymbol{d} = -\nabla Q$$

usually preferable to explicitly computing H^{-1}?

9. Why can BFGS be useful when the Hessian is expensive to compute?
10. Why might a metaheuristic algorithm deliberately accept a worse solution?
11. When would you prefer simulated annealing over a deterministic local optimizer?
12. What is the main difference between simulated annealing, genetic algorithms, and particle swarm optimization?
13. Why does fewer iterations not necessarily imply lower computational cost?
14. How are optimization, likelihood estimation, and statistical inference connected?

Section 3.20 Takeaways

- Many statistical estimators are solutions to optimization problems.
- Closed-form solutions are convenient but often unavailable.
- Numerical optimization connects statistical theory with practical computation.
- Root finding is closely related to optimization.
- Bisection is reliable but relatively slow.
- Newton's method is fast near a solution but uses derivative information.
- The secant method approximates derivative information numerically.
- Multivariate optimization replaces derivatives with gradients and Hessians.
- Gradient descent uses local first-order information.
- Newton's method uses curvature information through the Hessian.
- BFGS approximates curvature without repeatedly computing the full Hessian.
- Scaling can strongly affect computational performance.
- Deterministic methods are usually efficient for smooth, well-behaved objectives.
- Metaheuristic methods trade speed and predictability for broader exploration.
- Simulated annealing can escape local optima by occasionally accepting worse solutions.
- Genetic algorithms and particle swarm optimization use populations of candidate solutions.
- There is no universally best optimization algorithm.
- A good computational method balances accuracy, stability, speed, and exploration.

Section 3.21 Optimal topic: Real-World Applications

The applications in this chapter illustrate two uses of optimization: **estimating unknown parameters from data** and **choosing an action under practical constraints**. We will connect these ideas to drug development, investment portfolios, factory scheduling, and advertising budgets. The examples use simplified models of these real applications.

Example 6: Biomedical: Finding a Curve That Fits the Measurements

Biomedical: Finding a Curve That Fits the Measurements
The situation. A research team measures drug concentration and the improvement in a biomarker. At low concentrations, more drug may produce a much larger response. At high concentrations, the response may level off. The team wants a curve that describes this pattern.
The computer's job: adjust the curve until its predictions are close to the observed dots.
The following measurements are invented for illustration.

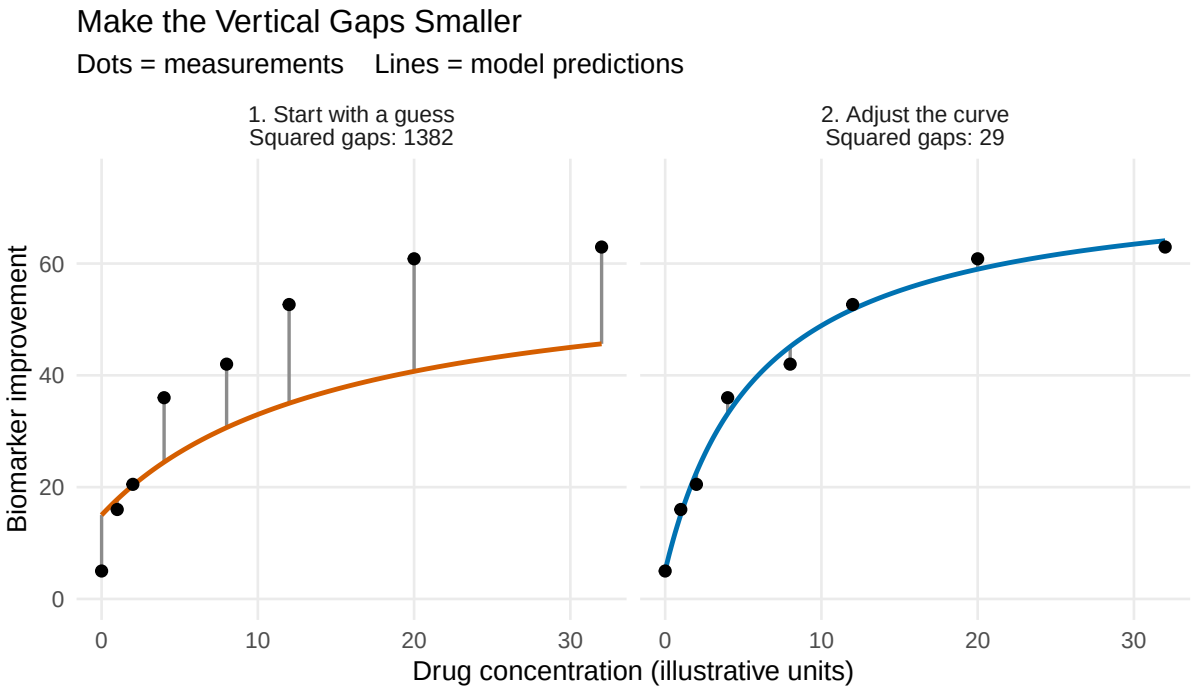


Figure 3.1: The same illustrative measurements, before and after fitting. Vertical gaps show prediction errors. Optimization reduces the sum of their squares from about 1382 to 29.

Read the picture. The dots stay fixed. The optimizer changes the curve's starting height, upper level, and how quickly it bends. Shorter vertical gaps mean better predictions of these measurements.

Decision variables. Choose the three parameters of the curve

$$m(c; E_0, E_{\max}, EC_{50}) = E_0 + \frac{E_{\max} c}{EC_{50} + c}.$$

- E_0 is the starting height (response at concentration zero).
- $E_{\max}$ is the largest increase above that starting height.
- EC_{50} is the concentration at which half of that increase is reached.

The eight concentrations c_i and measured responses y_i in the figure are **fixed data**, not quantities the optimizer can change.

Objective function. Minimize the sum of squared prediction errors:

$$\min_{E_0, E_{\max}, EC_{50}} Q(E_0, E_{\max}, EC_{50}) = \sum_{i=1}^{8} \left[y_i - E_0 - \frac{E_{\max} c_i}{EC_{50} + c_i} \right]^2.$$

Constraints. For this increasing, saturating response curve, require

$$E_0 \in \mathbb{R}, \qquad E_{\max} > 0, \qquad EC_{50} > 0.$$

We impose no sign restriction on the baseline E_0. In the R code, setting $E_{\max} = e^{\theta_2}$ and $EC_{50} = e^{\theta_3}$ enforces positivity automatically, while $E_0 = \theta_1$ remains unrestricted.

Optimal solution (numerical). For the illustrative measurements, the fitted parameters are approximately

$$(\widehat{E}_0, \widehat{E}_{\max}, \widehat{EC}_{50}) = (5.0815,\ 70.0297,\ 5.9845).$$

Thus, the fitted curve and its objective value are

$$\widehat{m}(c) = 5.0815 + \frac{70.0297c}{5.9845 + c}, \qquad Q(\widehat{E}_0, \widehat{E}_{\max}, \widehat{EC}_{50}) \approx 28.8719.$$

The minimum found numerically corresponds to the small squared gaps in the right panel. Because this is a nonlinear fitting problem, BFGS convergence alone does not certify a global minimum; different starting values can lead to different results.

Why an algorithm? Moving one parameter changes many predictions at once. The parameters enter nonlinearly, so ordinary linear regression cannot estimate all three together. BFGS, used above, repeatedly proposes changes that reduce the total squared gap.

Exposure-response models help researchers plan further drug studies; dose selection also requires safety information and uncertainty estimates. Application context: FDA exposure-response guidance.

Try explaining it: In the right panel, what has changed: the measurements, the model parameters, or both?

Example 7: Finance: Splitting Money Between Two Funds

Finance: Splitting Money Between Two Funds

The situation. Suppose a fund manager has $10,000 to invest. Begin with just two funds and the following illustrative annual estimates:

Choice	Expected return	Volatility (standard deviation)
Fund A	8%	20%
Fund B	4%	6%

Fund A offers more expected return, but its returns vary more. Assume a return correlation of 0.10. The manager wants **the highest expected return while keeping portfolio volatility at or below 10%**. Volatility measures variation; a 10% limit does not cap possible losses at 10%.

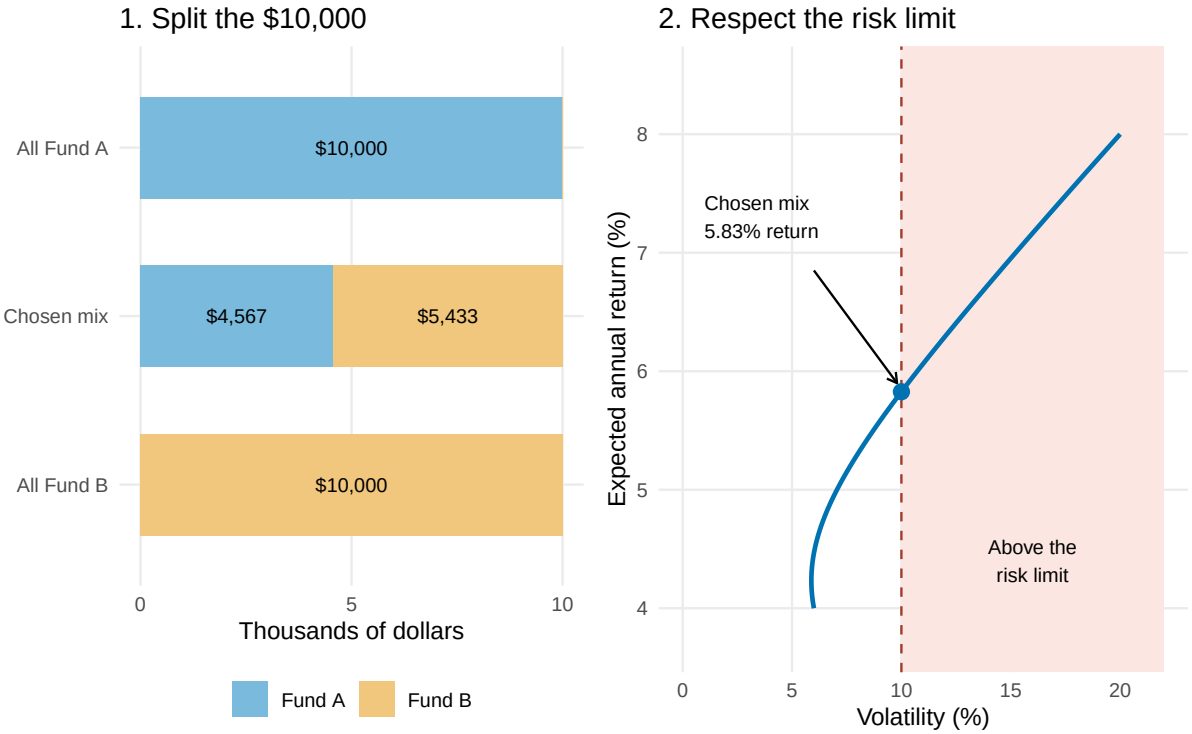


Figure 3.2: Left: three ways to split the same $10,000. Right: each point on the curve is a possible split. The selected mix has the highest expected return among points at or left of the 10% volatility limit. All inputs are illustrative.

Read the picture. Moving more money to Fund A moves us toward higher expected return, but eventually crosses the dashed risk limit. The chosen mix puts about **$4,567 in A and $5,433 in B**, for an expected return of **5.83%** under these assumptions. All-A has a higher expected return, but is outside the allowed region.

Decision variable. Let w be the fraction of the $10,000 invested in Fund A. The remaining fraction, $1 - w$, is invested in Fund B.

Objective function. Maximize the expected annual return rate:

$$\max_{w} \; R(w) = 0.08w + 0.04(1 - w) = 0.04 + 0.04w.$$

Constraints. All money is invested, with no borrowing or short selling, and portfolio volatility must not exceed 10%:

$$0 \le w \le 1, \qquad \sigma^2(w) \le 0.10^2 = 0.01.$$

The weights w and $1 - w$ already sum to one, so they enforce the $10,000 budget. Using the two standard deviations and the correlation of 0.10, the variance is explicitly

$$\begin{aligned}\sigma^2(w) &= (0.20w)^2 + [0.06(1-w)]^2 \\ &\quad + 2(0.10)(0.20)(0.06)w(1-w) \\ &= 0.0412w^2 - 0.0048w + 0.0036.\end{aligned}$$

Optimal solution. Expected return increases with w, so choose the largest feasible weight. It lies on the risk boundary:

$$0.0412w^2 - 0.0048w + 0.0036 = 0.01.$$

The positive root gives

$$w^* \approx 0.4566657, \qquad 1 - w^* \approx 0.5433343.$$

Invest approximately **$4,566.66 in Fund A and $5,433.34 in Fund B**. At the optimum,

$$R(w^*) \approx 0.0582666 = 5.82666\%, \qquad \sigma(w^*) = 10\%.$$

Equivalently, this return corresponds to an expected annual gain of about **$582.67** on the $10,000 investment. This is the unique global maximum: within $0 \le w \le 1$, every feasible weight lies between zero and w^*, and $R(w)$ is strictly increasing.

Why an algorithm? In this two-fund example, `uniroot()` locates the boundary $\sigma(w) = 0.10$. With 20 funds, sector limits, and holding limits, many weights must be chosen together. Constrained portfolio solvers handle those interacting choices. Application context: MOSEK portfolio optimization cookbook.

Try explaining it: If the dashed risk limit moved to the right, would the best allowed mix contain more or less of Fund A?

Example 8: Industry: Finish the Same Orders Earlier

Industry: Finish the Same Orders Earlier

The situation. A small factory has three orders, A, B, and C. Every order must go through **Cutting, then Finishing**. Each machine handles one order at a time. These illustrative processing times stay the same in both schedules:

Order	Cutting	Finishing
A	4 hours	1 hour
B	1 hour	4 hours
C	2 hours	2 hours

The computer's job: choose the order of work so that all three orders finish as early as possible.

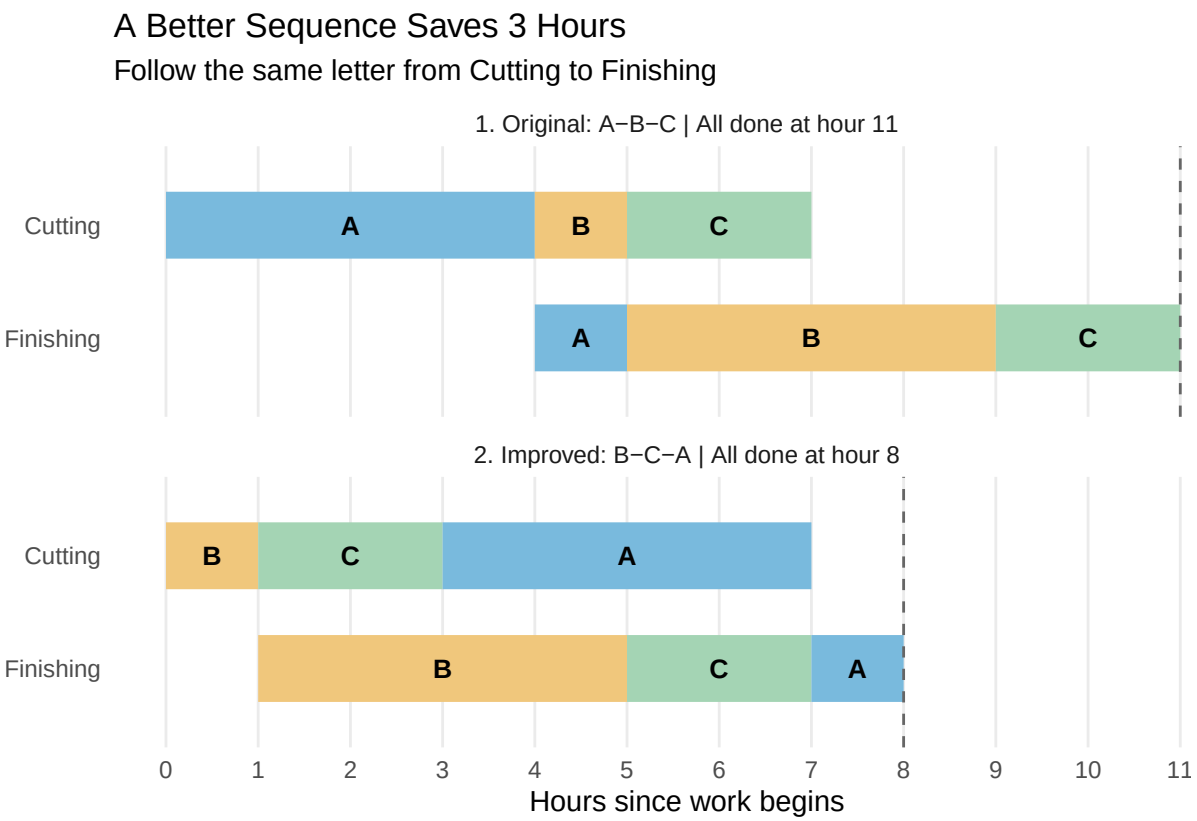


Figure 3.3: Same orders, same machines, same processing times. Changing the sequence reduces the last completion time from 11 to 8 hours. Empty horizontal space is idle time.

Read the picture. In the top schedule, Finishing waits four hours for A. In the bottom schedule, the short cutting step for B lets Finishing start after just one hour. The machines then overlap useful work more effectively. **No machine runs faster; the sequence is better.**
Decision variables. Choose a start time S_{jm} for each order $j \in \{A, B, C\}$ on each machine $m \in \{C, F\}$, where C means Cutting and F means Finishing. Also choose $C_{\max}$, the time by which all orders must be complete. Let p_{jm} denote the fixed processing times in the table. All orders are available at time zero, and each operation runs without interruption once it starts.
Objective function. Minimize the last completion time (the makespan):

$$\min_{\{S_{jm}\},\, C_{\max}} \; C_{\max}.$$

Constraints. The start times must satisfy four rules:

1. **Work cannot start before time zero:** for every order j and machine m,

$$S_{jm} \geq 0.$$

2. **Cutting precedes Finishing:** for each order j,

$$S_{jF} \geq S_{jC} + p_{jC}.$$

3. **A machine cannot process two orders at once:** for every pair of different orders j, k and each machine m,

$$S_{jm} + p_{jm} \leq S_{km} \quad \text{or} \quad S_{km} + p_{km} \leq S_{jm}.$$

   In words, either j finishes before k starts, or k finishes before j starts. Choosing between these alternatives determines the job sequence.

4. **Every order finishes by $C_{\max}$:** for each order j,

$$S_{jF} + p_{jF} \leq C_{\max}.$$

Optimal solution. One optimal sequence is **B, then C, then A** on both machines, with the following start and end times:

Order	Cutting: start to end	Finishing: start to end
B	0 to 1	1 to 5
C	1 to 3	5 to 7
A	3 to 7	7 to 8

All times are in hours. This feasible schedule achieves

$$C^*_{\max} = 8 \text{ hours}.$$

Why is 8 hours globally optimal? Finishing requires $1 + 4 + 2 = 7$ hours of work on a single machine. It cannot begin before hour 1, because even the shortest cutting operation takes 1 hour. Therefore every feasible schedule satisfies $C_{\max} \geq 1 + 7 = 8$. The schedule above reaches that lower bound, so no schedule can finish earlier. The original A-B-C schedule takes 11 hours.
For three orders, all six common orderings can also be checked. Larger factories require more systematic search; 20 jobs already give $20! \approx 2.43 \times 10^{18}$ possible orderings on one machine.
Why an algorithm? Constraint programming and mixed-integer optimization search subject to the rules. Simulated annealing and genetic algorithms can also explore valid schedules, though they do not certify the best possible result. This is a discrete choice problem: swapping two jobs is not a gradient step. Application context: OR-Tools job-shop scheduling.
Try explaining it: Find B in both panels. Why does doing its short cutting step early help the whole factory?

An online retailer has **$100,000** for search advertising and social media. Start with a familiar decision: should it spend $50,000 on each, or split the money differently?

Example 9: Business Analytics: Same Budget, More Orders

Business Analytics: Same Budget, More Orders

The situation. The first dollars spent on search work well, but that channel soon reaches many of the customers it can reach. Social media responds more gradually. We use two invented response curves to make the allocation problem visible; these are not measured campaign results.

First, look at how each channel responds to spending. A steep curve means that another dollar produces a relatively large gain. A flat curve means that another dollar adds little.

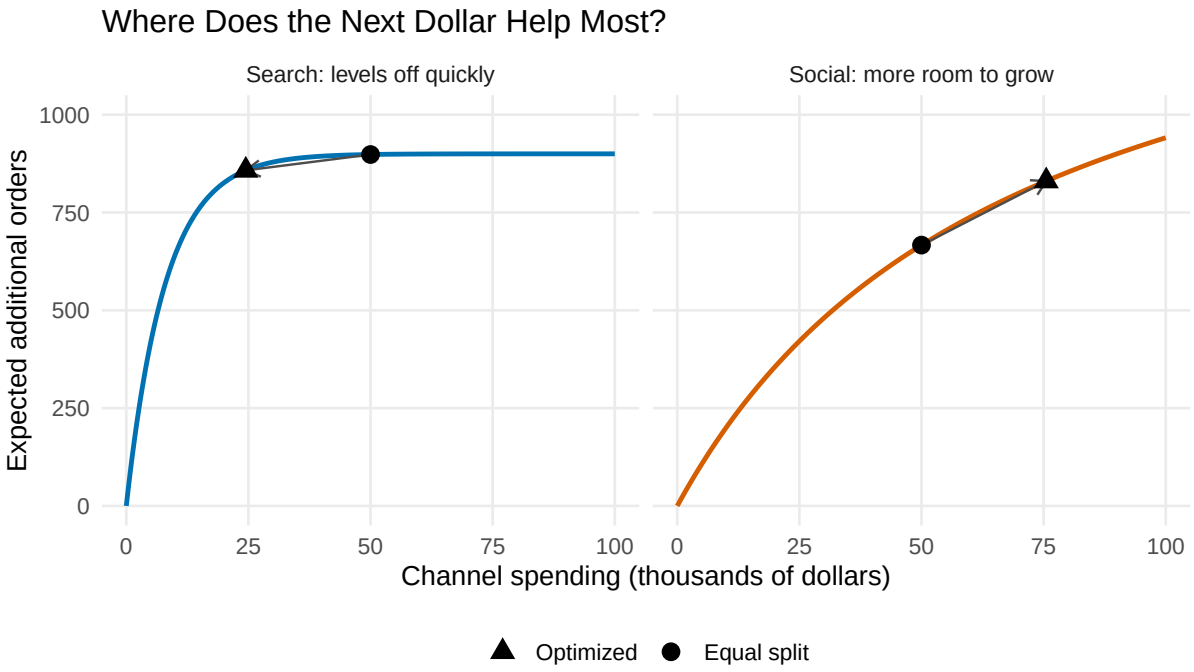


Figure 3.4: Illustrative response curves. Circles mark equal spending; triangles mark the optimized allocation. Arrows show the change: reduce search spending where the curve is flat and increase social spending where more gains remain.

Read the picture. At the equal split, search is already nearly flat. Moving some of that money to social loses relatively few search orders and gains more social orders. The optimizer keeps reallocating until the next dollar has the same marginal benefit in both channels.

Now compare the decisions. The total budget stays at $100,000 in both rows. Only its allocation changes.

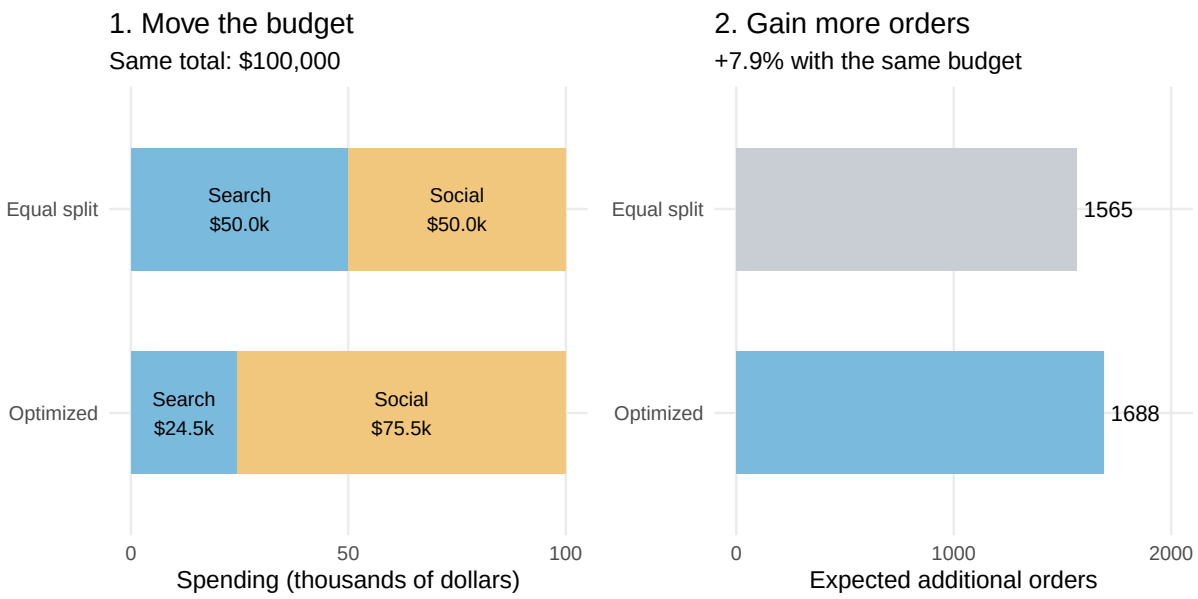


Figure 3.5: A visual before-and-after comparison. The optimized split spends about $24,461 on search and $75,539 on social, producing about 123 more expected orders with the same budget under the illustrative model.

The change is from about **1,565 to 1,688 expected additional orders**: approximately **123 extra orders, or 7.9%**, without increasing spending.

Decision variables. Let s and u be spending on search and social advertising, respectively, **in thousands of dollars**. For example, $s = 25$ means $25,000 spent on search.

Objective function. Maximize the expected number of additional orders:

$$\max_{s,u} \; F(s,u) = 900(1 - e^{-s/8}) + 1600\frac{u}{70+u}.$$

The first term is the search contribution and the second is the social contribution. We assume these contributions add without overlap and that all orders have the same value.

Constraints. Spend the full $100,000 budget, with neither channel receiving a negative amount:

$$s \geq 0, \qquad u \geq 0, \qquad s + u = 100.$$

There are no additional channel spending limits in this example. Substituting $u = 100 - s$ gives the equivalent problem used by `optimize()`:

$$\max_{0 \leq s \leq 100} \; f(s) = 900(1 - e^{-s/8}) + 1600\frac{100-s}{170-s}.$$

Optimal solution. The best allocation is approximately

$$s^* = 24.46073, \qquad u^* = 75.53927,$$

or **$24,460.73 on search and $75,539.27 on social**. Its objective value is

$$F(s^*, u^*) \approx 1688.1476 \text{ expected additional orders.}$$

For comparison, equal spending gives $F(50, 50) \approx 1564.9293$. The optimal allocation therefore adds about **123.2183 expected orders**, a **7.87%** increase with the same budget.

Why is this allocation optimal? Both response curves are strictly concave, so the feasible problem has a unique global maximum. The optimum lies inside $0 < s < 100$ and solves

$$f'(s) = 112.5e^{-s/8} - \frac{112000}{(170 - s)^2} = 0.$$

At s^*, both channels have the same marginal gain: approximately 5.2876 additional orders per \$1,000 of spending. Shifting a small amount of money from one channel to the other can no longer improve the total.

Why an algorithm? Setting $u = 100 - s$ leaves one adjustable number. `optimize()` searches for its best value. Here the total-response curve is strictly concave, so there is a unique maximum. With many channels and spending limits, this becomes a multivariate constrained problem.

Marketing tools such as Google's Meridian apply this response-curve idea to budget allocation. In practice, the curves must be estimated and validated; the optimizer's result depends on their accuracy. Application context: Meridian budget optimization.

Try explaining it: Point to the two equal-length budget bars. Where do the extra orders come from if no extra money is spent?

Examples are adapted from the following sources:

- Peng, R.D. Advanced Statistical Computing.

Section 4

Generating Random Variables

In Chapter 3: Optimization, we used computation to estimate parameters from data. Here we go in the other direction: **given a probability model, how do we generate observations from it?**

We start with uniform random numbers, use them to construct one-dimensional distributions, and then build mixtures and random vectors. At each step, we ask the same three questions: **What is the construction? Why does it have the desired distribution? How can we check it?**

Section 4.1 Overview

Generating random variables is the first step in a simulation study. Once we can generate data with a known truth, we can study an estimator's bias, a confidence interval's coverage, or a test's power.

The main methods in this chapter follow a common pattern:

Available building block	Construction	Target
Uniform probability levels	Apply a quantile function	A specified univariate distribution
An easy proposal distribution	Accept or reject proposals	A distribution with a known density
Familiar independent variables	Transform or combine them	Related distributions, sums, and mixtures
Independent standard normals	Apply a matrix or normalize lengths	Correlated vectors and random directions

A pseudorandom number generator produces a deterministic sequence from an initial state. A well-designed generator makes that sequence useful as an approximation to independent random draws. In the mathematical arguments below, we work with ideal independent uniforms; software approximates them with finite precision.

> **i** Reproducibility
>
> In R, use `set.seed()` to reproduce an example. The same seed, generator settings, software environment, and sequence of calls reproduce the same simulation. The seed identifies a computation; it does not certify the probability model.

```r
set.seed(8670)
u_first <- runif(5)

set.seed(8670)
u_repeated <- runif(5)

rbind(first = u_first, repeated = u_repeated)
#>               [,1]      [,2]     [,3]      [,4]      [,5]
#> first    0.3114911 0.7752026 0.874258 0.7551533 0.9447904
#> repeated 0.3114911 0.7752026 0.874258 0.7551533 0.9447904
identical(u_first, u_repeated)
#> [1] TRUE
```

Set the seed **before a simulation experiment**, then let the generator advance. Resetting the same seed inside every replication creates repeated copies of the same random input.
For a reproducible analysis, also record `RNGkind()` and `sessionInfo()`. For parallel work, use a framework that manages separate random-number streams, such as the streams supported by R's `parallel` package. See the R documentation on random-number generation.

Example 1: Sampling from a Finite Population

Sampling with replacement allows an observation to appear more than once. Sampling without replacement makes later draws depend on which observations have already been selected.

```r
set.seed(8670)
observed_waits <- c(2, 3, 3, 5, 7, 12)  # illustrative waiting times

sample(observed_waits, size = 6, replace = TRUE)
#> [1] 12  5  3 12 12  3
sample(observed_waits, size = 6, replace = FALSE)
#> [1] 12  3  2  7  3  5
```

Connection to the Bootstrap

The empirical distribution places probability $1/n$ on each of the n observed records, counting repeated values separately. Drawing n times independently from this distribution is the basic **nonparametric bootstrap** resampling step. Sampling all n records without replacement only permutes them, so the sample mean cannot change.
This is our first example of an important principle: **the sampling rule is part of the statistical model.**

R Distribution Functions: A Reference

R uses four related prefixes. For example, `dnorm()` evaluates a normal density, `pnorm()` evaluates its CDF, `qnorm()` evaluates its quantile function, and `rnorm()` generates draws.

Distribution	Density or PMF	CDF	Quantile	Generator	Parameters to check
Uniform	`dunif`	`punif`	`qunif`	`runif`	`min`, `max`
Normal	`dnorm`	`pnorm`	`qnorm`	`rnorm`	`mean`, **`sd`**
Exponential	`dexp`	`pexp`	`qexp`	`rexp`	**`rate`**
Gamma	`dgamma`	`pgamma`	`qgamma`	`rgamma`	`shape`, `rate` or `scale`
Beta	`dbeta`	`pbeta`	`qbeta`	`rbeta`	`shape1`, `shape2`
Binomial	`dbinom`	`pbinom`	`qbinom`	`rbinom`	`size`, `prob`
Geometric	`dgeom`	`pgeom`	`qgeom`	`rgeom`	`prob`; counts failures
Poisson	`dpois`	`ppois`	`qpois`	`rpois`	`lambda`
Negative binomial	`dnbinom`	`pnbinom`	`qnbinom`	`rnbinom`	`size`, `prob` or `mu`
Chi-square	`dchisq`	`pchisq`	`qchisq`	`rchisq`	`df`
Student's t	`dt`	`pt`	`qt`	`rt`	`df`
F	`df`	`pf`	`qf`	`rf`	`df1`, `df2`
Lognormal	`dlnorm`	`plnorm`	`qlnorm`	`rlnorm`	`meanlog`, `sdlog`

See R's distribution reference for the available families.

Check the Parameterization

`rnorm(n, sd = 4)` has variance 16. An exponential rate of 4 gives mean $1/4$. In the gamma family, `scale = 1 / rate`. A lognormal `meanlog` is the mean of $\log X$, not the mean of X. A generator can run without errors while representing the wrong model if its parameterization is misunderstood.

Section 4.2 Inverse Transformation Method

Imagine dividing the interval $(0, 1)$ into probability levels. A uniform draw selects a level without favoring one part of that interval. The quantile function converts that probability level into a value on the scale of the target distribution.

Definition 1: Generalized Inverse CDF

For any CDF F, define

$$F^{-1}(u) = \inf\{x \in \mathbb{R} : F(x) \geq u\}, \qquad 0 < u < 1.$$

It is the first value of x at which accumulated probability reaches the level u.

Theorem 1: Inverse Transformation

If $U \sim \text{Unif}(0,1)$, then $X = F^{-1}(U)$ has CDF F. Independent uniforms transformed separately give independent observations from F.

Proof. The generalized inverse satisfies

$$\begin{aligned} P\{F^{-1}(U) \le x\} &= P\{U \le F(x)\} \\ &= F(x). \end{aligned}$$

Thus the generated values have the required CDF.

Use an Inequality in the Definition

The condition is $F(x) \ge u$, not $F(x) = u$. A discrete CDF jumps over some probability levels. The generalized inverse still assigns every $0 < u < 1$ to an outcome.

Theorem 2: Probability Integral Transform

If X has a **continuous** CDF F, then $F(X) \sim \text{Unif}(0,1)$. This is the reverse direction of the inverse-transform construction.

A Connection to Model Checking

Under a correctly specified continuous predictive distribution, transformed observations have a uniform marginal distribution. Dependence must be checked separately. If F is estimated from the same observations, formal tests need to account for that estimation.
For discrete X, $F(X)$ is not uniform. An optional randomized version spreads each probability mass over its corresponding interval:

$$U = F(X^-) + V\{F(X) - F(X^-)\},$$

where V is an independent uniform and $F(X^-)$ is the CDF just below X.

Algorithm 1: Inverse Transformation

Input: a target CDF F and sample size n. **Output:** n independent draws from F.

1. **Prepare the map.** Obtain $F^{-1}(u)$ analytically or numerically. For a continuous increasing CDF, solve $u = F(x)$ for x.
2. **Draw probability levels.** Generate independent $U_1, \ldots, U_n \sim \text{Unif}(0,1)$.
3. **Convert to observations.** Return $X_i = F^{-1}(U_i)$ for each i. The output is on the target's scale, not the probability scale.

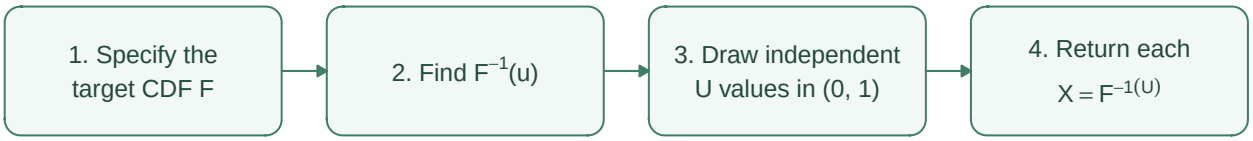


Figure 4.1: Inverse transformation: prepare one map, then apply it to independent uniforms.

4.2.1 Continuous Distributions

Example 2: An Increasing Density on the Unit Interval

Consider

$$f(x) = 3x^2, \qquad 0 < x < 1.$$

First integrate the density, then solve for the observation:

$$F(x) = \int_0^x 3t^2\,dt = x^3, \qquad u = x^3 \implies x = u^{1/3}.$$

Thus $X = U^{1/3}$. For one draw, $u = 0.125$ gives $x = 0.5$; checking $F(0.5) = 0.125$ confirms the direction of the transformation.

```
u <- 0.125              # a given probability level
x <- u^(1 / 3)          # convert it to an observation
c(x = x, F_x = x^3)
#>     x   F_x
#> 0.500 0.125
```

Why do the observations concentrate near 1? Equal-width intervals near 1 contain more probability because the CDF rises faster there. The inverse maps more uniform probability levels into those intervals.

The arrows in Figure 4.2 show what the inverse does: choose a height u, move across to the CDF, then read the corresponding x on the horizontal axis.

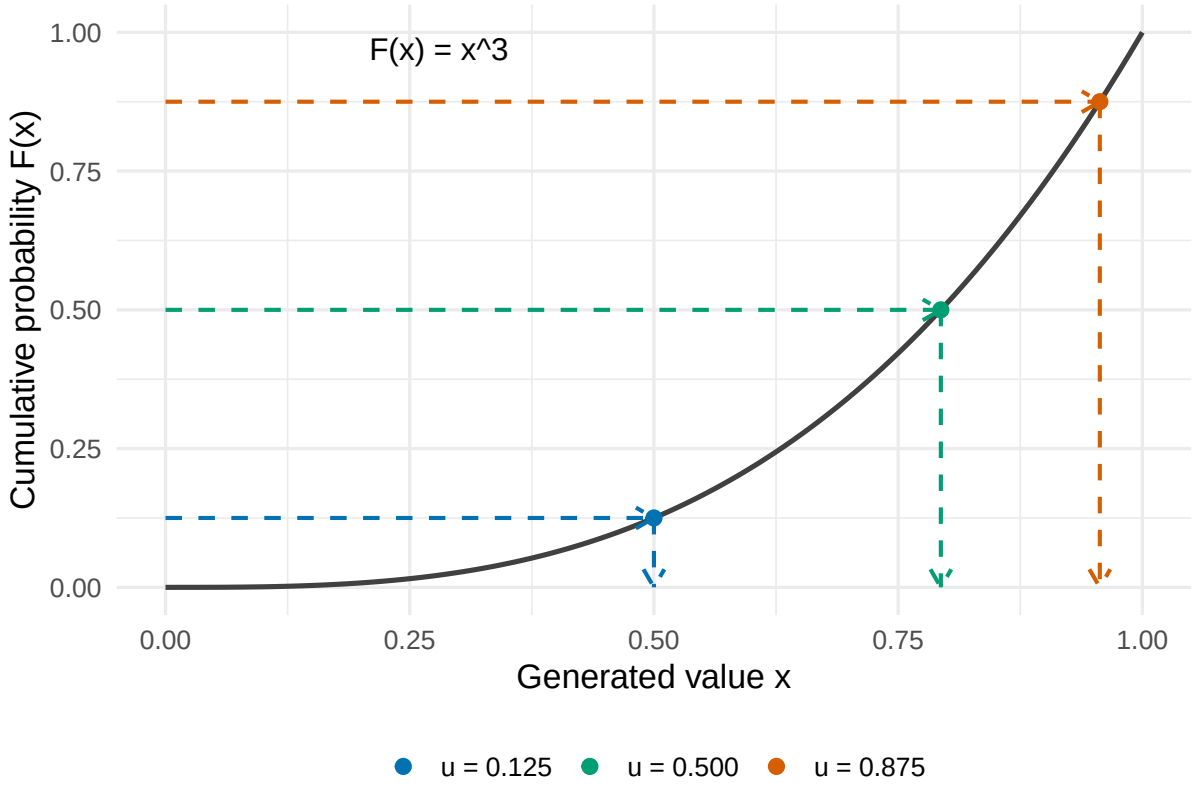


Figure 4.2: The inverse CDF converts a probability level into an observation. Each horizontal arrow starts at u; the downward arrow lands at x = u^(1/3). The dashed guides are a construction, not additional random draws.

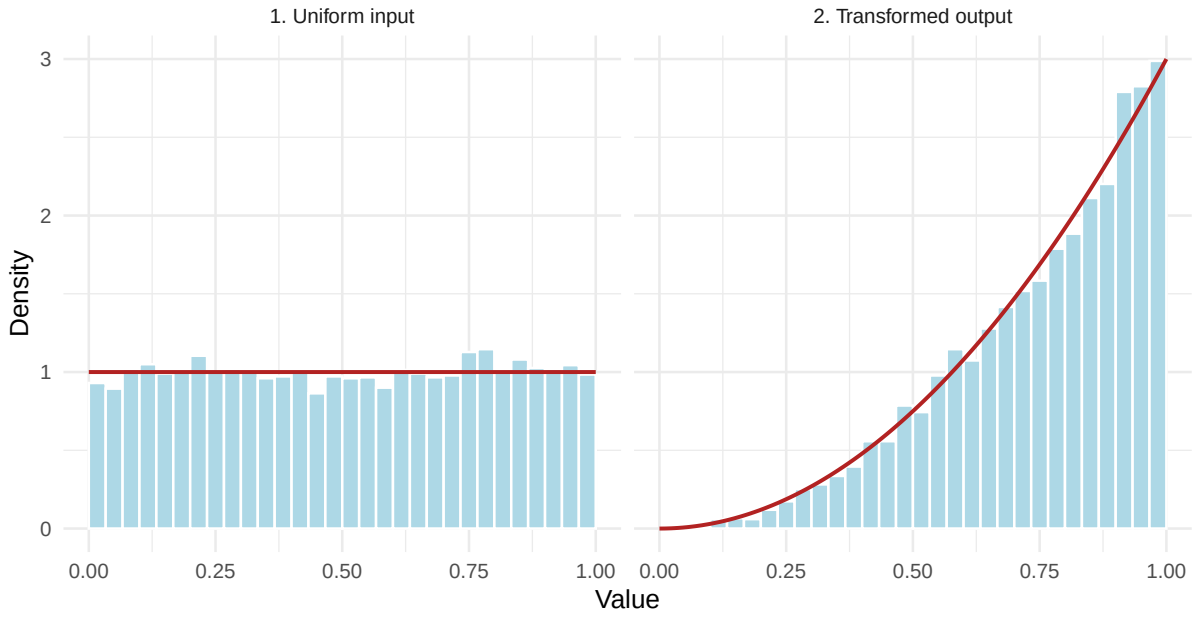


Figure 4.3: Left: uniform probability levels. Right: mapping them through the quantile function produces values with the increasing target density.

Here $\mathbb{E}(X) = 3/4$ and $\text{Var}(X) = 3/80$. A simulation provides an independent numerical check.

```
knitr::kable(booktabs = TRUE,
  data.frame(
    quantity = c("Mean", "Variance"),
    simulated = c(mean(x_power), var(x_power)),
    theoretical = c(3 / 4, 3 / 80)
  ),
  digits = 4
)
```

quantity	simulated	theoretical
Mean	0.7531	0.7500
Variance	0.0372	0.0375

Agreement should be approximate. Random samples are not required to reproduce population moments exactly.

Example 3: Exponential Random Variables

For $T \sim \text{Exp}(\lambda)$ with **rate** $\lambda > 0$,

$$F(t) = 1 - e^{-\lambda t}, \qquad F^{-1}(u) = -\frac{\log(1-u)}{\lambda}.$$

Hence we can generate waiting times using

$$T = -\frac{\log(1-U)}{\lambda}.$$

Since U and $1-U$ have the same distribution, $-\log(U)/\lambda$ also works. Both formulas have the same marginal distribution.

```
set.seed(8670)
lambda <- 0.7
u <- runif(10000)
waiting_time <- -log1p(-u) / lambda

c(
  simulated_mean = mean(waiting_time),
  theoretical_mean = 1 / lambda,
  simulated_prob_above_3 = mean(waiting_time > 3),
  theoretical_prob_above_3 = exp(-lambda * 3)
)
#>           simulated_mean         theoretical_mean   simulated_prob_above_3
#>                1.4497408                1.4285714                0.1247000
#> theoretical_prob_above_3
#>                0.1224564
```

`log1p(-u)` computes $\log(1-u)$ accurately when u is close to zero. This connects random-number generation to the numerical stability issues discussed earlier.

Equal Distributions Do Not Mean Independent Draws

The formulas $-\log(U)/\lambda$ and $-\log(1-U)/\lambda$ have the same marginal distribution. Applying both to the **same** U produces dependent outputs. Use independent uniforms when independence is required.

Optional Connection: Survival and Waiting Times

Exponential waiting times model the time between events in a homogeneous Poisson process. The constant rate is a substantive assumption: it implies a memoryless waiting time. Equipment aging or time-varying arrival rates may require a different distribution.
More generally, if a nonnegative event time has continuous cumulative hazard $H(t)$ and survival function $S(t) = e^{-H(t)}$ tending to zero, then

$$T = H^{-1}\{-\log(U)\}.$$

This turns survival-model simulation into an inversion problem. For example, the Weibull cumulative hazard $H(t) = (t/b)^a$ gives $T = b\{-\log(U)\}^{1/a}$.

Exercise 4.1 (In Class: Rate or Mean?)
Multiple choice. To generate exponential waiting times with mean 5, use **(A)** `rexp(n, rate = 5)` or **(B)** `rexp(n, rate = 1 / 5)`.
If the rate doubles, does the mean waiting time double or halve? Explain in one sentence.

Solution 4.1

1. **Translate the mean into a rate.** For an exponential variable, $E(X) = 1/\lambda$. Set $1/\lambda = 5$, giving $\lambda = 1/5$.
2. **Match the R argument.** Use `rate = 1 / 5`, so the correct choice is **B**.
3. **Check the direction.** Doubling the rate gives mean $1/(2/5) = 2.5$. Multiplying waiting times by two would give mean 10 instead.

Example 4: Numerical Inversion: Using Root Finding

An inverse may be inconvenient algebraically even when the CDF is easy to evaluate. For each u, solve

$$F(x) - u = 0.$$

For example, consider

$$F(x) = \frac{x + x^3}{2}, \quad 0 \le x \le 1, \qquad f(x) = \frac{1 + 3x^2}{2}.$$

The CDF is strictly increasing, so each $u \in (0, 1)$ has exactly one root in $(0, 1)$. We can use

the bracketing methods from the optimization chapter.

```
custom_cdf <- function(x) (x + x^3) / 2  # used only on [0, 1]

custom_quantile <- function(u) {
  vapply(u, function(ui) {
    uniroot(
      function(x) custom_cdf(x) - ui,
      interval = c(0, 1), tol = 1e-10
    )$root
  }, numeric(1))
}

set.seed(8670)
u <- runif(2000)
x_numeric <- custom_quantile(u)

c(
  largest_cdf_residual = max(abs(custom_cdf(x_numeric) - u)),
  simulated_mean = mean(x_numeric),
  theoretical_mean = 5 / 8
)
#> largest_cdf_residual       simulated_mean     theoretical_mean
#>         4.630307e-11         6.305761e-01         6.250000e-01
```

There are two distinct errors: the root solver introduces **numerical error**, while finite simulation introduces **Monte Carlo error**. Tightening `tol` reduces the former; increasing the number of independent draws reduces the latter. Neither repairs a wrong CDF.

Example 5: Optional: A Truncated Normal Distribution

Suppose only values in (a, b) are eligible for a study. If F is continuous and $F(b) > F(a)$, then the conditional CDF within this interval is

$$P(X \leq x \mid a < X < b) = \frac{F(x) - F(a)}{F(b) - F(a)}.$$

Consequently,

$$X = F^{-1}\left[F(a) + U\{F(b) - F(a)\}\right]$$

generates from the truncated distribution.

```r
set.seed(8670)
a <- 1
b <- 2
u <- runif(5000)
truncated_normal <- qnorm(pnorm(a) + u * (pnorm(b) - pnorm(a)))

c(
  simulated_mean = mean(truncated_normal),
  theoretical_mean = (dnorm(a) - dnorm(b)) / (pnorm(b) - pnorm(a)),
  fraction_in_interval = mean(truncated_normal > a & truncated_normal < b)
)
#>       simulated_mean     theoretical_mean fraction_in_interval
#>             1.388024             1.383169             1.000000
```

Truncation, Censoring, and Extreme Tails

Truncation changes which individuals enter the dataset. Censoring instead records incomplete information about some individuals who remain in the dataset, such as knowing that a failure time exceeds the study duration. These require different simulation rules. The direct CDF subtraction above is suitable for this moderate interval; extreme tails require numerically stable tail probabilities or a specialized truncated-distribution generator.

4.2.2 Discrete Distributions

For ordered support values $x_1 < x_2 < \cdots$, the quantile rule selects the first value whose cumulative probability reaches u:

$$X = x_j \quad \text{when} \quad F(x_{j-1}) < U \leq F(x_j).$$

Each outcome receives a subinterval of $(0, 1)$ whose length equals its probability.

Example 6: Bernoulli and Categorical Outcomes

For $X \sim \text{Bernoulli}(p)$, the generalized inverse gives

$$X = \mathbf{1}\{U > 1 - p\}.$$

The threshold divides the unit interval into pieces of lengths $1 - p$ and p.
The same idea applies to three categories with probabilities 0.2, 0.5, and 0.3. In Figure 4.4, interval **length** represents probability. A uniform draw at $u = 0.62$ selects the middle category because $0.2 < 0.62 \leq 0.7$.

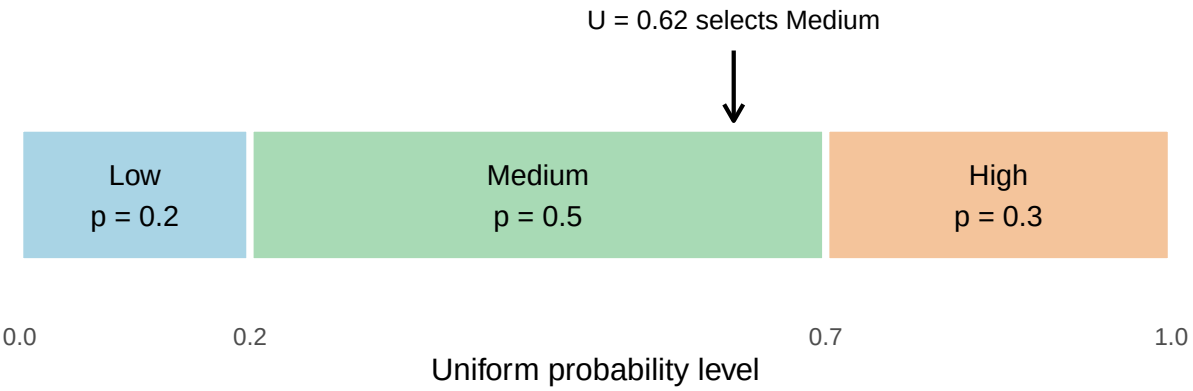


Figure 4.4: Discrete generation allocates the unit interval to outcomes. The middle category receives half of the interval, so it is selected with probability 0.5. Boundary points follow the generalized-inverse convention and have probability zero for an ideal continuous uniform.

```r
set.seed(8670)
p <- 0.4
bernoulli <- as.integer(runif(10000) > 1 - p)

labels <- c("Low", "Medium", "High")
probabilities <- c(0.2, 0.5, 0.3)
cutoffs <- cumsum(probabilities)
u <- runif(10000)

# First cutoff that reaches u: this also handles u at a cutoff.
index <- vapply(u, function(ui) which(ui <= cutoffs)[1], integer(1))
category <- factor(labels[index], levels = labels)

mean(bernoulli)
#> [1] 0.4094
prop.table(table(category))
#> category
#>    Low Medium   High
#> 0.1966 0.5008 0.3026

# Built-in alternative for the same categorical distribution.
head(sample(labels, size = 10, replace = TRUE, prob = probabilities))
#> [1] "Medium" "High"   "High"   "Medium" "Low"    "High"
```

Example 7: Geometric Random Variables

For the geometric distribution, use R's convention: X counts **failures before the first success**. Then

$$P(X = x) = p(1-p)^x, \qquad x = 0, 1, 2, \ldots,$$

and $F(x) = 1 - (1-p)^{x+1}$. Solving $F(x-1) < u \leq F(x)$ gives

$$F^{-1}(u) = \left\lceil \frac{\log(1-u)}{\log(1-p)} \right\rceil - 1.$$

```r
set.seed(8670)
p <- 0.25
u <- runif(10000)
failures <- ceiling(log1p(-u) / log1p(-p)) - 1

c(
  simulated_mean = mean(failures),
  theoretical_mean = (1 - p) / p,
  largest_difference_from_qgeom = max(abs(failures - qgeom(u, prob = p)))
)
#>                simulated_mean              theoretical_mean
#>                        3.0503                        3.0000
#> largest_difference_from_qgeom
#>                        0.0000
```

The number of **trials including the success** is $X + 1$, with mean $1/p$. Mixing up these conventions creates an off-by-one modeling error. See R's geometric distribution documentation.

Optional: Poisson Inversion

For a Poisson distribution, cumulative probabilities can be constructed from

$$p_0 = e^{-\lambda}, \qquad p_{k+1} = \frac{\lambda}{k+1} p_k, \qquad F(k+1) = F(k) + p_{k+1}.$$

Search for the first k satisfying $F(k) \geq u$. This explains an inversion algorithm, but the recurrence can underflow for large λ. In practice, use `qpois(runif(n), lambda)` or the specialized generator `rpois(n, lambda)`. An `r` function need not implement the simple algorithm used in our derivation.

The inverse-transform method is especially convenient when a quantile is available. When a density is easier to work with than its CDF, the next method gives another route.

Section 4.3 Acceptance-Rejection Method

Inverse transformation requires a usable quantile function. Acceptance-rejection requires a different resource: an easy **proposal distribution** that can cover the target.

Let f be the normalized target density and g a proposal density. We need a finite constant M such that

$$f(y) \leq M g(y)$$

everywhere on the target support. In particular, $g(y)$ must be positive wherever $f(y) > 0$.

Algorithm 2: Acceptance-Rejection Sampling

Input: target density f, proposal density g, a valid bound M, and sample size n.

1. **Check before sampling.** Verify $f(y) \le Mg(y)$ throughout the target support. Start with zero accepted values.
2. **Propose and compare.** Draw $Y \sim g$ and, independently, $U \sim \text{Unif}(0,1)$. Compute $a = f(Y)/\{Mg(Y)\}$.
3. **Keep or discard.** If $U \le a$, save Y and increase the accepted count by one. Otherwise discard Y.
4. **Repeat with fresh Y and U.** Stop only when n values have been accepted. Return those n values.

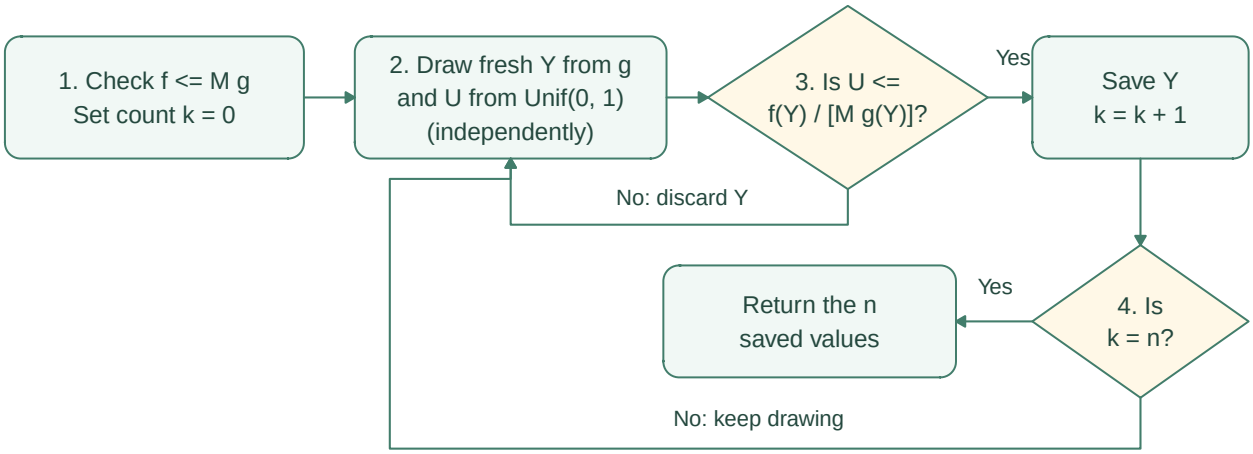


Figure 4.5: Rejection sampling: count accepted values, not attempted proposals.

Theorem 3: Distribution and Cost of Accepted Proposals

Let f and g be normalized densities with $f(y) \le Mg(y)$ throughout the target support. In the algorithm above:

- The probability of accepting each proposal is $1/M$.
- Each accepted value has density f.
- Independent proposal/decision pairs produce independent accepted observations.
- The expected number of proposals for n accepted observations is nM.

Proof. For continuous proposals,

$$P(\text{accept}) = \int g(y)\frac{f(y)}{Mg(y)}\,dy = \frac{1}{M}.$$

The density of an accepted proposal is therefore

$$f_{Y|\text{accept}}(y) = \frac{g(y)P(\text{accept} \mid Y = y)}{P(\text{accept})} = f(y).$$

Why Thinning Works

The proposal visits some regions too frequently. The acceptance rule thins those visits by a location-specific amount. After thinning, the relative frequencies match the target.
For discrete distributions, use sums instead of integrals in the proof. The number of proposals through one acceptance is geometric on $\{1, 2, ...\}$ with mean M. The realized cost varies from run to run.
This produces independent accepted draws, unlike a Markov chain method whose successive states are generally dependent.

Example 8: Beta(2,2): Choosing an Efficient Envelope

For the Beta$(2, 2)$ target,

$$f(y) = 6y(1 - y), \qquad 0 < y < 1.$$

With a uniform proposal, $g(y) = 1$. The derivative is $6 - 12y$, which is zero at $y = 1/2$; the second derivative is -12 and the endpoint densities are zero. Thus

$$M_{\min} = \max_{0<y<1} 6y(1 - y) = \frac{3}{2}.$$

The overall acceptance rate is $1/M = 2/3$. For a particular proposal y, the decision uses $f(y)/(Mg(y)) = 4y(1 - y)$:

y	u	4y(1-y)	Decision
0.2	0.5	0.64	Accept; save 0.2
0.5	0.9	1.00	Accept; save 0.5
0.9	0.5	0.36	Reject; draw a new pair

```
set.seed(8670)
y <- runif(1)                    # propose the value to keep
u <- runif(1)                    # independent decision draw
accept_prob <- 4 * y * (1 - y)   # f(y) / (M * g(y))
u <= accept_prob                 # if TRUE, save y, not u
#> [1] TRUE
```

Repeat until n values have been saved. Choosing $M = 6$ is also valid, but the overall acceptance rate falls to $1/6$.

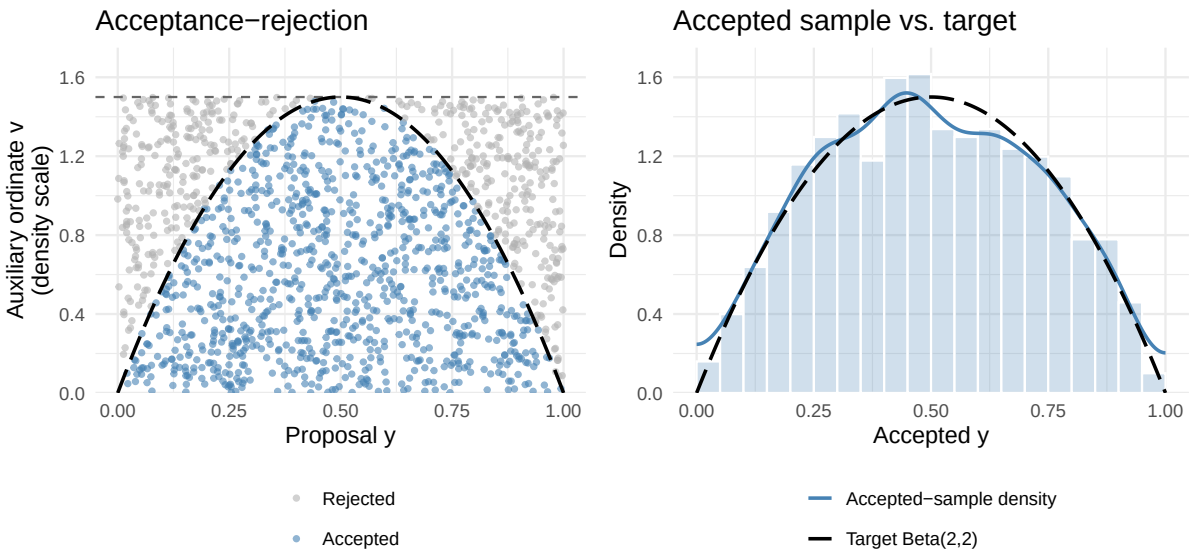


Figure 4.6: Left: each point is (Y, V), where Y is a uniform proposal and V = 1.5 U for an independent U ~ Uniform(0,1). Accept Y when V is below the normalized target f(Y); V is an auxiliary ordinate on the density scale, not a density estimate. Right: the density-normalized histogram and boundary-corrected density estimate of accepted Y values are compared with the normalized Beta(2,2) target f(y) = 6y(1-y), shown as a black dashed curve in both panels.

i What the left-hand points represent

Each point in the left panel of Figure 4.6 is (Y, V). Generate a proposal $Y \sim \text{Uniform}(0,1)$ and an independent decision variable $U \sim \text{Uniform}(0,1)$, then plot $V = Mg(Y)U = 1.5U$. Thus the plotted ordinate ranges from 0 to 1.5, while the original decision variable U ranges from 0 to 1. The two acceptance rules are equivalent:

$$V \le f(Y) \quad \Longleftrightarrow \quad U \le \frac{f(Y)}{Mg(Y)} = 4Y(1-Y).$$

Blue points satisfy this rule. **Save their horizontal coordinates Y; discard U and V.** A point's ordinate is an auxiliary random value used for the decision, not $f(Y)$, an estimated density, or an acceptance probability. The left axis therefore says **Auxiliary ordinate v (density scale)**. The right panel estimates the density of the retained Y values, so its axis says **Density**. The horizontal envelope at 1.5 is a bound on f, not a probability density with area 1.

Normalized and unnormalized targets

An **unnormalized target** is a nonnegative function $q(y)$ whose relative heights describe the desired distribution. Its total area is $Z = \int q(y)\,dy$, where $0 < Z < \infty$. The **normalized density** is $f(y) = q(y)/Z$, so $\int f(y)\,dy = 1$. Dividing by the area preserves the shape; it does not set the maximum to 1.
For the Beta(2,2) example, choose $q(y) = y(1-y)$ on $[0,1]$ and zero elsewhere. Then

$$Z = \int_0^1 y(1-y)\,dy = \left[\frac{y^2}{2} - \frac{y^3}{3}\right]_0^1 = \frac{1}{6}, \qquad f(y) = \frac{q(y)}{1/6} = 6y(1-y).$$

Quantity	Unnormalized target q	Normalized density $f = q/Z$
Function on $[0,1]$	$y(1-y)$	$6y(1-y)$
Total area	$1/6$	1
Maximum at $y = 1/2$	$1/4$	$3/2 = 1.5$

Why use the normalized version in this figure? The right-hand histogram uses bar height count/($n_{\text{accepted}} \times$ bin width), so its bar areas sum to 1; the smooth density estimate also has area 1. Overlaying f, which also has area 1, makes a direct comparison possible. Overlaying q would put the target six times too low even when the accepted sample is correct.
Probabilities are **areas over intervals**: $P(a \le Y \le b) = \int_a^b f(y)\,dy$. For example, a histogram density of 1.6 over a bin of width 0.05 represents relative frequency $1.6 \times 0.05 = 0.08$. The target is already normalized even though its peak is 1.5.

Rejection sampling can also use an unnormalized target

Normalization is needed for a density comparison, but the sampling algorithm need not know Z. If the proposal density g is normalized and a known constant C satisfies $q(y) \le Cg(y)$ everywhere, accept with probability $q(y)/(Cg(y))$. Then

$$P(\text{accept}) = \frac{Z}{C}, \qquad f_{Y|\text{accept}}(y) = \frac{g(y)\,q(y)/(Cg(y))}{Z/C} = \frac{q(y)}{Z} = f(y).$$

Here $g(y) = 1$, $q(y) = y(1-y)$ and $C = 1/4$, giving acceptance probability $4y(1-y)$ and overall rate $(1/6)/(1/4) = 2/3$. These are exactly the same as using normalized f with $M = 3/2$. The corresponding unnormalized plot would use ordinate $W = CU = U/4$ and compare it with $q(Y)$; both the target and envelope are one-sixth of the scales used in Figure 4.6. **Keep the target and envelope on the same scale.** The formula $1/M$ for the overall acceptance rate assumes a normalized target; with q, the rate is Z/C.

⚠ Normalized or unnormalized: when to use which

- **Density plots and probabilities: use the normalized density $f = q/Z$.** Compare f with a density histogram or KDE, and integrate f to calculate probabilities. An area under q must be divided by Z before it is a probability.
- **Rejection sampling: either form works.** Use normalized f when it is available. If Z is unknown or costly to compute, use q with a known valid envelope Cg, where g is a normalized proposal density and $q(y) \le Cg(y)$ everywhere. Accept when $U \le q(Y)/(Cg(Y))$; this decision does not need Z.
- **For identical accept/reject decisions, rescale both target and envelope.** For Beta(2,2), $f(y) = 6y(1-y)$ and $q(y) = y(1-y)$ give the same decision because

$$\frac{f(Y)}{3/2} = \frac{q(Y)}{1/4} = 4Y(1-Y).$$

Keeping $3/2$ as the envelope for q is still valid here, but less efficient. **Do not compare q directly with a density histogram:** their vertical scales do not match.

Figure 4.7 uses the **same proposal and decision pairs (Y, U) in both panels**. The right panel divides the target, envelope, and plotted ordinate by 6. Consequently, every point keeps the same accept/reject classification:

$$U \le \frac{f(Y)}{3/2} = \frac{q(Y)}{1/4} = 4Y(1-Y).$$

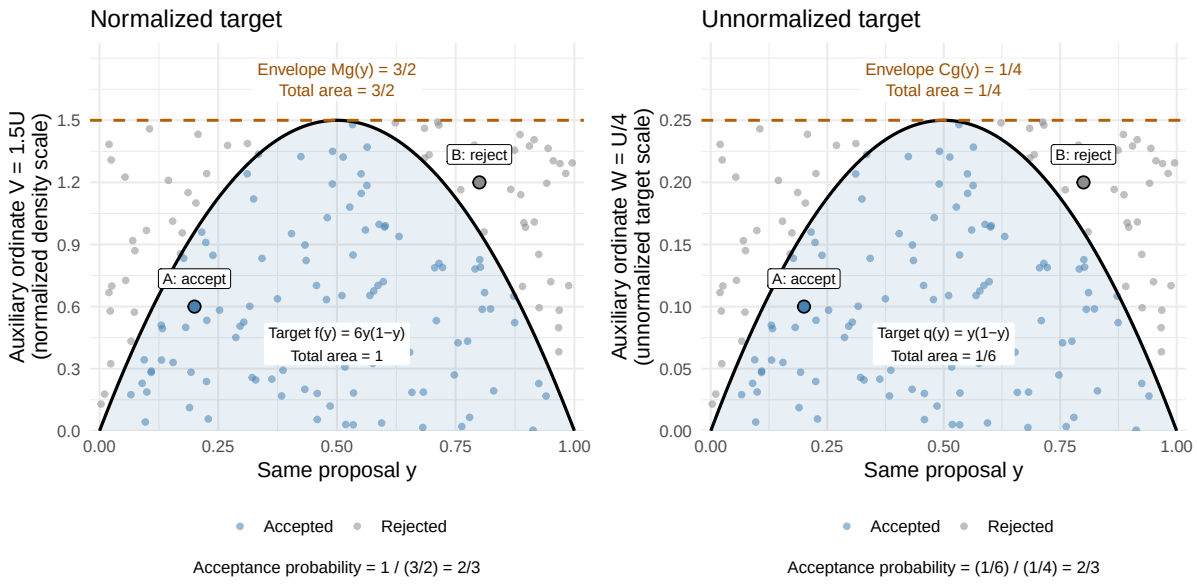


Figure 4.7: The same (Y,U) pairs on two vertical scales. Solid black curves are targets; orange dashed lines are envelopes. Left: normalized target area 1 and envelope area 3/2. Right: unnormalized target area 1/6 and envelope area 1/4. Their area ratios are both 2/3, the theoretical acceptance probability. Point A uses (Y,U) = (0.2,0.4) and is accepted; point B uses (0.8,0.8) and is rejected in both panels. Areas are measured over the unit interval; each target and its envelope use the same scale.

The full R function in the online notes stops at exactly n accepted observations. The table compares its proposal counts with the expected cost nM.

M	Proposals	Expected proposals	Observed rate	Target rate
1.5	3028	3000	0.661	0.667
6.0	11836	12000	0.169	0.167

Each proposal here uses two uniforms, one for Y and one for the decision. Thus $n = 2000$ with $M = 1.5$ uses 3000 proposals and 6000 uniform draws **in expectation**. For another proposal generator, its internal random-number cost could differ.

Exercise 4.2 (In Class: Check One Proposal)
For the same target and uniform proposal, take $y = 0.8$ and $u = 0.7$.

1. With $M = 1.5$, compute the acceptance threshold and decide whether to keep y.
2. **T/F.** We can improve efficiency by using $M = 1$ and capping the acceptance probability at 1.

Solution 4.2

1. **Compute the threshold.** At $y = 0.8$, $f(y) = 6(0.8)(0.2) = 0.96$ and $Mg(y) = 1.5(1) =$

1.5. Thus $a = 0.96/1.5 = 0.64$.
2. **Make the decision.** Since $u = 0.7 > 0.64$, reject y. Draw a fresh proposal and a fresh uniform; do not increase the accepted count.
3. **Check the proposed shortcut: F.** With $M = 1$, the envelope fails at $y = 0.5$: $f(y) = 1.5 > Mg(y) = 1$. Capping the ratio changes the accepted distribution; choose a valid M before sampling.

A Bound Must Hold Everywhere

The envelope must satisfy $f(y) \leq Mg(y)$ throughout the support. A plausible histogram cannot establish that condition.

Unnormalized Targets

Sometimes we know only $h(x)$ where

$$f(x) = \frac{h(x)}{Z}, \qquad Z = \int h(x)\,dx.$$

If we can establish $h(x) \leq Cg(x)$ everywhere, we may accept with probability $h(Y)/\{Cg(Y)\}$ without knowing Z. The acceptance rate is then Z/C, not $1/C$.
This is useful for some posterior distributions, where the normalizing constant is difficult to compute. The challenge remains finding an easy proposal and a valid global bound.

Connections and Limitations

Connection to optimization. Finding the smallest bound involves maximizing f/g, or its logarithm. A numerical optimizer or a finite grid may help explore this ratio, but does not by itself prove a global bound, especially on unbounded support.
Connection to importance sampling. Both methods use a proposal g and the ratio f/g. Rejection sampling discards proposals to obtain draws from the target; importance sampling keeps proposals and weights their contributions when estimating an expectation.
Computational limitation. In high dimensions, a proposal that misses the target's concentration or dependence can yield a very small acceptance rate. This helps motivate more specialized methods, including the Markov chain Monte Carlo methods encountered in other statistics courses.

Section 4.4 Using Probability Distribution Theory

Known distributional relationships are also simulation algorithms. They often reveal *why* a distribution appears in statistical inference.

Theorem 4: Box-Muller Transformation (Optional)

If U_1, U_2 are independent uniforms, define

$$R = \sqrt{-2\log U_1}, \qquad \Theta = 2\pi U_2,$$

and

$$Z_1 = R\cos\Theta, \qquad Z_2 = R\sin\Theta.$$

Then Z_1 and Z_2 are independent standard normal variables.

Radius and Direction

A bivariate standard normal has circularly symmetric density. Its direction is uniform, and its squared radius has a χ^2_2 distribution, equivalently an exponential distribution with rate $1/2$. Box-Muller generates that radius and direction, then converts polar coordinates to Cartesian coordinates. The polar Jacobian contributes a factor r, giving the radial density $re^{-r^2/2}$.

Example 9: Constructing a Bivariate Normal Cloud

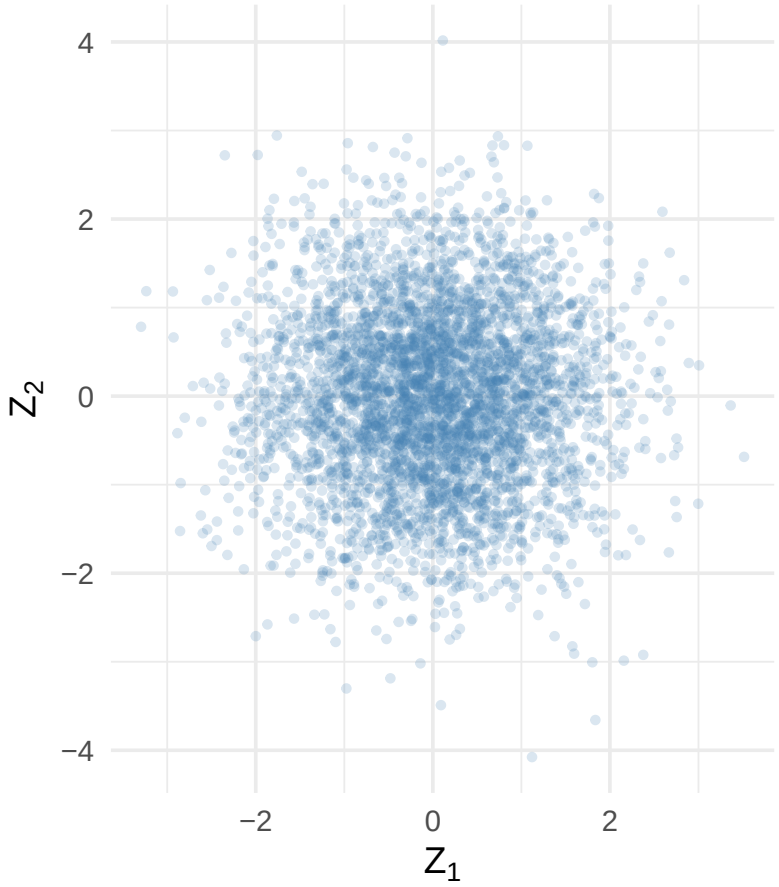


Figure 4.8: Box-Muller creates a circular normal cloud from two independent uniforms. The two normal coordinates are independent even though their formulas share a radius and angle.

Independence Comes from the Joint Distribution

Shared ingredients do not automatically imply dependence, and an approximately zero sample correlation does not prove independence. Here independence follows from the joint density. For routine work, use `rnorm()`; this construction explains one way a normal generator can work.

Theorem 5: Normal, Chi-Square, t, and F Relationships

If $Z_1, \ldots, Z_k$ are independent $N(0,1)$ variables,

$$V = \sum_{j=1}^{k} Z_j^2 \sim \chi^2_k.$$

If $Z \sim N(0,1)$ is independent of $V \sim \chi^2_\nu$, then

$$T = \frac{Z}{\sqrt{V/\nu}} \sim t_\nu.$$

If $V_1 \sim \chi^2_{d_1}$ and $V_2 \sim \chi^2_{d_2}$ are independent, then

$$\frac{V_1/d_1}{V_2/d_2} \sim F_{d_1,d_2}.$$

Why a t Statistic Has Heavier Tails

A t statistic divides a standardized normal estimation error by an estimated scale. When the scale estimate is unusually small, the ratio can be large. This explains the heavier tails of the t distribution and the larger critical values used when estimating an unknown normal variance.

Example 10: Generating a t Distribution with Five Degrees of Freedom

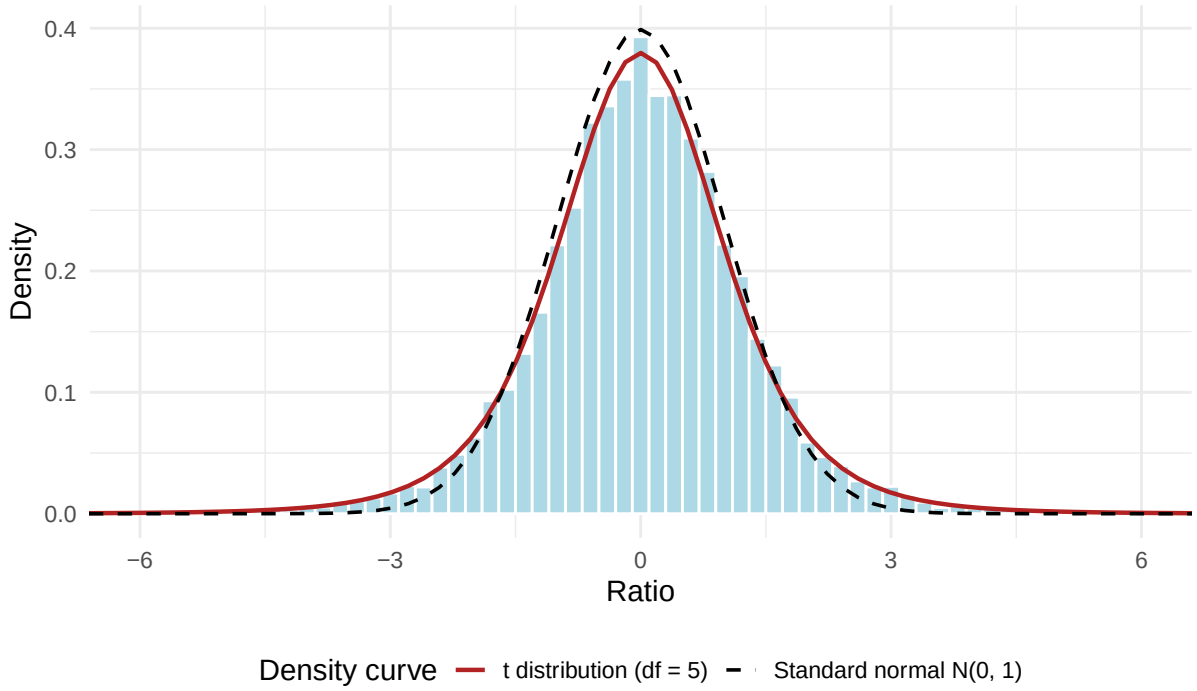


Figure 4.9: The light-blue bars show 10,000 simulated ratios. The red solid line is the theoretical t density with 5 degrees of freedom; the black dashed line is the standard normal density. The ratios follow the t distribution, which has heavier tails than the standard normal.

Generate the Numerator and Denominator Independently

The numerator and denominator must be generated independently. Reusing the same normals in both changes the distribution. In this example, $\text{Var}(T) = \nu/(\nu-2) = 5/3$, so the raw t_5 distribution also has greater variance than $N(0,1)$.

Theorem 6: Beta as a Ratio of Independent Gammas

If $G_1 \sim \text{Gamma}(a, \lambda)$ and $G_2 \sim \text{Gamma}(b, \lambda)$ are independent with the **same rate**, then

$$X = \frac{G_1}{G_1 + G_2} \sim \text{Beta}(a, b).$$

i A Proportion of a Random Total

The ratio is a random share of a positive total. This gives intuition for beta models of probabilities and proportions. The common rate cancels from the ratio.

Example 11: Generating and Checking Beta(3,2) Draws

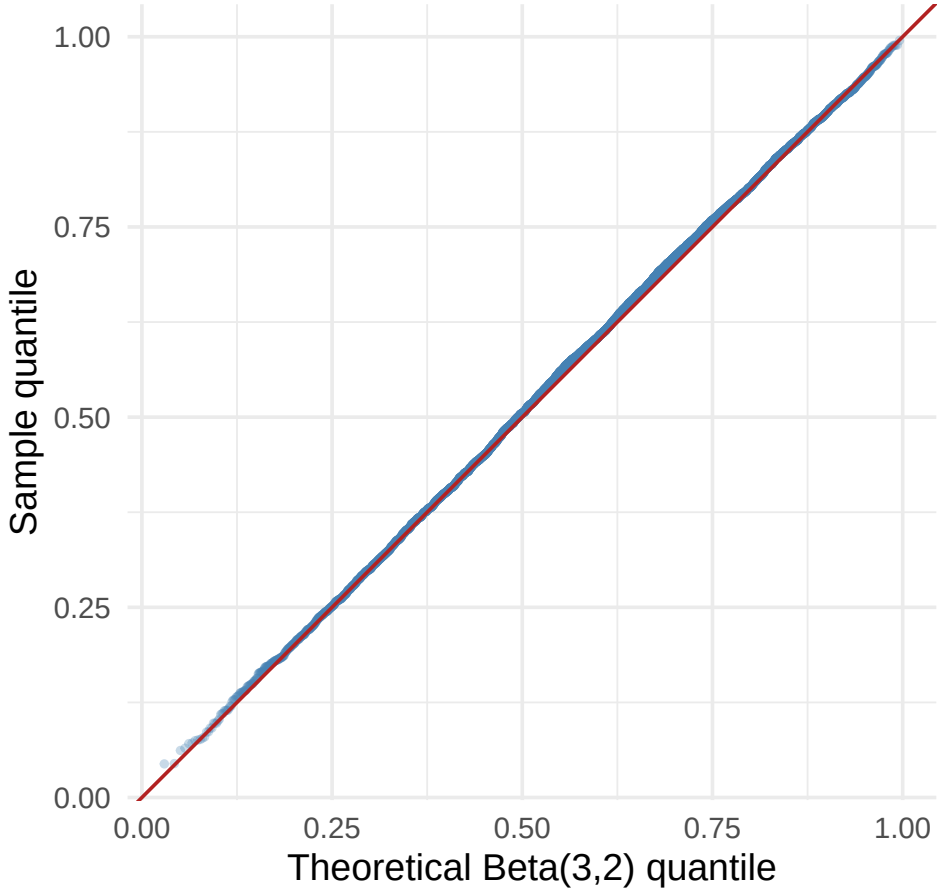


Figure 4.10: A quantile comparison checks the entire distribution of the gamma ratio against Beta(3,2), including its tails.

Connection to Dirichlet Distributions (Optional)

Normalizing several independent gammas with a common rate similarly produces a **Dirichlet** random vector. Its components sum to one, making it useful for uncertain category probabilities and mixture weights.

These identities construct new variables from familiar ones. We next distinguish two ways of combining distributions: adding contributions and selecting a population.

Section 4.5 Sums and Mixtures

4.5.1 Sums and Convolution

A sum combines contributions from **all** its terms:

$$S = X_1 + \cdots + X_k.$$

For independent continuous variables, the density of $X_1 + X_2$ is

$$f_S(s) = \int f_{X_1}(x) f_{X_2}(s - x)\, dx.$$

The integral is a convolution. Simulation avoids computing this integral: generate the terms jointly according to the model and add them.

Theorem 7: Useful Convolution Identities

Under the independence conditions below:

- A sum of independent χ^2 variables is chi-square with the sum of the degrees of freedom.
- A sum of k independent $\text{Exp}(\lambda)$ variables is $\text{Gamma}(k, \lambda)$.
- A sum of k independent geometric failure counts with success probability p is negative binomial with `size = k` and `prob = p`.

What a Sum Represents

A total processing time adds durations from all stages. A count of failures before the kth success adds the failures before each successive success. The sum corresponds to an actual mechanism, not just an algebraic identity.

4.5.2 Mixture Distributions

Definition 2: Finite Mixture

A finite mixture has CDF and density

$$F_X(x) = \sum_{j=1}^{K} w_j F_j(x), \qquad f_X(x) = \sum_{j=1}^{K} w_j f_j(x),$$

where $w_j \geq 0$ and $\sum_j w_j = 1$.
Introduce an unobserved label C:

$$P(C = j) = w_j, \qquad X \mid C = j \sim F_j.$$

The law of total probability gives the mixture distribution after we average over the unknown label.

Algorithm 3: Generating a Finite Mixture

Input: component distributions $F_1, \ldots, F_K$, weights $w_k \geq 0$ with $\sum_k w_k = 1$, and sample size n.

1. **Choose a component.** Draw one label C with probabilities $w_1, \ldots, w_K$.
2. **Sample within it.** Draw one observation X from F_C using fresh randomness. For a vector observation, the same label selects its entire component distribution.
3. **Save and repeat.** Repeat steps 1–2 independently for each of the n observations. Do not replace the label draw by a weighted average of component draws.

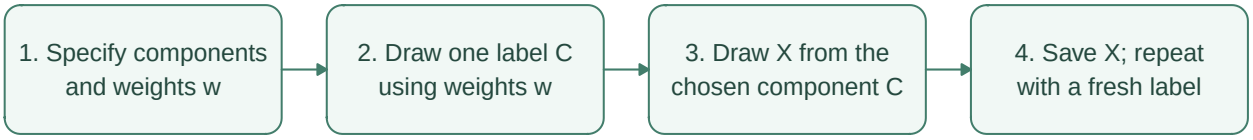


Figure 4.11: A mixture chooses a component before generating an observation.

Mixture Weights Average Distributions

The weights average **distributions**. They do not instruct us to return the numerical weighted average $w_1 X_1 + \cdots + w_K X_K$.

Example 12: Three Different Data-Generating Mechanisms

Let $X_1 \sim N(0,1)$ and $X_2 \sim N(4,1)$ be independent. Compare:

$$\begin{array}{ll} \text{Sum:} & S = X_1 + X_2 \sim N(4,2), \\ \text{Weighted average:} & A = (X_1 + X_2)/2 \sim N(2, 1/2), \\ \text{Mixture:} & X \sim 0.5N(0,1) + 0.5N(4,1). \end{array}$$

The mixture has mean 2 and variance 5. It describes observations from two subpopulations;

each observation belongs to one subpopulation.

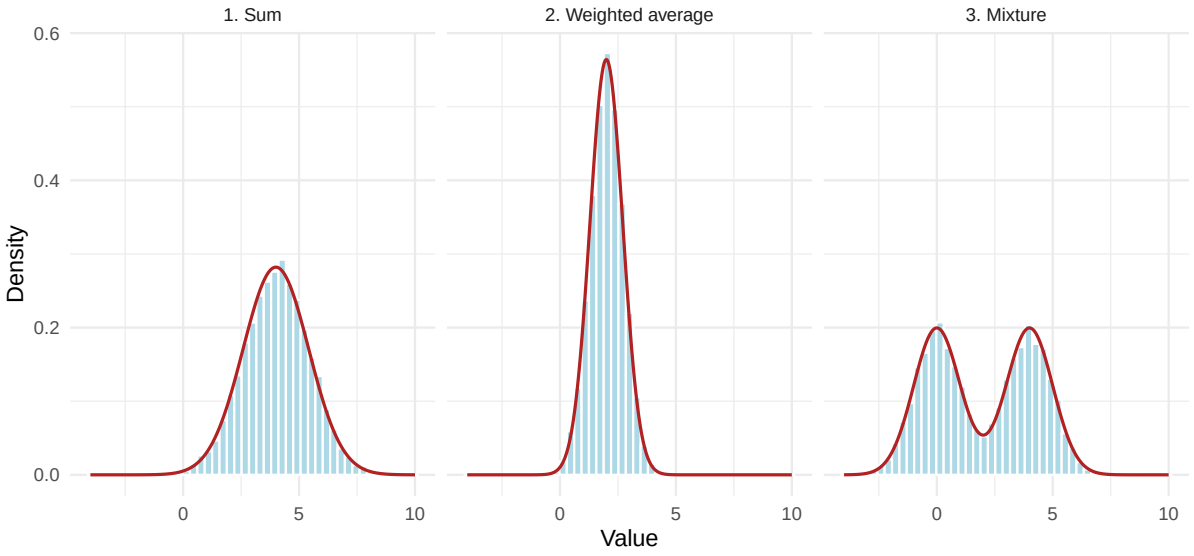


Figure 4.12: Adding two observations, averaging them, and selecting one population produce different distributions. The solid curves show their respective theoretical densities.

Theorem 8: Mean and Variance of a Mixture

If component j has mean μ_j and variance σ_j^2, then

$$\mathbb{E}(X) = \sum_j w_j \mu_j = \mu,$$

and the law of total variance gives

$$\begin{aligned} \mathrm{Var}(X) &= \mathbb{E}\{\mathrm{Var}(X \mid C)\} + \mathrm{Var}\{\mathbb{E}(X \mid C)\} \\ &= \underbrace{\sum_j w_j \sigma_j^2}_{\text{within-component variation}} + \underbrace{\sum_j w_j (\mu_j - \mu)^2}_{\text{between-component variation}} . \end{aligned}$$

Heterogeneity and Latent Labels

This decomposition explains why unobserved heterogeneity can inflate variability. A mixture of distinct normal components is generally nonnormal, but it need not be bimodal. Identical components yield the original normal distribution.

Connection to clustering and EM. Simulation starts with the label and then generates the observation. In mixture-model estimation, we see the observation and infer uncertainty about its label. This reversal is the basis for the responsibility calculations in the EM algorithm.

Exercise 4.3 (In Class: Mixture or Average?)
Let $X_1 \sim N(0,1)$ and $X_2 \sim N(4,1)$ be independent. **T/F.** Returning $(X_1 + X_2)/2$ generates

the same distribution as choosing one of the two components with equal probabilities. Both have mean 2; what else should you check?

Solution 4.3

1. **Check the shared mean.** Both constructions have mean $(0+4)/2 = 2$, so the mean cannot tell them apart.
2. **Compute the average's variance.** Independence gives $\mathrm{Var}\{(X_1+X_2)/2\} = (1+1)/4 = 0.5$.
3. **Compute the mixture's variance.** Within-component variance is 1; between-component variance is $\{(0-2)^2 + (4-2)^2\}/2 = 4$. The total is $1+4 = 5$. Hence **F**: the generators differ. Check variance and shape, not just the mean.

Example 13: Poisson-Gamma Mixing and Overdispersion
A mixture label need not take only finitely many values. Consider

$$\Lambda \sim \mathrm{Gamma}(a, b), \qquad Y \mid \Lambda \sim \mathrm{Poisson}(\Lambda),$$

where b is a rate. Then

$$\mathbb{E}(Y) = \frac{a}{b}, \qquad \mathrm{Var}(Y) = \frac{a}{b} + \frac{a}{b^2}.$$

The second term is variability in the underlying rate. Marginally, Y is negative binomial with `size = a` and `prob = b / (b + 1)`.

```
set.seed(8670)
n <- 20000
a <- 2
b <- 0.5
latent_rate <- rgamma(n, shape = a, rate = b)
counts <- rpois(n, lambda = latent_rate)

knitr::kable(booktabs = TRUE, data.frame(
  quantity = c("Mean", "Variance"),
  simulated = c(mean(counts), var(counts)),
  theoretical = c(a / b, a / b + a / b^2)
), digits = 3)
```

quantity	simulated	theoretical
Mean	3.979	4
Variance	11.949	12

Optional Connection: Heterogeneous Event Rates

Customers, neighborhoods, or machines can have different event rates. Even when each unit follows a Poisson model conditional on its rate, the pooled counts may have variance greater than their mean. This is one reason negative binomial regression is useful.

Shared Latent Variables Create Dependence

We drew one independent rate for each count. If several observations share a single group-specific rate, they become dependent after that rate is integrated out. **Where we place a draw in the simulation loop determines which observations share uncertainty.**

The same generative logic extends to random vectors. The additional task is to reproduce how their coordinates vary together.

Section 4.6 Generating Multivariate Random Variables

4.6.1 Multivariate Normal Distribution

Suppose two measurements each have a standard normal marginal distribution. We still need to specify how they move together. Independent `rnorm()` calls impose independence; they cannot represent a positive correlation merely because the marginal histograms look correct.

For a normal vector, the covariance matrix supplies this information:

$$\boldsymbol{X} \sim N_d(\boldsymbol{\mu}, \boldsymbol{\Sigma}).$$

Theorem 9: Linear Transformation of a Normal Vector

If $\boldsymbol{Z} \sim N_d(0, I_d)$ and $AA^\top = \boldsymbol{\Sigma}$, then

$$\boldsymbol{X} = \boldsymbol{\mu} + A\boldsymbol{Z}.$$

Proof.

$$\mathbb{E}(\boldsymbol{X}) = \boldsymbol{\mu}, \qquad \mathrm{Cov}(\boldsymbol{X}) = AI_dA^\top = \boldsymbol{\Sigma}.$$

The normality follows because every linear combination of $\boldsymbol{X}$ is a linear combination of independent normals.

Match the Matrix Factor to the Row Convention

For n **rows**, each with target mean $\boldsymbol{\mu}$ and symmetric positive-definite covariance $\boldsymbol{\Sigma}$:

1. **Factor once.** Compute `R <- chol(Sigma)`. R returns an **upper triangular** factor with $R^\top R = \boldsymbol{\Sigma}$.

2. **Generate independent inputs.** Fill an $n \times d$ matrix Z with independent $N(0,1)$ draws.
3. **Transform each row.** Multiply on the right by R, then add the mean to every row:

$$X = ZR + 1_n \boldsymbol{\mu}^\top.$$

For a **column** vector, the corresponding formula is $X = \boldsymbol{\mu} + R^\top Z$. Multiplying rows by `t(R)` instead gives covariance $RR^\top$, generally the wrong matrix. See the R documentation for `chol()`.

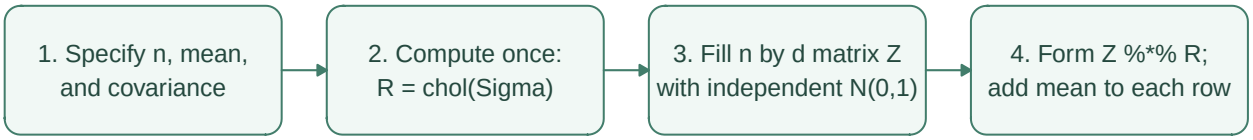


Figure 4.13: Cholesky generation in R: observations are rows, so multiply by R on the right.

Work one two-dimensional case before using the matrix formula. To obtain standard normal marginals with correlation 0.6, use

$$\Sigma = \begin{pmatrix} 1 & 0.6 \\ 0.6 & 1 \end{pmatrix}, \qquad R = \begin{pmatrix} 1 & 0.6 \\ 0 & 0.8 \end{pmatrix}, \qquad R^\top R = \Sigma.$$

For independent standard normals Z_1, Z_2, row multiplication gives

$$X_1 = Z_1, \qquad X_2 = 0.6Z_1 + 0.8Z_2.$$

The second variance is $0.6^2 + 0.8^2 = 1$, and the covariance is $0.6\,\mathrm{Var}(Z_1) = 0.6$. If $(z_1, z_2) = (1, -1)$, the output is $(1, -0.2)$.

```
Sigma_small <- matrix(c(1, 0.6, 0.6, 1), nrow = 2)
R_small <- chol(Sigma_small)              # upper triangular factor
z_row <- matrix(c(1, -1), nrow = 1)       # one observation, two coordinates
z_row %*% R_small                        # one correlated observation
#>      [,1] [,2]
#> [1,]    1 -0.2
```

Exercise 4.4 (In Class: Match the Matrix to the Code)
Using the same R, transform the row $(z_1, z_2) = (2, 0)$. **Multiple choice.** For independent standard normal rows in `Z`, which gives covariance Σ: **(A)** `Z %*% R` or **(B)** `Z %*% t(R)`?

Solution 4.4

1. **Multiply the row by each column.** For $(2,0)R$, the first entry is $2(1) + 0(0) = 2$; the

second is $2(0.6) + 0(0.8) = 1.2$. The result is $(2, 1.2)$.

2. **Match the covariance.** A row multiplied by R corresponds to a column transformed by $R^\top$, so its covariance is $R^\top R = \Sigma$. Choose **A**.
3. **Diagnose B.** Multiplying rows by `t(R)` gives $RR^\top = \begin{pmatrix} 1.36 & 0.48 \\ 0.48 & 0.64 \end{pmatrix}$, which does not match the target.

The construction has a geometric interpretation. In Figure 4.14, the same points start in a circular cloud, become an ellipse after multiplication by the matrix factor, and then move to the desired mean. Color tracks each point's original first coordinate.

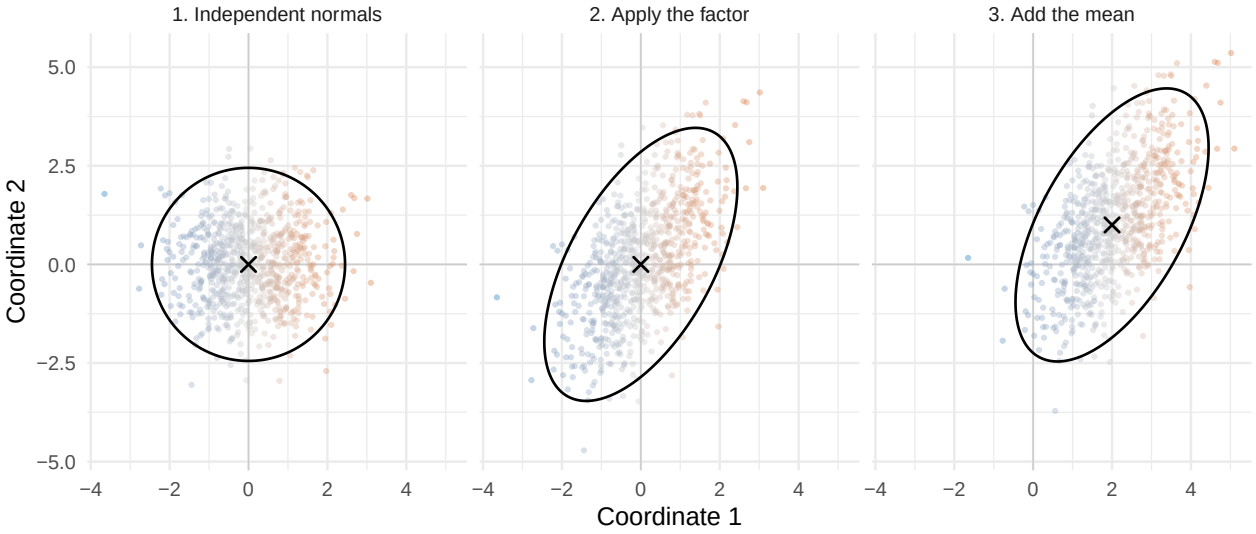


Figure 4.14: A matrix factor introduces scale and dependence; adding the mean shifts location. The black curves are corresponding 95% normal probability contours. All three panels use the same underlying draws and the same axis scale.

Example 14: A Generator with a Specified Mean and Covariance

The reusable `rmvn_chol()` function in the online notes generates rows of independent normals, multiplies by `chol(Sigma)`, and adds `mu` to each row. Here $n = 10000$, $\mu = (1, 2)^\top$, and $\Sigma = \begin{pmatrix} 1 & 0.8 \\ 0.8 & 2 \end{pmatrix}$. Compare the sample mean and covariance below with these targets.

```
#> [1] 1.017927 2.037802
#>           [,1]      [,2]
#> [1,] 1.0040551 0.8101849
#> [2,] 0.8101849 1.9951480
#>      [,1] [,2]
#> [1,]  1.0  0.8
#> [2,]  0.8  2.0
```

The function also returns an $n \times d$ matrix for $n = 0$ or $n = 1$. This matters when a component in a simulated mixture receives few or no observations.

Example 15: The Same Marginals, Different Joint Tail Probabilities

Consider two standardized laboratory measurements with unit marginal variances. We compare independence with correlation $\rho = 0.8$ while keeping the marginals the same.

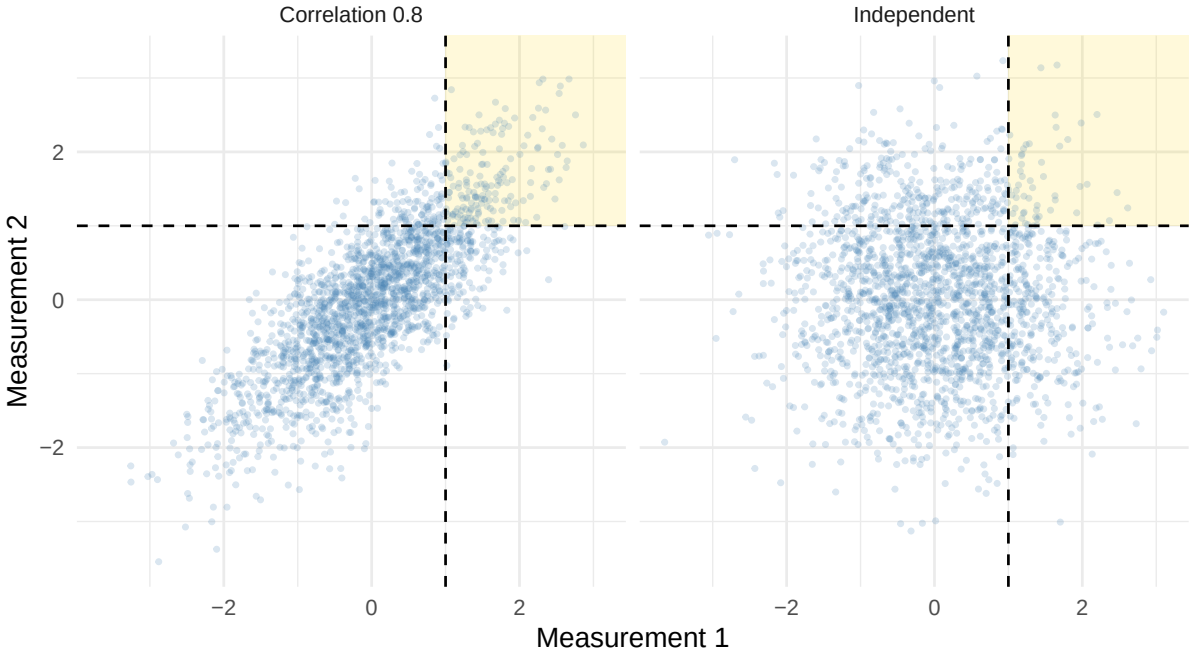


Figure 4.15: The marginal distributions are the same in both panels. Positive correlation puts more probability in the upper-right region where both measurements exceed 1.

```
knitr::kable(booktabs = TRUE, joint_results, digits = 4,
             col.names = c("Model", "Joint probability", "MCSE"))
```

Model	Joint probability	MCSE
Independent	0.0261	0.0009
Correlation 0.8	0.0971	0.0017

```
(1 - pnorm(1))^2  # exact probability under independence
#> [1] 0.02517149
```

The simulated joint probability rises from approximately 0.026 to 0.097 under these two models. The **Monte Carlo standard error (MCSE)** measures simulation noise; we derive it below. The same issue arises with simultaneous equipment failures, correlated regression errors, and repeated measurements. A study of a joint event must preserve the relevant dependence.

Theorem 10: Optional: Wishart Construction

If $\boldsymbol{X}_1, \ldots, \boldsymbol{X}_m$ are independent $N_d(0, \boldsymbol{\Sigma})$ vectors, then

$$W = \sum_{i=1}^{m} \boldsymbol{X}_i \boldsymbol{X}_i^\top \sim \text{Wishart}_d(m, \boldsymbol{\Sigma}).$$

Optional Connection: Uncertainty in a Sample Covariance

With observations stored in rows, this is simply `crossprod(X)`. For a normal sample of size n with an estimated mean, its usual sample covariance satisfies

$$(n-1)S \sim \text{Wishart}_d(n-1, \boldsymbol{\Sigma}).$$

This explains why Wishart simulation is relevant to uncertainty in covariance estimates and multivariate inference.

```
set.seed(8670)
Sigma_w <- matrix(c(1, 0.4, 0.4, 2), 2)
wishart_df <- 12
W <- crossprod(rmvn_chol(wishart_df, c(0, 0), Sigma_w))
W / wishart_df  # expectation equals Sigma_w, but one draw is noisy
#>             [,1]        [,2]
#> [1,]  0.63083420 -0.04991003
#> [2,] -0.04991003  2.89511646
```

4.6.2 Multivariate Normal Mixtures

One Component Label per Vector

Each observation needs **one component label for the entire vector**. Drawing a different label for each coordinate changes the joint model.

Example 16: A Mixture of Two Bivariate Normals

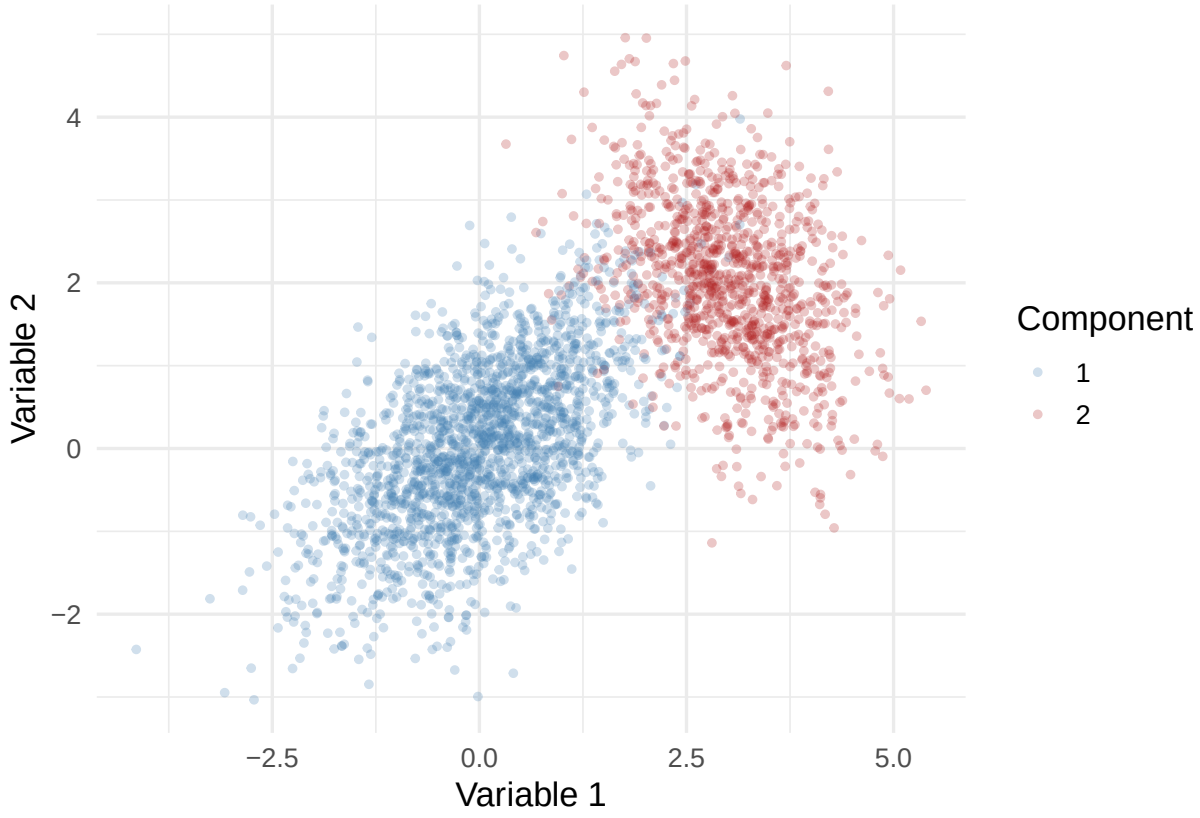


Figure 4.16: Each point is generated from one bivariate normal component. The label determines both its coordinates and its covariance structure.

Within-Group and Between-Group Covariance

For vector means $\boldsymbol{\mu}_j$ and covariances $\boldsymbol{\Sigma}_j$,

$$\mathrm{Cov}(\boldsymbol{X}) = \sum_j w_j \boldsymbol{\Sigma}_j + \sum_j w_j (\boldsymbol{\mu}_j - \boldsymbol{\mu})(\boldsymbol{\mu}_j - \boldsymbol{\mu})^\top.$$

Between-group differences can create marginal association even if variables are independent *within* every group. This is useful when designing simulation studies for clustering or confounding.

4.6.3 Uniform Directions and Points in a Ball

Theorem 11: Uniform Direction from a Standard Normal Vector

Let $\boldsymbol{Z} \sim N_d(0, I_d)$. Its distribution is unchanged by rotations, so it has no preferred direction. Consequently,

$$\boldsymbol{U} = \frac{\boldsymbol{Z}}{\|\boldsymbol{Z}\|}$$

is uniform on the **surface** of the unit sphere in $\mathbb{R}^d$. The zero vector has probability zero under the ideal normal model.

Theorem 12: Uniform Volume Requires a Nonuniform Radius

Uniform direction alone puts every point at distance 1. To fill the ball uniformly in volume, independently generate a radius R satisfying

$$P(R \le r) = r^d, \quad 0 \le r \le 1.$$

This follows because a ball of radius r has r^d times the volume of the unit ball. Therefore,

$$R = V^{1/d}, \qquad \boldsymbol{X} = R\boldsymbol{U},$$

where V is uniform and independent of the direction.

Example 17: Generating a Circle and a Disk

```r
runif_sphere <- function(n, d) {
  stopifnot(
    length(n) == 1L, is.finite(n), n >= 0, n == floor(n),
    length(d) == 1L, is.finite(d), d >= 1, d == floor(d)
  )
  Z <- matrix(rnorm(n * d), nrow = n, ncol = d)
  lengths <- sqrt(rowSums(Z^2))
  sweep(Z, MARGIN = 1, STATS = lengths, FUN = "/")
}
```

We divide each row by its length directly. Constructing an $n \times n$ diagonal matrix of inverse lengths would waste memory.

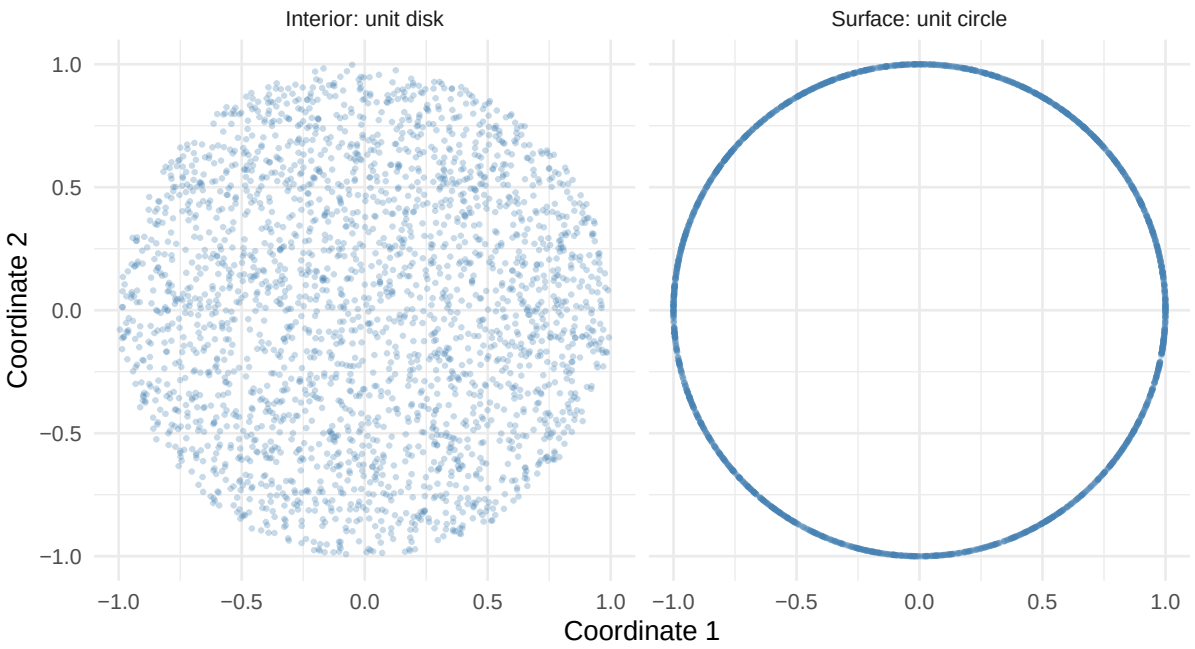


Figure 4.17: In two dimensions, normalized normals lie on the circle. Multiplying by an independent square-root-uniform radius fills the disk uniformly in area.

i Geometry in High Dimensions

Random directions appear in rotationally invariant perturbations, geometric Monte Carlo methods, and randomized algorithms. In high dimensions, a uniform point in a ball is typically close to its boundary: $P(R \leq 0.9) = 0.9^d$. At $d = 100$, this probability is about 0.000027.

Exercise 4.5 (Optional Self-Check: A Uniform Disk)
T/F. A uniform angle and an independent $R \sim U(0, 1)$ give a uniform point inside the unit disk. Compare the proposed $P(R \leq 1/2)$ with the fraction of disk area inside radius $1/2$.

Solution 4.5

1. **Compare probability with area.** A uniform radius gives $P(R \leq 1/2) = 1/2$, but the inner disk occupies only $(1/2)^2 = 1/4$ of the area. Hence **F**.
2. **Use the required radial CDF.** Uniform area requires $F_R(r) = r^2$. Solve $u = r^2$ to get $R = \sqrt{U}$.

Section 4.7 Checking a Generator

A mathematical construction tells us the target law. Numerical checks then help detect programming errors and quantify how much disagreement to expect from a finite sample.

A histogram is a useful first view, but its appearance depends on bin width and sample size. Different distributions can have the same mean, variance, or a similar-looking central region.

A convincing validation combines theory with checks that target different possible mistakes:

Check	What it can reveal
Support and constraints	Negative waiting times, noninteger counts, proportions outside $(0, 1)$
Mean and variance	Incorrect location, scale, or parameterization
Quantiles, ECDF, and tail probabilities	Distributional errors hidden by matching moments
Correlations and joint events	Incorrect dependence or matrix orientation
Repeated runs and random-number use	Reused seeds or repeated observations
Acceptance rate and proposal count	A costly envelope or an implementation error

For the rejection sampler, the ECDF should track the target CDF, and the transformed values should be approximately uniform.

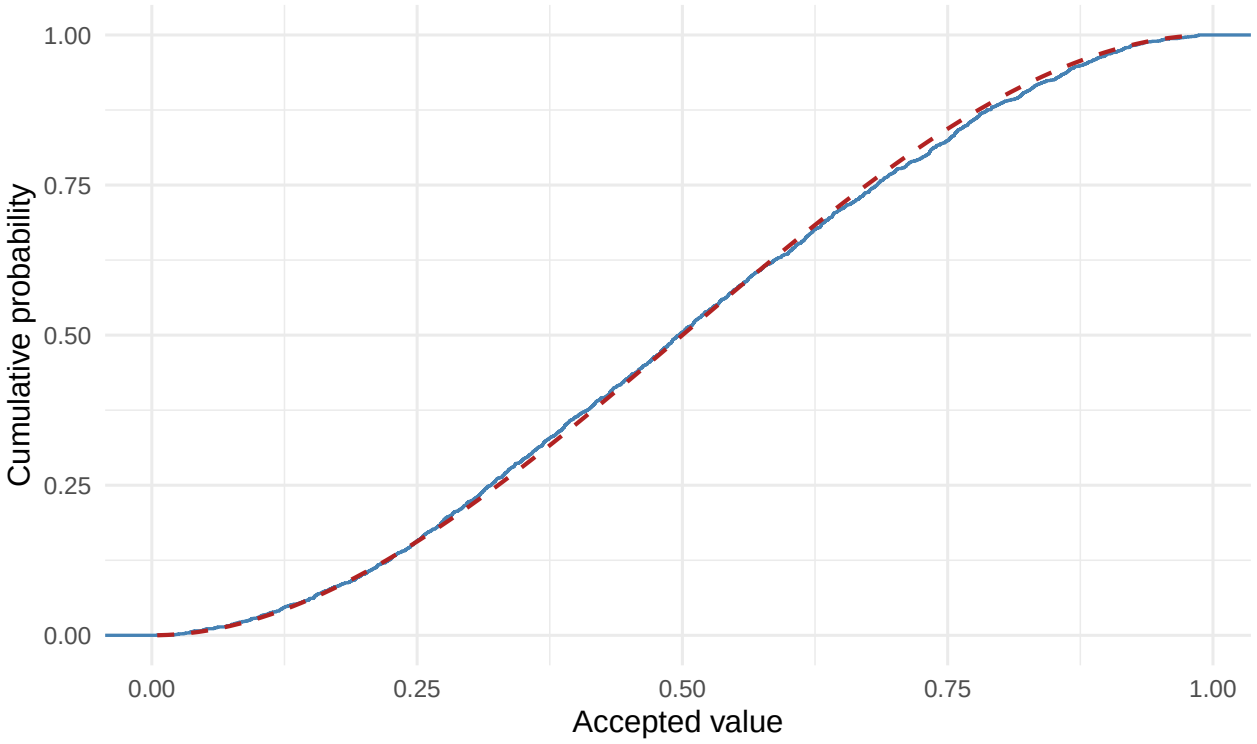


Figure 4.18: The empirical CDF of accepted proposals tracks the Beta(2,2) CDF. Sampling variation creates small discrepancies even for a correct generator.

Formal goodness-of-fit tests can supplement these checks, but passing a test is not a proof of correctness. A correct generator can occasionally be rejected, and a small sample may fail to detect a faulty one. The algorithm's mathematical justification remains essential.

Monte Carlo Standard Error

Suppose we estimate $p = P(X > c)$ using B independent simulated observations:

$$\hat{p} = \frac{1}{B}\sum_{b=1}^{B} \mathbf{1}\{X_b > c\}.$$

Each indicator is Bernoulli, so

$$\mathrm{Var}(\hat{p}) = \frac{p(1-p)}{B}, \qquad \widehat{\mathrm{MCSE}}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{B}}.$$

For an estimated mean of independent $h(X_b)$ with finite variance, the corresponding MCSE is

$$\widehat{\mathrm{MCSE}}\left(\frac{1}{B}\sum_{b=1}^{B} h(X_b)\right) = \frac{s_h}{\sqrt{B}}.$$

Example 18: Checking an Exponential Tail Probability

```
set.seed(8670)
B <- 10000
event <- rexp(B, rate = 0.7) > 3
p_hat <- mean(event)
mcse <- sqrt(p_hat * (1 - p_hat) / B)

knitr::kable(booktabs = TRUE, data.frame(
  estimate = p_hat,
  MCSE = mcse,
  exact = exp(-0.7 * 3),
  lower_MC_limit = p_hat - 1.96 * mcse,
  upper_MC_limit = p_hat + 1.96 * mcse
), digits = 4,
col.names = c("Estimate", "MCSE", "Exact", "Lower MC limit", "Upper MC limit"))
```

Estimate	MCSE	Exact	Lower MC limit	Upper MC limit
0.121	0.0033	0.1225	0.1146	0.1274

What More Simulation Can and Cannot Fix

These limits quantify numerical uncertainty from the finite simulation. They do not quantify uncertainty about whether the exponential model or its rate describes a real population.
To halve MCSE, we generally need about **four times** as many independent replications. For rare events, zero simulated occurrences do not imply zero probability: the plug-in MCSE also

becomes zero and is misleading. A binomial interval or a method designed for rare events is then more informative.

Three Different Sources of Uncertainty

- **Sampling uncertainty** concerns what would happen with another real dataset.
- **Monte Carlo uncertainty** comes from using finitely many simulated replications to study that behavior.
- **Model uncertainty** concerns whether the assumed mechanism is an adequate description of the setting.

More simulation reduces Monte Carlo uncertainty. It does not automatically reduce the other two.

Choosing a Generation Method

Available structure	Natural approach	Main issue to check
A reliable built-in generator	Use the distribution's `r` function	Parameterization and joint assumptions
A tractable quantile	Inverse transformation	Correct generalized inverse and endpoints
A tractable CDF without a convenient inverse	Numerical root finding	Bracketing, monotonicity, tolerance, and cost
A target density and a suitable proposal	Acceptance-rejection	Global envelope and acceptance rate
A known distributional identity	Transform simpler independent variables	Conditions such as independence and common rates
A latent-class or hierarchical model	Generate latent variables, then observations	Which observations share each latent draw
A normal covariance structure	Matrix factorization	Positive definiteness and factor orientation
Rotational or volume symmetry	Geometric construction	Surface versus volume and the correct radius

Methods are often combined. A hierarchical model may use inverse transformation for event times, normal matrix factors for random effects, and conditional Bernoulli draws for outcomes. The goal is a generator that is statistically correct, numerically reliable, and efficient enough for the problem. A sophisticated algorithm cannot compensate for an incorrect data-generating mechanism.

The Main Ideas to Keep

- A probability model becomes operational when we can generate data from it.
- Uniform draws can be transformed into observations through a generalized inverse CDF.
- Rejection sampling reshapes an easy proposal; a valid, tight envelope controls efficiency.
- Distributional identities connect simulation to the foundations of statistical inference.
- A mixture selects a component; a sum combines contributions from all components.
- Hierarchical draws specify both marginal variation and dependence.
- Multivariate generation must reproduce covariance and joint behavior, not only marginal histograms.
- Validation combines mathematical reasoning, numerical checks, and distributional diagnostics.
- More independent replications reduce Monte Carlo error; they do not correct a wrong model.

The next step is to use these generators to approximate expectations, integrals, and sampling distributions. That is the central task of **Monte Carlo simulation**.

Section 4.8 Concept Checks and Practice

Try the concept checks first, then the short calculations. Explain your choice before opening the solution.

Exercise 4.6 (Concept Check: Choose or Mark True/False)

1. **Multiple choice.** In inverse transformation, U is **(A)** an observation from the target or **(B)** a probability level to be transformed.
2. **T/F.** A larger valid rejection bound M increases the acceptance rate.
3. **T/F.** If $f = h/Z$ and $h \le Cg$, accepting with probability $h(Y)/(Cg(Y))$ requires knowing Z.
4. **Multiple choice.** To generate a t_4 variable from independent $Z \sim N(0,1)$ and $V \sim \chi^2_4$, use **(A)** $Z/\sqrt{V/4}$ or **(B)** $Z/(V/4)$.
5. **T/F.** A mixture of bivariate normals uses one component label for the whole row.
6. **T/F.** Correct marginal histograms establish that a multivariate generator is correct.

Solution 4.6

Item	Answer	Reason
1	B	Apply F^{-1} to convert the probability level into an observation.
2	F	The rate is $1/M$ for a normalized target and proposal.
3	F	Z is unnecessary for the decision; the overall rate is Z/C.

4	A	Divide by the square root of the independent chi-square variable per degree of freedom.
5	T	Both coordinates belong to the chosen component; separate labels change the joint model.
6	F	Marginals do not determine dependence; also check covariance and joint events.

Exercise 4.7 (Practice: Construct, Calculate, Check)

1. **Inverse transformation.** For $f(x) = 4x^3$ on $(0, 1)$, derive $F(x)$ and a generator. Transform $u = 1/16$, and find $E(X)$ and $P(X > 1/2)$.
2. **Rejection sampling.** For $f(y) = 3y^2$ on $(0, 1)$ and $g(y) = 1$, find the smallest valid M, the acceptance rate, and the expected number of proposals for 1000 accepted draws. Is $(y, u) = (0.5, 0.3)$ accepted?
3. **Repair the R code.** Rows of `Z` contain independent standard normals. The target has mean `mu` and positive definite covariance `Sigma`. Correct the second line and explain which covariance check detects the original error.

```
R <- chol(Sigma)
X <- sweep(Z %*% t(R), 2, mu, FUN = "+")
```

Solution 4.7

1. **Inverse transformation** (follow Figure 4.1).
   - **Find the CDF:** $F(x) = \int_0^x 4t^3\,dt = x^4$ for $0 < x < 1$.
   - **Invert and substitute:** $u = x^4$ gives $X = U^{1/4}$. At $u = 1/16$, $x = (1/16)^{1/4} = 1/2$.
   - **Check the target:** $E(X) = \int_0^1 4x^4\,dx = 4/5$ and $P(X > 1/2) = 1 - F(1/2) = 15/16$.
2. **Rejection sampling** (follow Figure 4.5).
   - **Find a global bound:** $f(y)/g(y) = 3y^2$ has supremum 3 on $(0, 1)$, so the smallest valid bound is $M = 3$.
   - **Compute efficiency:** acceptance rate $= 1/M = 1/3$; expected proposals $= nM = 1000(3) = 3000$.
   - **Decide on this proposal:** $a = f(0.5)/\{Mg(0.5)\} = 3(0.5)^2/3 = 0.25$. Since $0.3 > 0.25$, reject and draw a fresh pair.
3. **Repair the R code** (follow Figure 4.13).
   - **Identify the factor:** `chol(Sigma)` gives $R^\top R = \Sigma$.
   - **Match rows to the factor:** use `X <- sweep(Z %*% R, 2, mu, FUN = "+")`. This gives covariance $R^\top R$; the original gives $RR^\top$.

- **Check the result:** compare `cov(X)` with `Sigma`, allowing for Monte Carlo variation. Adding `mu` changes the mean, not covariance.

Section 4.9 Chapter Summary

This chapter answers one question: **How can we turn a probability model into simulated data?**

Main Ideas

- **Choose a suitable method.** Use inverse transformation when the quantile function is available, acceptance-rejection when an easy proposal and a valid bound are available, or known distributional relationships.
- **Preserve the joint model.** For a mixture, choose a component before drawing an observation. For multivariate data, preserve the required dependence as well as the marginal distributions.
- **Check the generator.** Compare its support, moments, plots, and joint behavior with the intended model.
- **Measure simulation error.** For independent replications, MCSE typically decreases like $1/\sqrt{B}$. More simulation reduces Monte Carlo uncertainty; it does not fix an inappropriate model.

Connections to Other Chapters

- **Chapter 1: Introduction to R** provides the functions, vectors, and loops used to implement simulations.
- **Chapter 2: Intro to Computational Statistics** provides the numerical and matrix tools needed to turn mathematical formulas into reliable code.
- **Chapter 3: Optimization** fits a model to data. This chapter generates data from a model. Combining the two lets us repeatedly simulate and refit data to study an estimator's accuracy.

Section 4.10 Supplementary

The content here won't be on the exam

Eigenvalues, PCA, and Reusing the Factorization

The spectral decomposition is

$$\boldsymbol{\Sigma} = P\Lambda P^\top, \qquad \boldsymbol{\Sigma}^{1/2} = P\Lambda^{1/2}P^\top.$$

The eigenvectors specify directions, and the square roots of the eigenvalues specify how much to stretch a standard normal cloud in those directions.

This connects generation to **principal component analysis**. PCA identifies directions of variation in observed data; simulation constructs that variation from independent coordinates. A covariance matrix can be positive semidefinite with some eigenvalues equal to zero. The normal vector then lies in a lower-dimensional subspace. Ordinary unpivoted Cholesky may fail, whereas an eigenvalue construction can handle that case. Tiny negative eigenvalues caused by roundoff may be set to zero after a tolerance check; substantial negative eigenvalues indicate an invalid covariance matrix and should not be silently removed.
If $\boldsymbol{\Sigma}$ stays fixed across many simulation batches, compute its factor once and reuse it. Factorization costs roughly order d^3, while multiplying n observations by a dense factor costs order nd^2. Avoid repeating work that does not change the model.

4.10.1 Real-World Applications

The core chapter is complete. These examples are for independent study. Choose one to connect the generators to a full analysis; their assumptions and parameters are illustrative.

Example 19: Application: Experimental Power

Suppose an experiment compares two independent groups, each with n observations:

$$X_i \sim N(0, \sigma^2), \qquad Y_i \sim N(\delta, \sigma^2).$$

Here δ is a meaningful difference we want to detect. To keep the benchmark exact, assume $\sigma = 1$ is known. The two-sided level-0.05 test rejects when

$$|Z| > z_{0.975}, \qquad Z = \frac{\bar{Y} - \bar{X}}{\sigma\sqrt{2/n}}.$$

For $\delta = 0$, the rejection probability is the Type I error rate. For $\delta \neq 0$, it is power.
We generate **whole datasets**, apply the proposed analysis to each dataset, and record the result. The data size n determines how informative each experiment is. The number of replications B determines how accurately we estimate its rejection probability.

n	delta	estimated	exact	MCSE
10	0.0	0.0503	0.0500	0.0035
20	0.0	0.0505	0.0500	0.0035
40	0.0	0.0508	0.0500	0.0035
80	0.0	0.0435	0.0500	0.0032
10	0.5	0.2020	0.2010	0.0063
20	0.5	0.3470	0.3526	0.0075
40	0.5	0.6038	0.6088	0.0077
80	0.5	0.8815	0.8854	0.0051

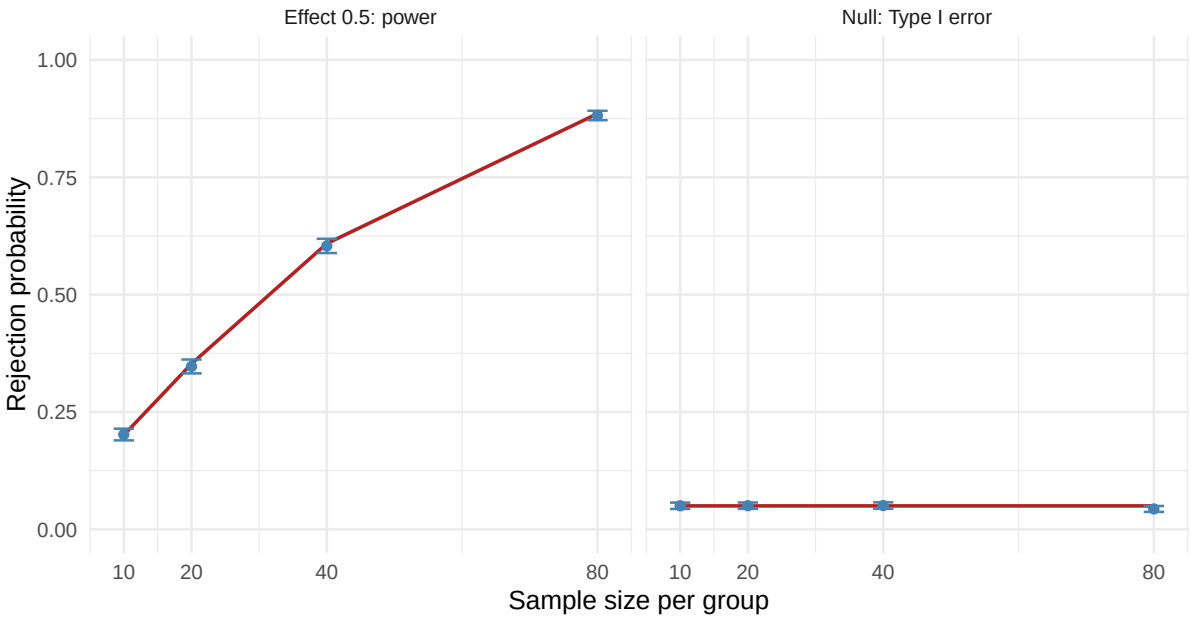


Figure 4.19: Points are simulated rejection rates with approximate 95% Monte Carlo error bars. Lines are exact normal-model probabilities. Increasing the experiment's sample size increases power, while the Type I error remains near 0.05.

Interpretation and Assumptions

The exact formula gives a benchmark for our simulation. When the model becomes more realistic, such as unequal variances, clustered observations, or missing outcomes, simulation may remain feasible even when a simple power formula does not.
If σ is estimated, replace the known-variance test with the actual planned procedure, such as a two-sample t test. Simulating one analysis and reporting power for another answers the wrong question.

Example 20: Application: Predicting Defect Counts

Suppose a process produces independent binary defect indicators conditional on an unknown defect probability p. Start with a Beta$(1, 1)$ prior and observe 3 defects among 18 items. Beta-binomial conjugacy gives

$$p \mid \text{data} \sim \text{Beta}(4, 16).$$

To predict the number of defects Y_{new} in a future batch of $m = 50$ items, generate

$$p^{(b)} \sim \text{Beta}(4, 16), \qquad Y_{\text{new}}^{(b)} \mid p^{(b)} \sim \text{Binomial}(50, p^{(b)}).$$

Each simulated future batch uses one shared draw of its possible process rate. This integrates uncertainty about p into the prediction.
The plug-in alternative uses only $\bar{p} = 4/20 = 0.2$:

$$Y_{\text{plug-in}} \sim \text{Binomial}(50, 0.2).$$

Both have mean 10, but the posterior predictive variance is larger:

$$\begin{aligned}\text{Var}(Y_{\text{new}} \mid \text{data}) &= m\bar{p}(1-\bar{p})\frac{a+b+m}{a+b+1} \\ &= 50(0.2)(0.8)\frac{70}{21} \approx 26.67,\end{aligned}$$

compared with the plug-in variance $50(0.2)(0.8) = 8$.

Model	Sim. Var	Exact Var	Sim. tail	Exact tail
Posterior predictive	27.065	26.667	0.194	0.187
Plug-in	8.004	8.000	0.061	0.061

The tail columns report $P(Y \geq 15)$.

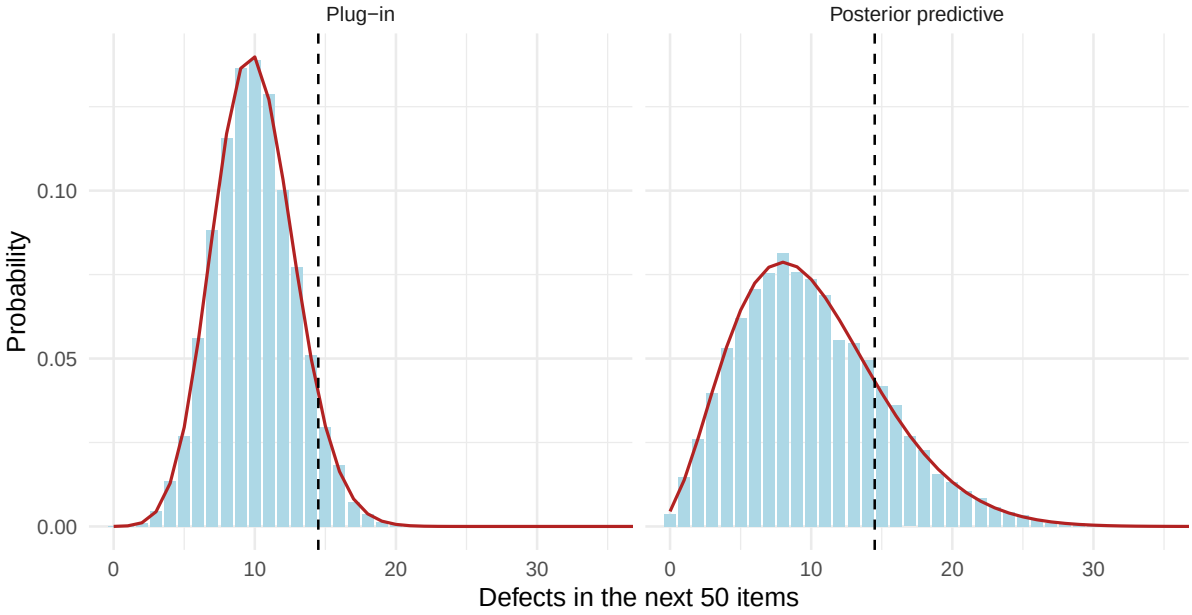


Figure 4.20: The posterior predictive distribution spreads probability over a wider range of defect counts. The plug-in model ignores uncertainty about the process rate. Lines show exact probabilities and bars show simulated proportions.

Interpretation and Shared Uncertainty

A decision triggered by a large defect count depends on the predictive tail, not just the expected count. Hierarchical simulation carries parameter uncertainty through to that tail.

If we drew a fresh independent p for each individual item instead of one shared p for the batch, the marginal total would be binomial with probability $\mathbb{E}(p)$. That would represent a different mechanism and lose the shared uncertainty in this posterior prediction.

Example 21: Application: Queue Delays

Consider a single server, first-come-first-served, initially idle. Let interarrival times be independent exponentials with rate λ, and service times be independent Gamma$(2, 1)$ variables, with mean 2 minutes. Assume the arrival and service processes are independent.
If A_i is the time between customers $i-1$ and i, S_i is customer i's service time, and W_i is the wait before service, then

$$W_1 = 0, \qquad W_i = \max\{0, W_{i-1} + S_{i-1} - A_i\}.$$

This recursion turns independent primitive random inputs into dependent waiting times. A long service can delay several later customers.
We compare arrival rates 0.35 and 0.45 per minute. Their utilization values are $\lambda\mathbb{E}(S) = 0.7$ and 0.9. Both are below one, but the system closer to capacity has less room to absorb bursts.

Arrival rate	Utilization	Fraction waiting > 5	MCSE
0.35	0.7	0.2385	0.0037
0.45	0.9	0.5642	0.0066

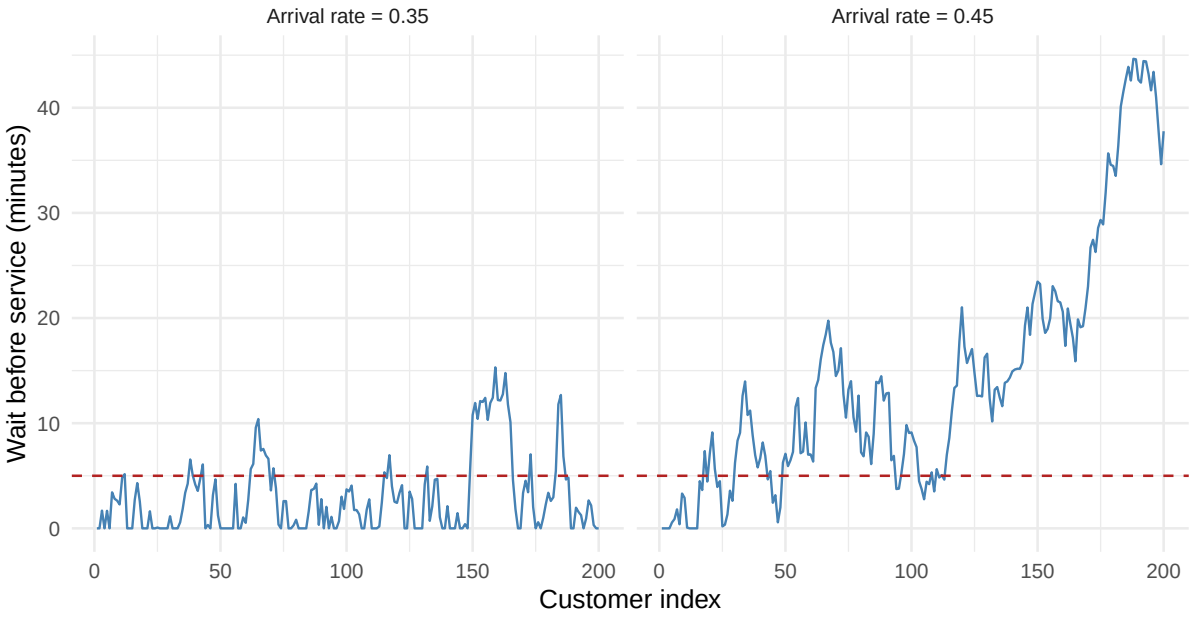


Figure 4.21: Two illustrative paths show how delays carry over between customers. Estimates in the table average 800 independent runs per scenario; one path alone can be misleading.

> **Interpretation and Assumptions**
>
> The estimand is the expected fraction of the first 200 customers who wait more than five minutes in a system starting empty. It is not automatically the long-run stationary probability. Studying steady-state performance requires addressing the initial transient and dependence.
> We compute MCSE from **independent run-level fractions**. Pooling all customers and treating their wait indicators as independent Bernoulli trials would ignore the dependence created by the queue.
> This example combines distribution choice, recursive computation, and statistical reasoning about the unit of replication.

References and Further Reading

This chapter adapts and expands the 2025 STAT 8670 chapter on generating random variables and its Quarto source in the master branch.

For software details, consult the official R documentation linked at the relevant examples. Useful local help pages include `?RNGkind`, `?Distributions`, `?qgeom`, `?chol`, and `?rWishart`. The course's original reading is Rizzo, M. L. (2007), *Statistical Computing with R*, Chapman & Hall/CRC.

Section 5

Monte Carlo Simulation and Variance Reduction

In Chapter 4, we learned to generate random values. Here we use them to answer a question about a probability model.

The key questions are conceptual:

1. **Why simulate?** The answer may be hard to calculate directly.
2. **What do we do?** Generate possible inputs, calculate their outputs, and summarize.
3. **Where does variation come from?** Fresh random inputs give a different set of outputs and a different estimate.
4. **What does variance reduction change?** It makes the estimate more stable while keeping the same target.

We begin with a shop deciding how many muffins to prepare, then use $\mathbb{E}[e^U]$ with $U \sim \text{Uniform}(0,1)$ to compare variance-reduction methods. Detailed formulas, proofs, and extra examples are in Supplementary.

Section 5.1 The Main Idea: Simulate, Calculate, Average

5.1.1 Why use simulation?

Suppose a model describes uncertain inputs, and we want its **average output** or the **probability of an event**. An exact calculation may involve a difficult integral or many inputs. If we can generate inputs from the model and calculate the corresponding outputs, simulation gives us a practical numerical estimate.

Simulation uses computer-generated outcomes to explore a specified model. More simulated outcomes improve numerical precision; they do not supply new real observations or make the model more realistic.

5.1.2 What happens in one run?

One **draw** produces one input and its output. One **run** repeats this process N times and reports one estimate:

1. **Simulate:** draw an input X_i from the specified probability model.
2. **Calculate:** use the model to obtain $Y_i = g(X_i)$.
3. **Average:** repeat independently and average the N outputs.

$$\theta = \mathbb{E}[g(X)], \qquad \hat{\theta} = \frac{1}{N}\sum_{i=1}^{N} Y_i.$$

Here θ is the fixed answer we want and $\hat{\theta}$ is the answer from this run. A **contribution** simply means one value Y_i going into the average.

5.1.3 A Practical Example: Are 12 Muffins Enough?

The question. A shop prepares 12 muffins each morning. What is the probability that demand exceeds the stock, so at least one customer cannot buy a muffin?

For this teaching example, **assume** 20 potential customers per day. Each independently wants one muffin with probability 0.6, whether or not stock is available. Days are independent, and these assumptions stay the same across runs. These numbers are illustrative, not estimates from shop data.

The average demand is $20 \times 0.6 = 12$, but **an average of 12 does not mean demand is always 12**. Simulation lets us try many artificial days and count how often the stock is insufficient.

Step 1: Simulate one day. For each of the 20 customers, generate a uniform number between 0 and 1. A number below 0.6 means the customer wants to buy. Count the buyers.

```
set.seed(8670)
wants_one <- runif(20) < 0.6
demand_today <- sum(wants_one)
c(demand = demand_today, insufficient = demand_today > 12)
#>       demand insufficient
#>           10            0
```

This artificial day has demand 10, so the stock is sufficient. A day with demand 15 would leave 3 customers unserved. **Demand 12 is still fully served**: our event is `demand > 12`, not `demand >= 12`.

Step 2: Repeat for many days and record 0 or 1. Use 1 when stock is insufficient and 0 otherwise. In R, `rbinom(N, size = 20, prob = 0.6)` is a shortcut that generates the number of buyers for each of N independent days using exactly the same model.

```
set.seed(8671)
N <- 1000
demand <- rbinom(N, size = 20, prob = 0.6)
insufficient <- as.integer(demand > 12)
c(days_short = sum(insufficient),
  estimated_probability = mean(insufficient))
#>            days_short estimated_probability
#>               426.000                 0.426
```

Step 3: Interpret the average. Stock was insufficient on 426 of the 1,000 simulated days, giving 42.6%. That is roughly **4 days out of 10 under this model**. It is an estimate of the chance for a day, not a prediction of exactly which future days will have a shortage.

Step 4: Rerun the whole experiment. Each run below simulates another 1,000 days and produces one estimated probability. We set the seed once before all the runs.

Run	Simulated days	Days with insufficient stock	Estimated chance
1	1000	403	40.3%
2	1000	417	41.7%
3	1000	418	41.8%

Why do the answers differ? The random buying decisions differ, so each run contains a different number of shortage days. The stock, purchase probability, and true shortage probability have not changed.

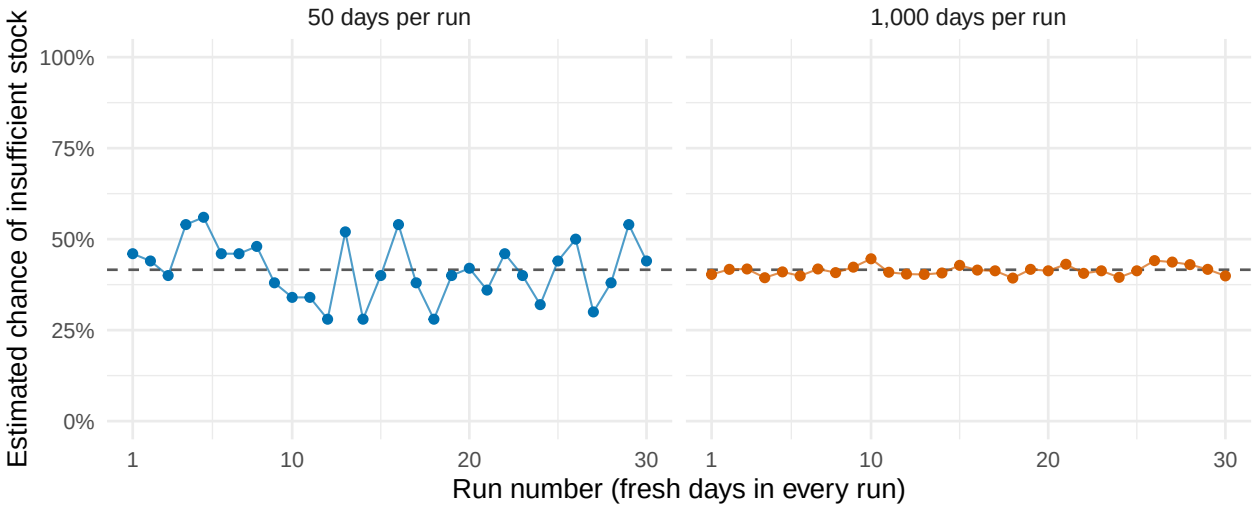


Figure 5.1: Each point is one complete simulation run, not one day. Both panels show 30 independent runs of the same shop model on the same vertical scale. Averaging 1,000 days gives a more stable estimate than averaging 50 days. The dashed line is the exact model probability, about 41.6%.

This simple model also has an exact binomial answer, about 41.6%, which checks our simulation. We practice the recipe here because it also works when the shop model becomes harder to calculate directly, such as when customers buy several items or demand changes during the day.

i Two different kinds of variation

- **Day to day:** customer demand varies. Even knowing the exact shortage probability would not tell us tomorrow's demand.
- **Run to run:** our estimated probability varies because each run uses only a finite number of simulated days. More simulated days reduce this numerical uncertainty; they do not make real customer demand less variable.

The practical answer is therefore: **12 muffins covers the average demand, but leaves about a 42% chance of insufficient stock under these assumptions.** Choosing a stock level also requires considering shortage and leftover costs.

5.1.4 A Small Mathematical Example

Example 1: One example to use throughout the chapter

Let $U \sim \text{Uniform}(0, 1)$. Estimate

$$\theta = \mathbb{E}[e^U] = \int_0^1 e^u \, du = e - 1 \approx 1.7183.$$

The exact answer lets us check our code. In harder problems, this answer may be unavailable. For one draw, if $U = 0.2$, the output is $e^{0.2} \approx 1.221$. This single output does not answer the average-output question; we need many draws.

```
set.seed(8670)
N <- 10000
u <- runif(N)              # simulate
plain_y <- exp(u)          # calculate one value per draw
c(estimate = mean(plain_y), exact = exp(1) - 1)  # average
#> estimate    exact
#> 1.726714 1.718282
```

Why does averaging help? Any one draw can be unusually high or low. In a large collection, those fluctuations tend to balance, and each individual draw has less influence on the final answer.

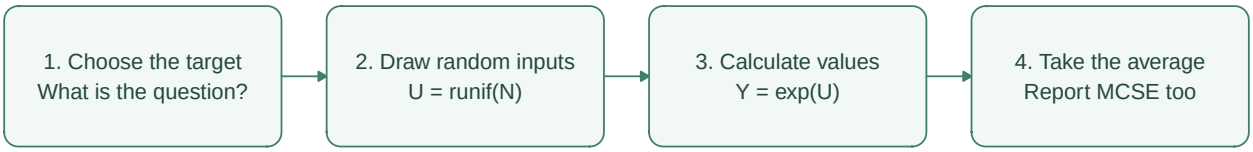


Figure 5.2: Choose the answer you want, simulate inputs, calculate one value per input, and take the average.

What to expect when you rerun the code

Fresh random draws usually give a different estimate. The model, function, and target stay the same. With independent draws and a finite mean, the average approaches the target as N grows, but it need not get closer after every draw.
Resetting to the **same seed** reproduces the same draws and answer. It does not remove simulation uncertainty. Set the seed once before a series of runs, so each run then receives fresh draws.

Section 5.2 Two Useful Translations

5.2.1 A probability is a proportion

Give each simulated outcome a **1** if the event occurs and **0** otherwise. The average of these zeros and ones estimates the event's probability.

Example 2: Estimate pi by counting points

Draw points uniformly in the unit square. The quarter circle has area $\pi/4$, so

$$\hat{\pi} = 4 \times \frac{\text{number of points inside the quarter circle}}{N}.$$

```
set.seed(8670)
N <- 4000
u <- runif(N)
v <- runif(N)
pi_contributions <- 4 * (u^2 + v^2 <= 1)
mean(pi_contributions)
#> [1] 3.127
```

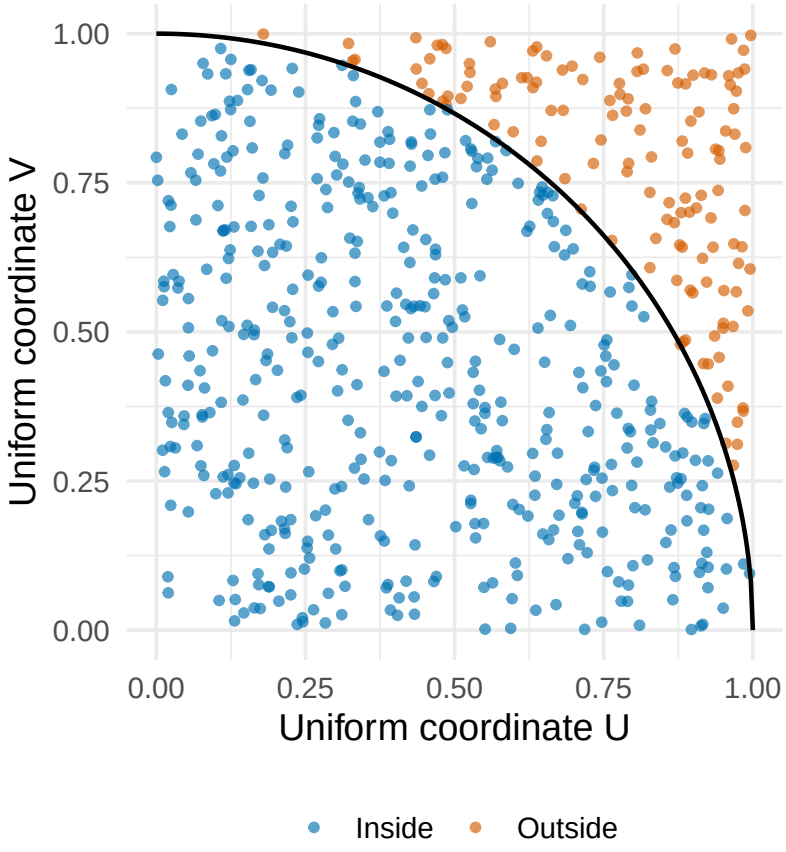


Figure 5.3: Uniform points turn the area ratio into a success probability. The plotted points are a subset of the simulation; the estimate uses all 4,000 points.

Why multiply by 4? The proportion inside estimates $\pi/4$. Multiplying by 4 gives an estimate of π.

5.2.2 An integral is width times average height

For $X \sim \text{Uniform}(a, b)$,

$$\int_a^b g(x)\,dx = (b-a)\mathbb{E}[g(X)].$$

This gives a simple recipe: **sample across the interval, average the function values, and multiply by the interval's width**.

Example 3: The width matters

For $\int_2^4 e^{-x}\,dx$, the interval has width $4-2=2$.

```
set.seed(8670)
N <- 10000
x <- runif(N, 2, 4)
y24 <- 2 * exp(-x)  # width times the function value
c(estimate = mean(y24), exact = exp(-2) - exp(-4))
#>  estimate     exact
#> 0.1161185 0.1170196
```

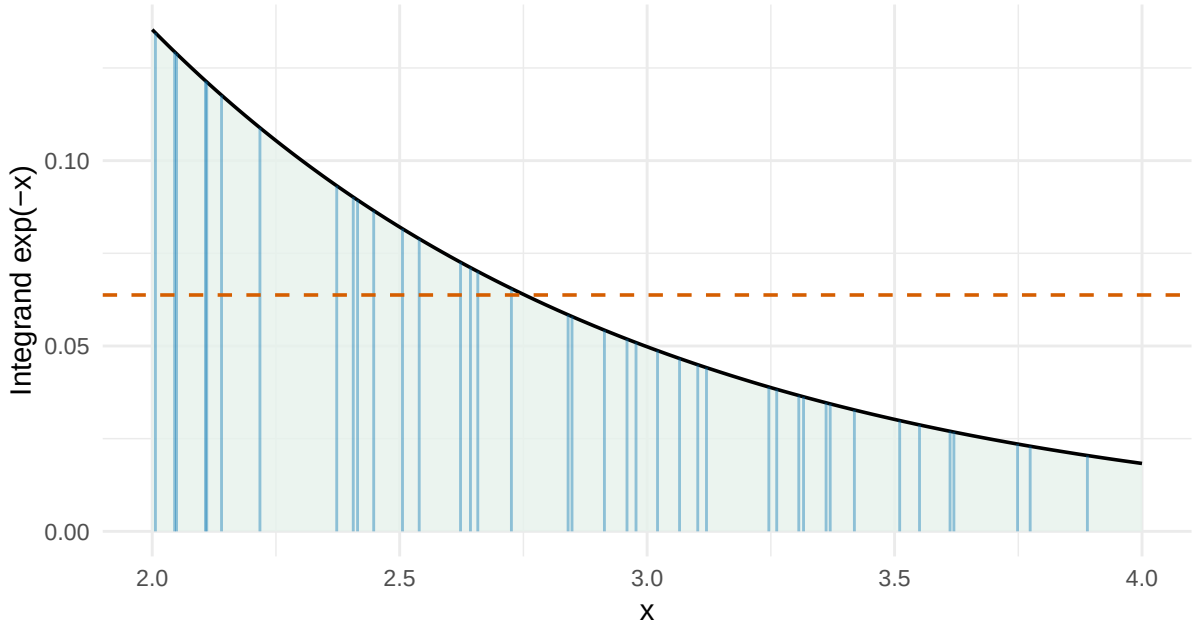


Figure 5.4: The orange dashed line is the sampled average height. Multiply it by width 2 to estimate the shaded area.

This curve is the **integrand**, not a probability density on $[2, 4]$. Its area is the integral we

want; it need not equal 1.

Exercise 5.1 (In Class: Find the Missing Factor)
A student uses `mean(exp(-x))` to estimate $\int_2^4 e^{-x}\,dx$. What factor is missing?

Solution 5.1
The width is 2. Use `2 * mean(exp(-x))`, or average `y24 <- 2 * exp(-x)`.

Section 5.3 Where Does the Variation Come From?

In our example, **`runif()` introduces the randomness**. Once U is known, `exp(U)` is determined. A different set of uniforms gives different outputs, so their average also changes.

There are two levels of variation:

Level	What changes?	What stays fixed?
Within one run	Individual inputs U_i and outputs e^{U_i}	The model and function
Across fresh runs	The average $\hat{\theta}$	The target $\theta = e - 1$

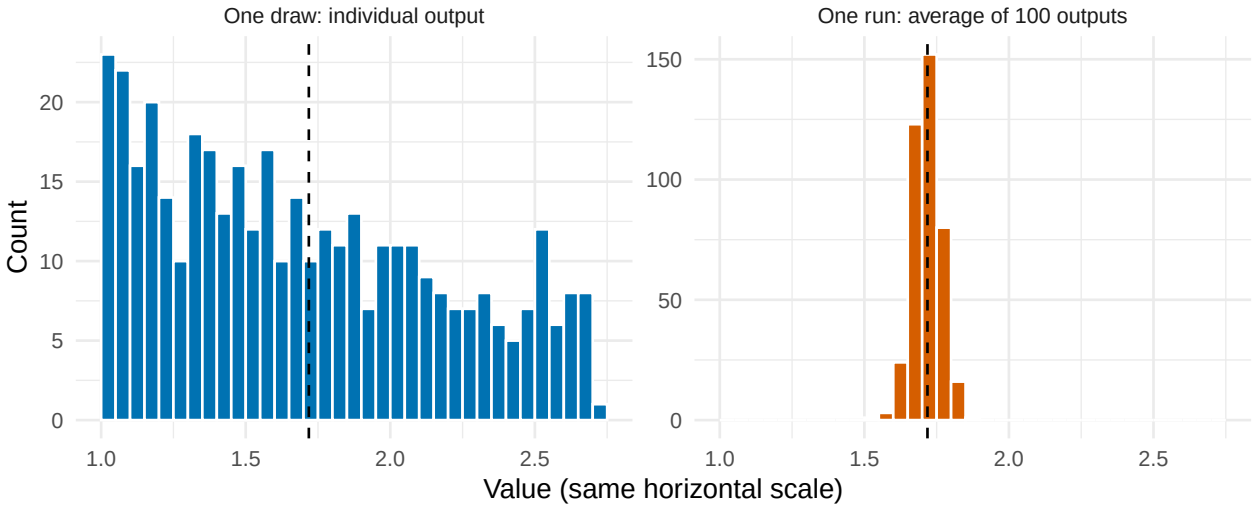


Figure 5.5: Left: 400 individual outputs. Right: 400 estimates, each averaging 100 fresh outputs. Both panels use the same horizontal scale. The dashed line is the fixed target e - 1; the averages cluster more tightly around it. The vertical scales differ.

Read the right panel carefully: one observation in that histogram is a whole run's average, not one simulated output. We repeat entire runs here to *see* the variability of the estimator.

5.3.1 SD describes outputs; MCSE describes the estimate

Quantity	What it describes
`sd(y)`	How much the individual simulated values vary
`sd(y) / sqrt(N)`	How much their average would vary across repeated simulations

The second quantity is the **Monte Carlo standard error**, or MCSE. We can estimate it from a single run; we do not need hundreds of runs to report it. For independent values with finite variance,

$$\widehat{\mathrm{MCSE}} = \frac{s_Y}{\sqrt{N}}.$$

```
set.seed(8670)
N <- 10000
u <- runif(N)
y <- exp(u)
estimate <- mean(y)
mcse <- sd(y) / sqrt(N)
c(estimate = estimate, MCSE = mcse)
#>    estimate        MCSE
#> 1.726713868 0.004941385
```

A smaller MCSE means less simulation noise. For a sufficiently large sample, an approximate 95% Monte Carlo interval is

$$\hat{\theta} \pm 1.96\,\widehat{\mathrm{MCSE}}.$$

```
c(lower = estimate - 1.96 * mcse,
  upper = estimate + 1.96 * mcse)
#>    lower    upper
#> 1.717029 1.736399
```

> ⚠ What this interval means
>
> It measures uncertainty caused by the **simulation**, with the probability model held fixed. It does not include uncertainty about whether the model matches reality or about parameters estimated from real data. A tiny MCSE does not fix a wrong formula or a coding mistake. For a rare event, seeing zero successes does not prove the probability is zero; the simple MCSE formula can be misleading in that case.

5.3.2 The rule of four

MCSE decreases like $1/\sqrt{N}$. To **halve MCSE**, use about **four times as many draws**.

Larger N makes the **average** more stable. Individual outputs still come from the same distribution and retain the same underlying variability.

For example, if 1,000 draws give MCSE 0.02, then 4,000 draws should give MCSE about 0.01. This is a planning approximation, not a guarantee for one run.

Exercise 5.2 (In Class: Halve the MCSE)
A run with 2,000 draws has MCSE 0.04. About how many draws are needed for MCSE 0.02?

Solution 5.2
Use about $4 \times 2000 = 8000$ draws.

Section 5.4 Variance Reduction: A More Stable Average

Increasing N is one way to improve precision. Another is to keep the same target but average **less variable values**.

We are reducing the variation of our **numerical estimate across runs**. We are not making the modeled system itself less random. Simply forcing every answer toward a convenient number would change the question; a valid method must preserve the target.

For the muffin shop, this means obtaining more stable estimates of the shortage probability **with the stock still fixed at 12**. Preparing more muffins changes the probability being estimated; that is a different stocking policy, not variance reduction.

We will try two methods:

Method	Idea
Antithetic variables	Pair a high value with a low value, then average the pair
Control variates	Use a quantity with a known mean to correct some of the noise

For a fair comparison, use the **same target** and count the work. In our examples, we give each method the same number of evaluations of `exp()`.

A method is better when its estimates vary less **across repeated experiments**. Being closer to the answer in one run can just be luck.

Section 5.5 Antithetic Variables: Pair Opposites

Keep the target $\theta = \mathbb{E}[e^U]$. Since both U and $1-U$ are uniform, both e^U and e^{1-U} have expectation θ.

If $U = 0.2$, the two values are approximately 1.221 and 2.226. Their average is 1.723. As U increases, one value rises and the other falls, so the pair average is more stable.

Algorithm 1: Three steps

1. Draw K independent uniforms.
2. For each uniform, calculate the **pair average**

$$A_k = \frac{e^{U_k} + e^{1-U_k}}{2}.$$

3. Average the K pair averages.

Each pair uses two evaluations of `exp()`: **500 pairs cost 1,000 evaluations**.

```
set.seed(8670)
K <- 500
u <- runif(K)
pair_y <- (exp(u) + exp(1 - u)) / 2
c(estimate = mean(pair_y), MCSE = sd(pair_y) / sqrt(K),
  exact = exp(1) - 1)
#>   estimate       MCSE      exact
#> 1.72051417 0.00282637 1.71828183
```

⚠ Count pairs when calculating MCSE

The two values within a pair are dependent. The **pair averages** are independent across pairs. Use `sd(pair_y) / sqrt(K)`, with K equal to the number of pairs.

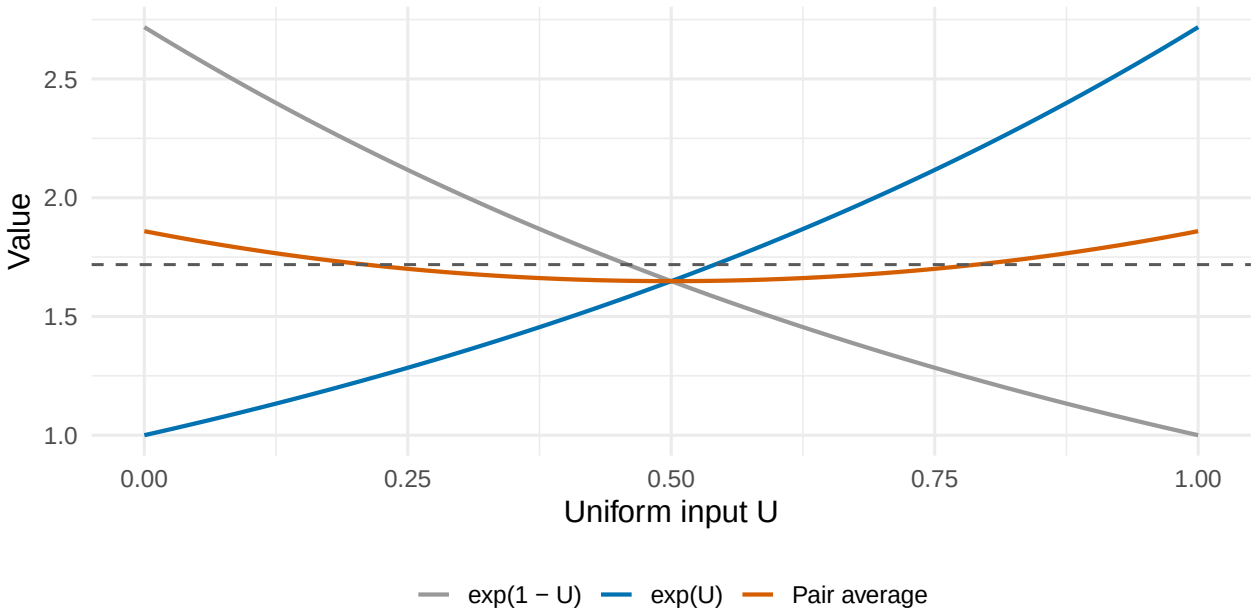


Figure 5.6: The pair average changes much less with U than either member of the pair. Its expectation is still e - 1 (dashed line).

When does it help? When the two output values tend to move in opposite directions. Reflection alone does not guarantee improvement. For example, $g(u) = (u - 1/2)^2$ gives identical values at u and $1 - u$, so the pair adds no new information.

Exercise 5.3 (In Class: What Is the Sample Size?)
You generate 500 independent pairs. Should the MCSE use `sqrt(500)` or `sqrt(1000)` in the denominator, and which values go into `sd()`?

Solution 5.3
Use `sd(pair_y) / sqrt(500)`: there are 500 independent pair averages.

Section 5.6 Control Variates: Use a Known Mean

Again, estimate $\mathbb{E}[e^U]$. We know that $\mathbb{E}[U] = 0.5$.

If the sampled uniforms happen to be large, the sampled values of e^U also tend to be large. We can use $U - 0.5$ to correct part of this fluctuation:

$$Y_{\text{adj}} = e^U + c(U - 0.5).$$

For any **fixed** c, the adjustment has expectation zero. It changes the variability while keeping the same target.

5.6.1 Which direction should the correction go?

We use $c = -1.7$, which is close to the best coefficient for this example:

- If the sampled inputs are unusually large on average, their outputs also tend to be large. The correction moves the estimate **down**.
- If the sampled inputs are unusually small on average, the correction moves the estimate **up**.
- Across repeated runs, the correction averages to **zero**, so the target stays the same.

The coefficient controls the size of this correction. Its optimal formula and derivation are in Supplementary.

```
set.seed(8670)
N <- 10000
u <- runif(N)
c_used <- -1.7
controlled_y <- exp(u) + c_used * (u - 0.5)
c(estimate = mean(controlled_y),
  MCSE = sd(controlled_y) / sqrt(N), exact = exp(1) - 1)
#>    estimate        MCSE       exact
#> 1.718693272 0.000625505 1.718281828
```

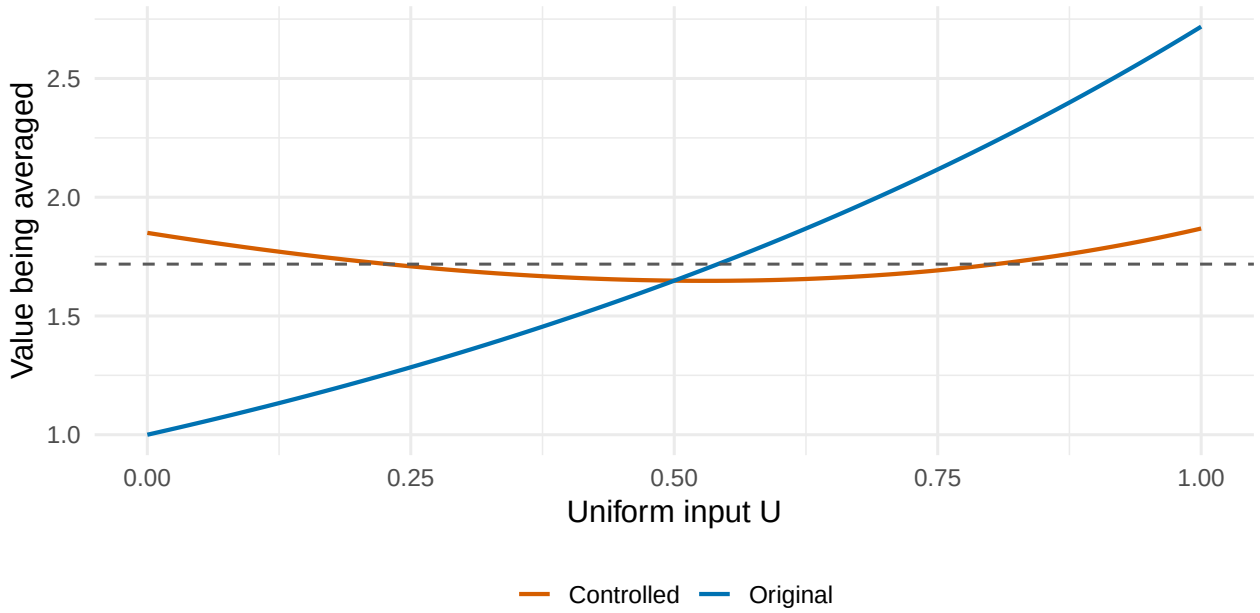


Figure 5.7: The adjusted curve is flatter. Both curves have mean e - 1 under uniform sampling; the dashed line marks that common target.

⚠ The control mean must be known

Use the known mean **0.5**, not `mean(u)`. Subtracting `mean(u)` would make the average adjustment zero and leave the original estimate unchanged.
If you need to estimate c, use a separate pilot sample and then keep c fixed for the main run.

A worked example is in Supplementary.

Exercise 5.4 (In Class: Which Sign Removes the Noise?)
In one run, `mean(u)` is 0.55 instead of the known mean 0.5. Would you expect `mean(exp(u))` to be too high or too low? With $c = -1.7$, which direction does the correction move it? Must this run become closer to the truth?

Solution 5.4
It tends to be too high. The average correction is $-1.7(0.55 - 0.5) = -0.085$, so it moves downward. Improvement is about smaller variability across repeated runs; a particular run is not guaranteed to be closer to the truth.

Section 5.7 Compare the Three Methods

We repeat the estimation of $\mathbb{E}[e^U]$ **800 times**. Every experiment gives each method 1,000 evaluations of `exp()`:

- **Plain:** average 1,000 values of e^U.
- **Antithetic:** average 500 pair averages.
- **Control:** average 1,000 adjusted values, using fixed $c = -1.7$.

Different experiments use fresh random draws. Less vertical spread in the boxplot means the method gives more stable answers.

Here $N = 1000$ is the work budget for one estimate, while $B = 800$ is how many times we repeat the experiment to compare methods. We keep the 800 estimates separate to inspect their spread.

Method	SD of repeated estimates	Variance removed (%)
Plain	0.01563	0.0
Antithetic	0.00276	96.9
Control	0.00203	98.3

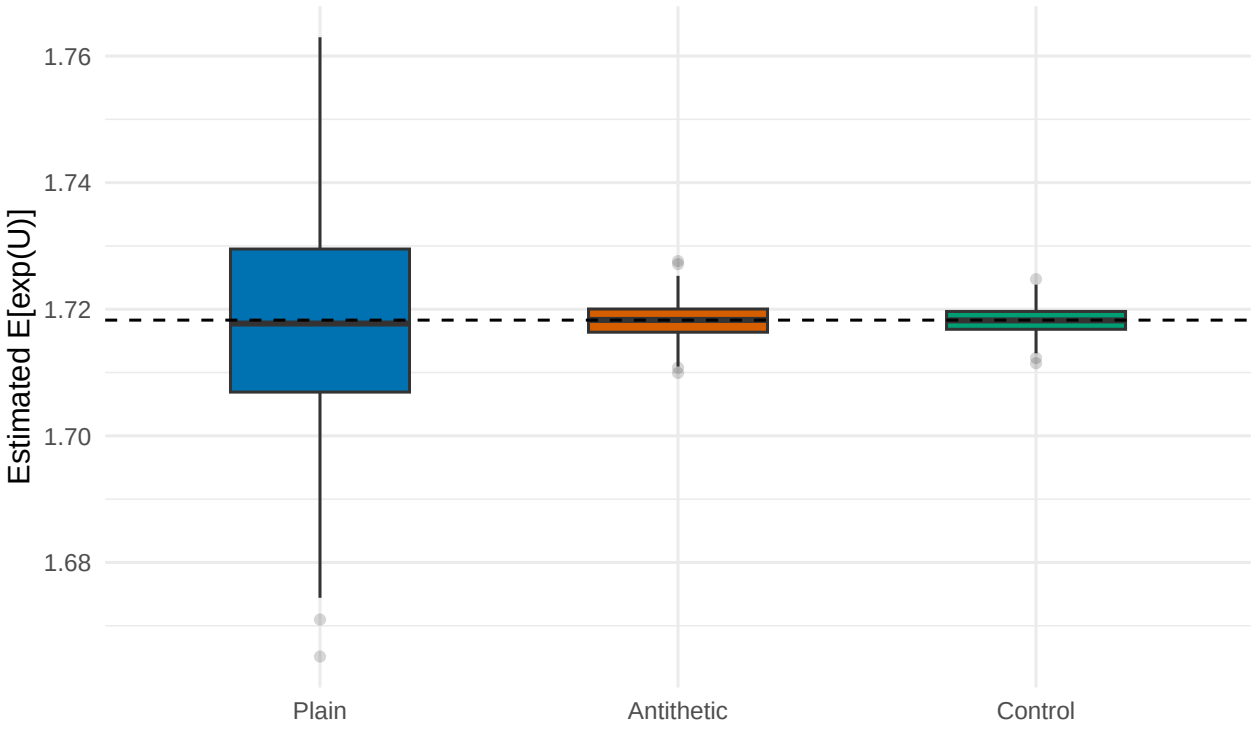


Figure 5.8: Each boxplot contains 800 estimates made with 1,000 evaluations per estimate. The dashed line is the true answer e - 1. Antithetic pairs and the fixed control adjustment reduce the spread.

What should students notice? All three methods aim at the same dashed line. Antithetic pairing and the control adjustment produce a tighter cluster of estimates. The improvement comes from how we construct the values being averaged.

These results apply to this example. A method's name alone does not guarantee a gain. If runtime matters, also count pilot runs and the work needed to calculate a control.

Section 5.8 Quick Check

Exercise 5.5 (True or False?)

1. A different random seed changes the target $\mathbb{E}[e^U]$.
2. Increasing N makes the average more stable, even though individual outputs still vary.
3. Reusing the same seed proves that the estimate has no simulation uncertainty.
4. Variance reduction aims to keep the same target while reducing the spread of estimates.

Solution 5.5

Item	Answer	Reason
1	F	The target is fixed; the random draws and estimate change.
2	T	Averaging reduces simulation noise in the estimate.
3	F	The same seed reproduces the same draws; fresh draws would give other estimates.
4	T	We improve the estimator's stability while preserving its expectation.

Exercise 5.6 (Check the Code)

1. For $\int_1^3 x^2\,dx$, you draw `x <- runif(N, 1, 3)`. What values should you average?
2. You have 500 antithetic pair averages stored in `pair_y`. Write the estimate and MCSE.
3. Why does `mean(exp(u) - 1.7 * (u - mean(u)))` equal `mean(exp(u))`?

Solution 5.6

1. Average `2 * x^2`, because the interval width is 2.
2. Use `mean(pair_y)` and `sd(pair_y) / sqrt(500)`.
3. The average of `u - mean(u)` is zero. Use the known mean 0.5 for the control adjustment.

Section 5.9 What to Remember

Simulation answers a model-based question by generating possible outcomes. One run produces one estimate. Fresh runs vary because their random inputs vary. MCSE describes this variation of the estimate; variance reduction makes it smaller while preserving the target.

Method	Values to average	MCSE
Plain MC	Individual values `y`	`sd(y) / sqrt(N)`
Antithetic	Pair averages `pair_y`	`sd(pair_y) / sqrt(K)`
Control	Adjusted values $Y_i + c(C_i - \mu_C)$	`sd(adjusted_y) / sqrt(N)`

Here N counts independent draws and K counts independent pairs. Keep the target unchanged, use the correct independent units, and compare methods at the same amount of work.

Section 5.10 Supplementary

The content here won't be on the exam

These examples are for independent study.

5.10.1 Why the average works

For independent, identically distributed values Y_i with $\mathbb{E}|Y_i| < \infty$, their average is unbiased for $\theta = \mathbb{E}[Y_i]$ and converges to θ almost surely. This is the strong law of large numbers.

If $\text{Var}(Y_i) = \sigma_Y^2 < \infty$, independence gives

$$\text{Var}(\overline{Y}) = \frac{\sigma_Y^2}{N}.$$

When $0 < \sigma_Y^2 < \infty$, the central limit theorem gives

$$\frac{\overline{Y} - \theta}{\sigma_Y / \sqrt{N}} \xrightarrow{d} N(0, 1),$$

which explains the approximate 95% interval. In R, `sd(y)` uses the sample-variance denominator $N - 1$.

5.10.2 Planning a sample size

For target MCSE ε, use approximately $N \geq \sigma_Y^2 / \varepsilon^2$. For a 95% interval half-width h, use approximately

$$N \geq \frac{1.96^2 \sigma_Y^2}{h^2}.$$

If the variance is unknown, estimate it in a pilot run. This gives a planning approximation, not a guaranteed bound.

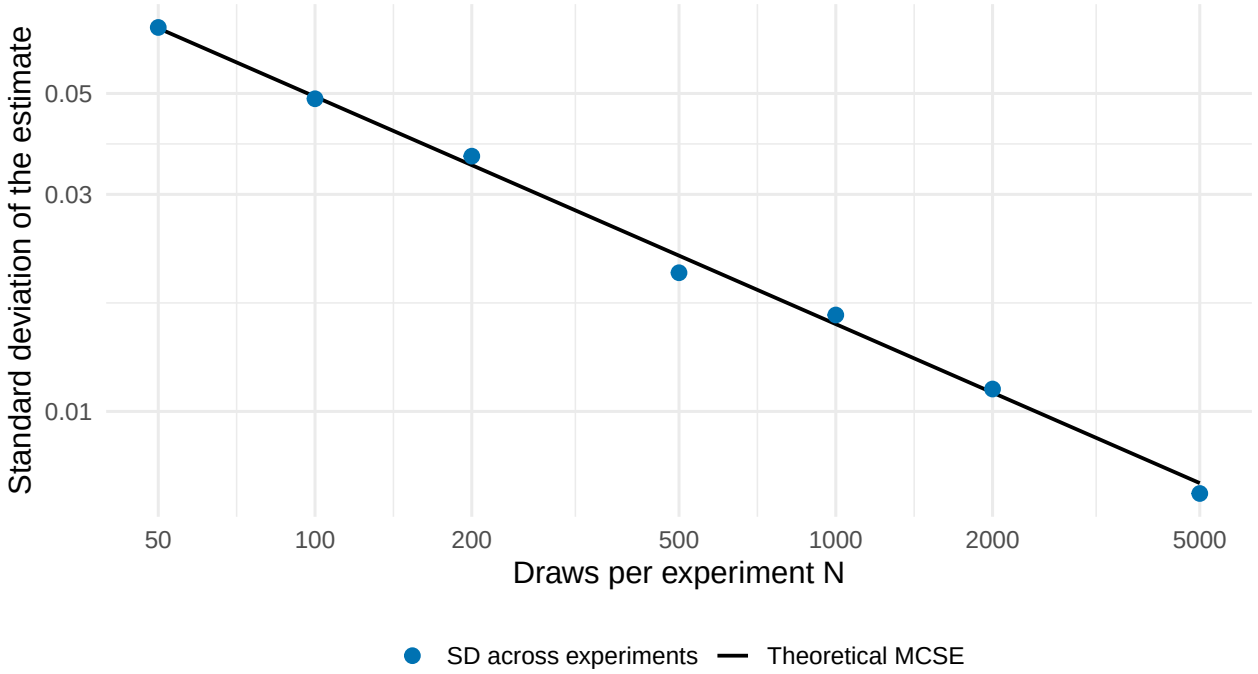


Figure 5.9: More draws reduce the spread of repeated estimates. The points come from 400 experiments at each sample size; the line shows the theoretical MCSE.

5.10.3 Reporting variance reduction

The percentage reported in our comparison table is

$$100\left(1-\frac{\text{variance of estimates with the new method}}{\text{variance of estimates with plain Monte Carlo}}\right).$$

If variance falls from 4 to 1, the reduction is 75%. Standard error falls from 2 to 1, a 50% reduction. Variance and standard error are on different scales.

5.10.4 Why antithetic pairing helps

Let $Y = g(U)$ and $Y' = g(1-U)$ have variance $\sigma^2 > 0$ and correlation ρ. For one pair average,

$$\operatorname{Var}\left(\frac{Y+Y'}{2}\right) = \frac{\sigma^2}{2}(1+\rho).$$

With K independent pairs and $N = 2K$ evaluations,

$$\operatorname{Var}(\hat{\theta}_{\text{anti}}) = \frac{\sigma^2}{N}(1+\rho), \qquad \operatorname{Var}(\hat{\theta}_{\text{plain}}) = \frac{\sigma^2}{N}.$$

Thus, negative correlation helps; positive correlation makes the estimator worse at this budget.

A monotone g with finite variance guarantees nonpositive covariance. To see why, let V be an independent copy of U. Then

$$2\,\mathrm{Cov}(a(U), b(U)) = \mathbb{E}[(a(U) - a(V))(b(U) - b(V))].$$

For $a(u) = g(u)$ and $b(u) = g(1-u)$, one function increases while the other decreases. The product is nonpositive.

5.10.5 Deriving the control coefficient

Assume Y and C have finite, positive variances. For fixed c and known $\mu_C = \mathbb{E}[C]$,

$$\mathrm{Var}\{Y + c(C - \mu_C)\} = \mathrm{Var}(Y) + c^2\,\mathrm{Var}(C) + 2c\,\mathrm{Cov}(Y, C).$$

Differentiate with respect to c and set the result to zero:

$$c^* = -\frac{\mathrm{Cov}(Y, C)}{\mathrm{Var}(C)}, \qquad \mathrm{Var}(Y_{c^*}) = \mathrm{Var}(Y)(1 - \rho_{YC}^2).$$

At the optimal coefficient, the fraction of variance removed is ρ_{YC}^2. Strong positive or negative correlation can both help.

For $Y = e^U$ and $C = U$, integration by parts gives $\mathbb{E}[Ue^U] = 1$. Therefore,

$$\mathrm{Cov}(e^U, U) = 1 - \frac{e-1}{2}, \quad \mathrm{Var}(U) = \frac{1}{12}, \quad c^* = 6e - 18 \approx -1.690309.$$

```
var_exp <- (exp(2) - 1) / 2 - (exp(1) - 1)^2
cov_exp_u <- 1 - (exp(1) - 1) / 2
c_star <- -cov_exp_u / (1 / 12)
var_optimal <- var_exp - cov_exp_u^2 / (1 / 12)
var_used <- var_exp + c_used^2 / 12 + 2 * c_used * cov_exp_u
c(best_coefficient = c_star,
  optimal_reduction_percent = 100 * (1 - var_optimal / var_exp),
  reduction_with_minus_1.7 = 100 * (1 - var_used / var_exp))
#>          best_coefficient optimal_reduction_percent  reduction_with_minus_1.7
#>                 -1.690309                 98.372048                 98.368815
```

The covariance is squared in the **variance reduction**, but not in the formula for c^*. Our fixed choice -1.7 gives nearly the same reduction as the optimum.

5.10.6 Estimating the coefficient in a pilot run

When c^* is unknown, estimate it using a separate pilot sample. Keep that estimate fixed, then generate fresh draws for the main run.

```r
set.seed(8670)
pilot_u <- runif(1000)
c_hat <- -cov(exp(pilot_u), pilot_u) / var(pilot_u)

production_u <- runif(10000)  # fresh draws; do not reset the seed
production_y <- exp(production_u) + c_hat * (production_u - 0.5)
c(coefficient = c_hat, mc_summary(production_y))
#>   coefficient      estimate          MCSE         lower         upper
#> -1.6987553233  1.7186253605  0.0006270268  1.7173963881  1.7198543330
```

Conditional on the pilot, the coefficient is fixed and the adjustment still has mean zero. Estimating and applying c on the same observations does not automatically preserve exact unbiasedness. Include the pilot cost when comparing total work.

5.10.7 Application: Estimate a Normal Probability

The original notes used $\Phi(x) = P(Z \le x)$ for $Z \sim N(0,1)$. We can estimate this probability in two ways.

Method 1: count successes. Draw standard normals and average the indicators of $Z \le x$.

```r
set.seed(8670)
x <- 2
N <- 10000
indicator_y <- as.numeric(rnorm(N) <= x)
c(estimate = mean(indicator_y),
  MCSE = sd(indicator_y) / sqrt(N),
  exact = pnorm(x))
#>    estimate        MCSE       exact
#> 0.978600000 0.001447209 0.977249868
```

Method 2: use a bounded integral. For $x > 0$, symmetry gives

$$\Phi(x) = \frac{1}{2} + \int_0^x \phi(z)\,dz = \frac{1}{2} + \mathbb{E}[x\phi(xU)], \qquad U \sim \text{Uniform}(0,1).$$

Thus we can average $0.5 + x\phi(xU)$.

```r
set.seed(8670)
u <- runif(N)
uniform_y <- 0.5 + x * dnorm(x * u)  # x > 0 here
c(estimate = mean(uniform_y),
  MCSE = sd(uniform_y) / sqrt(N),
  exact = pnorm(x))
#>    estimate        MCSE       exact
#> 0.973362243 0.002312272 0.977249868
```

Both methods estimate the same probability, but their variances differ. For $x < 0$, use symmetry, $\Phi(x) = 1 - \Phi(|x|)$; at $x = 0$, the answer is exactly 0.5.

We cannot draw uniformly over the infinite interval $(-\infty, x]$. The second method works because it uses the finite interval $[0, x]$.

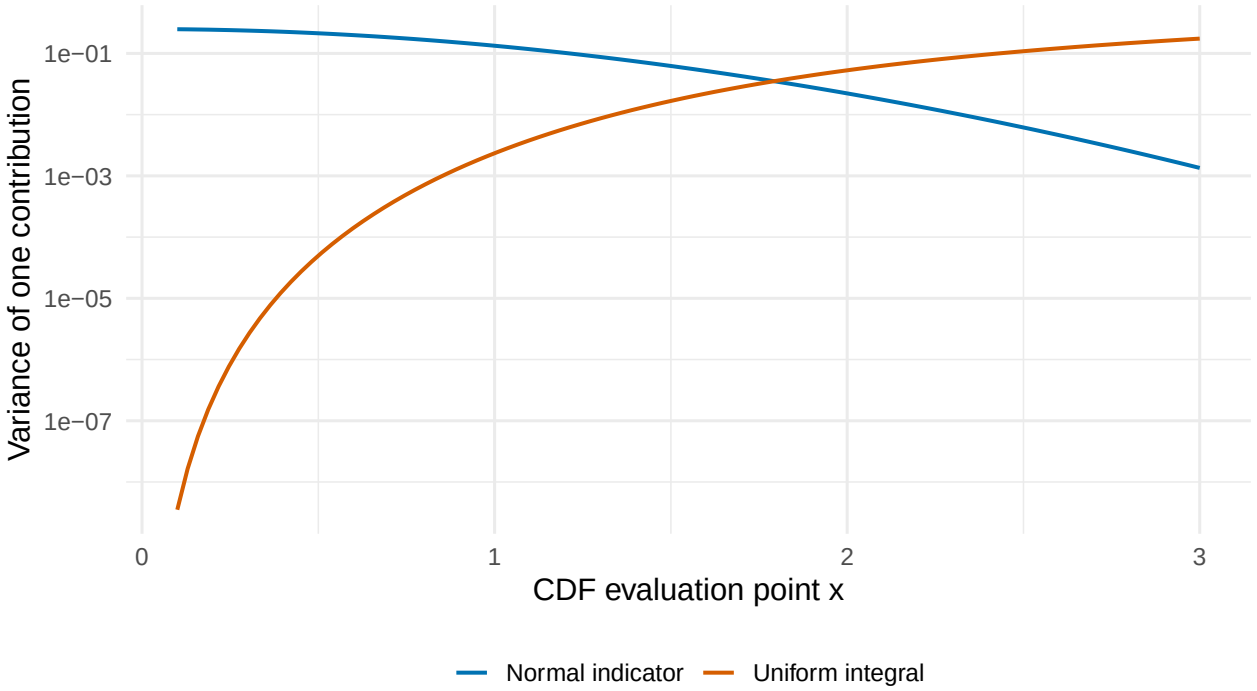


Figure 5.10: Two unbiased estimators can have different variances. The uniform-integral contribution is more stable near zero; the indicator is more stable farther into the positive tail. Divide the plotted variances by N for the variance of an ordinary Monte Carlo average.

The better choice depends on x. At the same number of independent draws, divide each plotted contribution variance by N to obtain the estimator variance. A contribution such as $0.5 + x\phi(xU)$ can exceed 1 even though its expectation is a probability.

5.10.8 Two more variance-reduction ideas

- **Importance sampling:** draw more often where the integrand matters, then correct for the changed sampling distribution with weights. For $\theta = \int g(x)f(x)\,dx$, draw $X \sim q$ and average $g(X)f(X)/q(X)$. The proposal must cover every region where $g(x)f(x) \neq 0$, and a finite second moment is needed for ordinary MCSE calculations.
- **Stratification:** divide the input space into groups with known probabilities, sample within each group, and combine the group averages using those probabilities as weights.

⚠ Normalization still matters

The ordinary importance-sampling formula above uses a normalized target density f. With only an unnormalized target, a self-normalized ratio estimator is a different method and is

generally biased at finite sample sizes. Cancellation of a constant in rejection sampling does not justify dropping a constant from an importance weight.

References and Further Reading

Adapted from the instructor's 2025 Monte Carlo notes. All figures are generated by the R code in this chapter.

- Rizzo, M. L. (2007), *Statistical Computing with R*, Chapter 6, Chapman & Hall/CRC.
- Wasserman, L. (2003), *All of Statistics*, Springer.
- R help: `?runif`, `?rnorm`, `?pnorm`, `?var`, and `?cov`.

Part I

Assignment

Assignment 1

Due date: Friday, September 11, 2026
Total: 30 points
Coverage: Chapter 1 (Introduction to R) and Chapter 2 (Computational Approaches).

Question	Topic	Points
1	R programming and statistical computing	10
2	Computational results using the iris data	10
3	R coding and visualization	6
4	Numerical linear algebra	4
Total		**30**

Submission

For this assignment, please use **R Markdown** or **Quarto**.
Submit both:

1. your source file (`.Rmd` or `.qmd`), and
2. the corresponding rendered PDF.

For multiple-choice questions, clearly indicate your selected answer.
For computational questions, include the R code used to obtain your answer.

Question 1: R Programming and Statistical Computing [10]

Select the **best answer**. Each question is worth **1 point**.

1a)

Which of the following creates a numeric vector in R?

A. `x <- list(1, 2, 3)`
B. `x <- c(1, 2, 3)`
C. `x <- data.frame(1, 2, 3)`
D. `x <- matrix(1, 2, 3)`

1b)

Which expression correctly returns the mean of `x` after removing missing values?

A. `mean(x, remove = TRUE)`
B. `mean(x, missing = FALSE)`
C. `mean(x, na.rm = TRUE)`
D. `mean(na.omit = x)`

1c)

Suppose

$$x = (10, 20, 30, 40).$$

What does the following R expression return?

```
x[c(1, 3)]
```

A. `10 20`
B. `10 30`
C. `20 40`
D. `30 40`

1d)

Which statement about R indexing is correct?

A. R indexing starts from 0.
B. R indexing starts from 1.
C. R indexing depends on the data type.
D. R indexing starts from -1.

1e)

Which function is most appropriate for applying a function to the **columns of a numeric matrix**?

A. `apply()`
B. `lm()`
C. `filter()`
D. `rbind()`

1f)

What is the main advantage of using `vapply()` instead of `sapply()`?

A. It always runs in parallel.
B. It requires a prespecified output type.
C. It works only on matrices.
D. It automatically removes missing values.

1g)

Which of the following is the native R pipe operator?

A. `%>%`
B. `->`
C. `|>`
D. `%*%`

1h)

Which `dplyr` function is used to retain rows satisfying a condition?

A. `select()`
B. `filter()`
C. `mutate()`
D. `summarise()`

1i)

Which statement concerning `apply()` and a `for` loop is most accurate?

A. `apply()` is always faster than a `for` loop.
B. A `for` loop is always faster than `apply()`.
C. Neither is always faster; performance depends on the computation.
D. `apply()` performs computations in parallel by default.

1j)

Why is reproducibility important in statistical computing?

A. It guarantees that every statistical model is correct.
B. It allows an analysis to be rerun and verified.
C. It eliminates sampling variability.
D. It eliminates numerical error.

Question 2: Computational Results Using the Iris Data [10]

Use the built-in `iris` dataset in R.

For each computational question, include the R code used to obtain your answer.

2a) [2]

Report the sample mean of `Sepal.Length`, rounded to three decimal places.

2b) [2]

Report the sample standard deviation of `Sepal.Width`, rounded to three decimal places.

2c) [2]

How many observations in the `iris` dataset belong to the species `virginica`?

A. 25
B. 40
C. 50
D. 75

2d) [2]

Fit the linear regression model

$$\texttt{Sepal.Length} = \beta_0 + \beta_1 \texttt{Sepal.Width} + \varepsilon.$$

Report the estimated slope coefficient $\hat{\beta}_1$, rounded to three decimal places.

2e) [2]

Based on the fitted model in part (d), which statement is correct?

A. Sepal length tends to increase as sepal width increases.
B. Sepal length tends to decrease as sepal width increases.
C. The estimated slope is exactly zero.
D. The model cannot be fitted because both variables are continuous.

Question 3: R Coding and Visualization [6]

Use the `iris` dataset for both parts.

3a) [3]

Using `ggplot2`, construct a scatter plot with:

- `Sepal.Width` on the x-axis,
- `Sepal.Length` on the y-axis,
- points coloured according to `Species`.

Your figure must contain:

1. appropriate axis labels,
2. an informative title, and
3. a readable legend.

Include both your R code and the resulting figure.

3b) [3]

Add a linear regression line to the plot from part (a).

Use

```
geom_smooth(method = "lm", se = FALSE)
```

and include the final plot in your report.

Question 4: Numerical Linear Algebra [4]

4a) [2]

Suppose Σ is a positive-definite covariance matrix with Cholesky decomposition

$$\Sigma = R^\mathsf{T} R,$$

where R is upper triangular.

Which expression correctly computes $\log\det(\Sigma)$?

A.

$$\sum_i \log(R_{ii})$$

B.

$$2\sum_i \log(R_{ii})$$

C.

$$\sum_i R_{ii}^2$$

D.

$$2\log\left(\sum_i R_{ii}\right)$$

4b) [2]

Let

$$R = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}, \qquad \Sigma = R^\mathsf{T} R.$$

Compute $\log\det(\Sigma)$.

A. $\log(6)$
B. $\log(12)$
C. $2\log(6)$
D. 6

Assignment 2

Due date: October 2, 2026, 23:59.
Total: 30 points
Coverage: Chapter 2 (Computational Approaches) and Chapter 3 (Optimization).

Question	Format	Points
1	8 multiple-choice questions on Chapter 2	8
2	8 multiple-choice questions on Chapter 3	8
3	6 true/false questions	6
4	2 short explanations with four prompts each	8
Total		**30**

Submission

Use **R Markdown** or **Quarto**. Submit both your **source file** (`.Rmd` or `.qmd`) and its **rendered PDF**.
Begin the PDF with an answer table for Questions 1-3, followed by Question 4. Use the supplied response template, or make your own table with the same part labels.

- **Questions 1-2:** Select one best answer, A, B, C, or D.
- **Question 3:** Write **True** or **False**.
- **Question 4:** Explain your reasoning in your own words, using one or two sentences per numbered prompt.

Only Question 4 requires explanations. R code and figures are optional. If you use R to check your work, include the code in your source file.

The questions use the main material in both chapters; the optional real-world applications at the end of Chapter 3 are not required.

Question 1: Computational Approaches [8]

Select the **best answer**. Each question is worth **1 point**.

1a)

Two formulas are mathematically equivalent in exact arithmetic. Which statement about their computer implementations is most accurate?
A. They can produce different numerical results because computers use finite-precision arithmetic.
B. They must produce exactly the same result in every decimal place.
C. They must use the same amount of memory.
D. They must take the same amount of time.

1b)

Let A be a nonsingular square matrix and b a compatible vector. Which R expression solves $Ax = b$ **without explicitly forming** A^{-1}?
A. `solve(A) %*% b`
B. `solve(A, b)`
C. `crossprod(A, b)`
D. `A %*% b`

1c)

For a numeric matrix `X`, which expression is mathematically equivalent to `crossprod(X)`?
A. `X %*% t(X)`
B. `X * X`
C. `sum(X * X)`
D. `t(X) %*% X`

1d)

A regression design matrix has one column that is almost a copy of another. What is the main numerical concern?
A. The number of observations must have changed.
B. The response variable must contain missing values.
C. The normal-equation system can be ill-conditioned, making coefficient estimates sensitive to small perturbations.
D. The residual sum of squares must be exactly zero.

1e)

Suppose a full-column-rank design matrix has the thin QR decomposition $X = QR$, with $Q^\top Q = I$. Which triangular system gives the least-squares coefficients?
A. $R\hat{\beta} = Qy$
B. $R\hat{\beta} = Q^\top y$
C. $Q\hat{\beta} = R^\top y$
D. $R^\top \hat{\beta} = Q^\top y$

1f)

Suppose $\Sigma = R^\top R$ is positive definite, with R upper triangular, and $z = x - \mu$. Which computation gives $z^\top \Sigma^{-1} z$ without explicitly forming an inverse?
A. Solve $R^\top v = z$, then compute $v^\top v$.
B. Solve $Rv = z$, then compute $v^\top v$.
C. Compute $v = Rz$, then compute $v^\top v$.
D. Compute $z^\top z$ without using R.

1g)

Use the same Cholesky convention, with

$$R = \begin{pmatrix} 4 & 1 \\ 0 & 2 \end{pmatrix}, \qquad z = \begin{pmatrix} 4 \\ 5 \end{pmatrix}, \qquad \Sigma = R^\top R.$$

What is $z^\top \Sigma^{-1} z$?
A. 29
B. 25
C. 5
D. 3

1h)

You evaluate multivariate normal log-densities at many different observations, using the **same positive-definite covariance matrix** Σ. Which work can be reused?
A. The centered vector $z = x - \mu$ for every observation.
B. The complete log-density value for every observation.
C. The quadratic form $z^\top \Sigma^{-1} z$ for every observation.
D. The Cholesky factor of Σ and its log-determinant.

Question 2: Optimization [8]

Select the **best answer**. Each question is worth **1 point**.

2a)

You want to maximize a log-likelihood $\ell(\theta)$ using an optimizer that **minimizes** its objective. Which objective should you supply?
A. $\ell(\theta)$
B. $[\ell(\theta)]^2$
C. $-\ell(\theta)$
D. $|\ell(\theta)|$

2b)

Which conditions give the usual root-bracketing guarantee for bisection on an interval $[a, b]$?
A. $f(a)$ and $f(b)$ are both positive.
B. f is differentiable at a, regardless of its behavior elsewhere.
C. The midpoint is smaller than both endpoints.
D. f is continuous on $[a, b]$ and $f(a)f(b) < 0$.

2c)

Apply **one Newton root-finding update** to $f(x) = x^2 - 3$, starting at $x_0 = 2$. What is x_1?
Use $x_1 = x_0 - f(x_0)/f'(x_0)$.
A. 1.75
B. 1.50
C. 2.25
D. 3.00

2d)

Minimize $Q(x) = (x - 3)^2$ using gradient descent. Starting at $x_0 = 0$ with step size $\alpha = 0.25$, what is the next iterate?
Use $x_1 = x_0 - \alpha Q'(x_0)$.
A. -1.50
B. 1.50
C. 0.75
D. 3.00

2e)

What is the main computational idea behind BFGS?
A. Enumerate every feasible parameter value.
B. Use the exact Hessian inverse at every iteration.
C. Guarantee a global optimum by adding random noise.
D. Update an approximation to curvature information instead of repeatedly computing the exact Hessian.

2f)

Consider $Q(\theta_1, \theta_2) = (\theta_1 - 1)^2 + 100(\theta_2 - 1)^2$. Why can a single fixed step size be difficult for gradient descent?
A. The much larger curvature in one direction can cause zig-zagging or force a small step size.
B. The function is not differentiable.
C. The function has infinitely many local minima.
D. The gradient is zero at every point.

2g)

In simulated annealing for a minimization problem, a proposed move increases the objective by $\Delta Q = 2$. At temperature $T = 1$, what is its acceptance probability?
Use $P(\text{accept}) = \exp(-\Delta Q/T)$.
A. 0
B. $e^{-2} \approx 0.1353$
C. 0.5
D. 1

2h)

A smooth, nonconvex objective is minimized with BFGS from several starting values. The returned objective values differ. What is the most appropriate next step?
A. Always keep the run with the fewest iterations.
B. Average all parameter vectors without evaluating the objective there.
C. Compare feasible returned objective values, examine diagnostics, and consider additional starts.
D. Declare the first converged result globally optimal.

Question 3: True or False [6]

Write **True** or **False** for each statement. Each statement is worth **1 point**. No explanation is required.

3a)

For a full-column-rank regression design matrix, a normal-equation implementation and a QR implementation must return coefficients that agree in every floating-point digit.

3b)

When optimizing a multivariate normal log-likelihood over μ and Σ with fixed dimension d and sample size, dropping an additive term that is constant in these parameters does not change the maximizer.

3c)

Replacing an explicit matrix inverse with a direct linear-system solve guarantees accurate coefficients even when the matrix is arbitrarily ill-conditioned.

3d)

A solution of $Q'(x) = 0$ can be a local maximum rather than a local minimum.

3e)

Under suitable smoothness conditions and with a starting point sufficiently close to a simple root, Newton's method has quadratic convergence.

3f)

A genetic algorithm uses one evolving candidate solution, whereas simulated annealing maintains a population of candidate solutions.

Question 4: Explain What the Results Mean [8]

Each part is worth **4 points**: **1 point for each numbered prompt**. Use one or two sentences per prompt. We grade the statistical and computational reasoning, not the exact wording of your response.

4a) Rank deficiency is not missing data [4]

Suppose a regression design matrix contains an intercept and two predictors, with $x_2 = 2x_1$ exactly and x_1 nonconstant. There are no missing values. You try

```
solve(crossprod(X), crossprod(X, y))
```

and receive a computationally singular system error. A QR-based fit instead reports the numerical rank and returns `NA` for one redundant coefficient.

Explain:

1. What feature of the design matrix causes the normal-equation solve to fail?
2. What does the `NA` coefficient mean in this situation?
3. Can a QR-based approach still provide ordinary least-squares fitted values? Explain how.
4. What is one sensible way to remove the redundancy while retaining the same ordinary least-squares fitted values?

4b) A converged result still needs interpretation [4]

You are minimizing

$$Q(x) = (x^2 - 1)^2 + 0.2x + 1, \qquad x \in \mathbb{R}.$$

Two BFGS runs give the following results, rounded to four decimal places:

Run	Starting value	Returned value	Objective value $Q(x)$	Convergence code
A	1	0.9740	1.1974	0
B	-1	-1.0241	0.7976	0

Here, code `0` means the optimizer reports successful completion according to its convergence criteria. You do not need to run the optimizer or prove global optimality.

Explain:

1. Which returned point would you retain as the **best found so far**, and why?
2. How can a deterministic method return different answers for the same objective?
3. Do these convergence codes alone establish that either point is a global minimum? Explain.
4. Describe one additional check or search you would perform, and explain what it would tell you.

Assignment 3

Due date: TBA.
Total: 30 points
Coverage: Chapter 4: Generating Random Variables and Chapter 5: Monte Carlo Simulation and Variance Reduction.

Question	Format	Points
1	8 multiple-choice questions on random variable generation	8
2	8 multiple-choice questions on simulation and variance reduction	8
3	6 true/false questions	6
4	2 short-answer parts with four prompts each	8
Total		**30**

Assignment PDF | Response template (.qmd)

Submission

Use **R Markdown** or **Quarto**. Submit your **source file** (`.Rmd` or `.qmd`) and **rendered PDF** to the **Assignment 3** folder on **iCollege**. Name both files `A3_LastName_FirstName` with the appropriate extension, and include your name in the PDF.
Use the response template, or follow its answer table and question labels.

- **Questions 1-2:** Select one best answer, A, B, C, or D.
- **Question 3:** Write **True** or **False**.
- **Question 4:** Give a short explanation or calculation for each prompt.

Each item is worth **1 point**. Only Question 4 requires explanations or calculations.
R code and plots are optional. If you use R, include the code in your source file. Before generating random values, set a seed (e.g., `set.seed(8670)`) and keep that line in your code. Check that your source renders to PDF without errors.

Note: The questions cover the main material in Chapters 4 and 5. Supplementary sections, proofs, and derivations of optimal variance-reduction coefficients are not required.

Question 1: Generating Random Variables [8]

Select the **best answer**. Each item is worth **1 point**.

1a)

Consider the following R code:

```r
x <- numeric(100)
for (i in seq_len(100)) {
  set.seed(8670)
  x[i] <- runif(1)
}
```

What is the main problem with using `x` as 100 independent uniform draws?
A. The values must all be zero.
B. The values are sampled without replacement.
C. Resetting the seed repeats the same first draw in every iteration.
D. A uniform generator cannot be used inside a loop.

1b)

Let $U \sim \text{Uniform}(0, 1)$. Which expression generates an exponential random variable with **mean 5**?
A. $-\log(1-U)/5$
B. $-5\log(1-U)$
C. $\exp(-5U)$
D. $5U$

1c)

The Beta$(2, 2)$ density is $f(x) = 6x(1-x)$ for $0 < x < 1$, and zero otherwise. Its height at $x = 0.5$ is 1.5. Which statement is correct?
A. It cannot be a density because its height exceeds 1.
B. $P(X = 0.5) = 1.5$.
C. The area under the curve is 1.5.
D. It is a valid density: its total area is 1, even though its height can exceed 1.

1d)

An unnormalized target is $h(x) = x(1-x)$ on $(0, 1)$, with total area $1/6$. Which formula turns it into a normalized density?
A. $f(x) = 6h(x)$
B. $f(x) = h(x)/6$
C. $f(x) = h(x) - 1/6$
D. $f(x) = h(x)$

1e)

For the target in 1d), compare two rejection samplers with a uniform proposal on $(0, 1)$:

- normalized scale: target $f(y) = 6y(1 - y)$, envelope height 1.5;
- unnormalized scale: target $h(y) = y(1 - y)$, envelope height 0.25.

Suppose both samplers use the same proposal Y and the same draw $U \sim \text{Uniform}(0, 1)$, independent of Y. How do their acceptance decisions compare?
A. The normalized version accepts six times as often.
B. The unnormalized version cannot be used for rejection sampling.
C. Both accept exactly when $U \leq 4Y(1 - Y)$.
D. Both accept exactly when $U \leq Y(1 - Y)$.

1f)

Use the normalized sampler in 1e). For one proposal, $Y = 0.75$ and $U = 0.60$. Its acceptance ratio is $f(Y)/1.5 = 0.75$. What should the sampler do?
A. Accept and store 0.75.
B. Accept and store 0.60.
C. Reject because $U < 0.75$.
D. Accept and store the density height 1.125.

1g)

For the sampler in 1e), a proposal is plotted at (Y, V) with $V = 1.5U$, where U is uniform on $(0, 1)$ and independent of Y. What does the point's height V represent?
A. The probability that the target equals Y.
B. The value of the target density $f(Y)$.
C. A second accepted draw from the target distribution.
D. An auxiliary random height between zero and the envelope, used to decide whether to accept Y.

1h)

An envelope lies above the target at 100 grid points. What else must be checked before using it for rejection sampling? Assume the proposal covers the target support.
A. The number of proposals is at least 100.
B. The envelope is at least as high as the target throughout its support.
C. The envelope has total area exactly 1.
D. Every proposal is accepted.

Question 2: Simulation and Variance Reduction [8]

Select the **best answer**. Each item is worth **1 point**.

2a)

Why use simulation to estimate an event probability when a direct calculation is difficult?
A. It guarantees that the chosen model matches the real world.
B. It removes all uncertainty after a single run.
C. It generates repeated outcomes from a model and estimates how often the event occurs.
D. It replaces the event probability by the mean of the model's inputs.

2b)

A shop has 12 muffins each day. The vector `demand` contains the number of customers wanting one muffin on each simulated day. Which R expression estimates the probability that **at least one customer cannot be served**?
A. `mean(demand > 12)`
B. `mean(demand)`
C. `sum(demand > 12)`
D. `mean(demand >= 12)`

2c)

Two simulation runs use the same model and number of days, but independent random draws. Why can their estimated shortage probabilities differ?
A. The stock level must have changed.
B. The model's true shortage probability changes after each run.
C. One of the runs must contain a coding error.
D. Different finite samples contain different numbers of shortage days.

2d)

A simulation averages independent outputs $Y_1, \ldots, Y_N$. Which statement correctly distinguishes the output standard deviation (SD) from the Monte Carlo standard error (MCSE)?
A. Both measure only the spread of individual Y_i values.
B. SD describes the spread of individual outputs; MCSE describes the uncertainty of their average due to simulation.
C. SD describes the uncertainty of the average; MCSE describes individual outputs.
D. MCSE measures how wrong the model's assumptions are.

2e)

With 1,000 independent simulated days, an estimated probability has MCSE about 0.02. With the same model and method, what MCSE should we expect using 4,000 days?
A. About 0.01.
B. About 0.005.
C. About 0.02.
D. Exactly zero.

2f)

What is the purpose of variance reduction?
A. Change the quantity of interest to one with a smaller numerical value.
B. Make real customer demand less variable by simulating more days.
C. Estimate the same quantity with less variation across repeated runs for a similar amount of computation.
D. Select the random seed that happens to give the closest answer.

2g)

An antithetic method generates 500 independent $U_j \sim \text{Uniform}(0,1)$ draws and forms the 500 pair averages $A_j = \{e^{U_j} + e^{1-U_j}\}/2$. Which values are the independent units used in $\text{sd}(\cdot)/\sqrt{N}$ to estimate MCSE?
A. The 1,000 separate function values, treating all of them as independent.
B. The 500 pair averages, so MCSE is $\text{sd}(A)/\sqrt{500}$.
C. Only the first pair average.
D. The two values in any single pair.

2h)

For $U \sim \text{Uniform}(0,1)$, a control-variate method uses $Y_{\text{adj}} = e^U - c(U - 0.5)$, with a fixed coefficient c. Why does this adjustment preserve the target mean $\mathbb{E}[e^U]$?
A. The adjusted output equals e^U for every draw.
B. Any constant can replace 0.5 without changing the mean.
C. The adjustment always makes the output positive.
D. The subtracted term has mean zero because $\mathbb{E}[U] = 0.5$.

Question 3: Interpreting Simulation [6]

Write **True** or **False** for each statement. Each item is worth **1 point**. No explanation is required.

3a)

Setting the seed once at the start allows the same simulation to be reproduced when the code and computing environment are unchanged.

3b)

Increasing the number of simulated days makes real customer demand less variable from day to day.

3c)

In rejection sampling, increasing a valid envelope by a constant factor greater than 1 preserves the distribution of accepted values but lowers the acceptance rate.

3d)

Pairing U and $1 - U$ must reduce Monte Carlo variance for every possible function of U.

3e)

A small MCSE guarantees that the model assumptions accurately describe the real world.

3f)

A comparison of simulation methods should examine the variability of repeated estimates of the same quantity, using a similar amount of computation per run.

Question 4: Simulating Daily Demand [8]

Each item is worth **1 point**. Label your answers **4a.1-4a.4** and **4b.1-4b.4**. Keep explanations to one sentence and show the requested calculations.

4a) Estimating the shortage probability [4]

A shop prepares 12 muffins each morning and has 20 potential customers per day. Assume that each customer independently wants one muffin with probability 0.6 and that demand is independent across days.

Let D be the number of customers wanting a muffin in one day. We want to estimate $p = P(D > 12)$, the probability that at least one customer cannot be served.

1. The mean demand is $20 \times 0.6 = 12$. Why might 12 muffins still not be enough?
2. Describe how to simulate one day's demand D using independent uniform draws. Specify the number of draws and how each draw determines whether a customer wants a muffin. A line of R code is also acceptable.
3. Complete the indicator rule: $Y = 1$ if _____, and $Y = 0$ otherwise. Write the condition as an inequality involving D.
4. A run of 1,000 independent days includes 430 shortage days. Compute $\hat{p}$ as a fraction and a decimal.

4b) Variation between runs [4]

A second run with the same assumptions and 1,000 new simulated days produces 397 shortage days.

1. The two estimates are 0.430 and 0.397. Why can they differ even if both programs are correct?
2. For the first run, use $\hat{p} = 0.430$ and $N = 1{,}000$ to compute $\widehat{\text{MCSE}} = \sqrt{\hat{p}(1-\hat{p})/N}$. Give the value to **four decimal places**.
3. If N increases to 4,000 and $\hat{p}$ stays close to 0.430, what approximate MCSE do you expect? Give the value to **four decimal places**.
4. When only the number of simulated days increases, complete each statement with **decreases**, **stays the same**, or **increases**: the spread of $\hat{p}$ across runs _____; the day-to-day spread of demand D under the model _____.

Part II

Exam

Exam 1

Term: 2026F (Fall 2026)
Date: October 7, 2026
Status: Completed
Time: 75 minutes
Total: 100 points
Coverage: Chapter 1 (R Programming), Chapter 2 (Computational Approaches), and Chapter 3 (Optimization).

Question	Format	Points
1	9 multiple-choice questions, 6 points each	54
2	6 true/false questions, 5 points each	30
3	3 short conceptual responses: 3a(i), 3a(ii), and 3b(i)	16
Total		**100**

i Exam instructions

- Work independently. Only a **non-programmable calculator** is permitted as an electronic device.
- **Questions 1–2:** Select one best answer, A–D, or write **True** or **False**. No explanations are required.
- **Question 3:** Answer each numbered part in one or two sentences. Brief formulas or R expressions may be used.

! Solutions

Expand the **Answer** below each question to view the solution and explanation. A compact answer key and the Question 3 grading rubric appear at the end.

Question 1: Multiple Choice [54]

Select the **single best answer**. Each part is worth **6 points**.

1a)

Let `x <- c(4, NA, 9, -1)`. Which expression returns exactly the nonmissing values greater than zero, with no `NA` in the result?

A. `x[x > 0]`
B. `x[is.na(x) | x > 0]`
C. `x[!is.na(x) & x > 0]`
D. `x[!is.na(x)]`

Answer

Answer: C. The mask must exclude missing values and require positivity. The result is `c(4, 9)`.

1b)

Let `fit <- list(coef = 1.2, se = 0.4)`. How do `fit["coef"]` and `fit[["coef"]]` differ?

A. Both return a numeric scalar.
B. The first returns a one-element list; the second returns its numeric element.
C. The first returns a numeric scalar; the second returns a list.
D. The second expression is invalid because lists cannot have names.

Answer

Answer: B. Single brackets retain a sublist; double brackets extract the element.

1c)

What does `c(1, 2, 3, 4) + c(10, 20)` return in R?

A. `c(11, 22, 13, 24)`
B. `c(11, 12, 23, 24)`
C. `c(11, 22)`
D. An error because the vector lengths differ.

Answer

Answer: A. The shorter vector is recycled as `c(10, 20, 10, 20)`.

1d)

In floating-point arithmetic, a product of many mathematically positive, very small densities is stored as zero. Which change best addresses this numerical problem?

A. Take the logarithm of the already computed zero.
B. Round every density to fewer decimal places.
C. Replace the likelihood by the sum of the densities.
D. Evaluate log-densities directly and add them.

Answer

Answer: D. Accumulate log-densities before the product underflows. Taking a logarithm after underflow cannot recover the lost information.

1e)

Two upper triangular Cholesky factors R_1, R_2 have the same size and the same positive diagonal entries in corresponding positions, but different off-diagonal entries. For $\Sigma_i = R_i^\top R_i$, what must be equal?

A. Every entry of Σ_1 and Σ_2.
B. The log-determinants of Σ_1 and Σ_2.
C. Every correlation in the two covariance matrices.
D. The quadratic forms $\mathbf{z}^\top \Sigma_1^{-1} \mathbf{z}$ and $\mathbf{z}^\top \Sigma_2^{-1} \mathbf{z}$ for every $\mathbf{z}$.

Answer

Answer: B. The determinant of a triangular matrix depends only on its diagonal. Off-diagonal changes can alter covariance entries and quadratic forms.

1f)

Let $X \in \mathbb{R}^{n \times p}$ have full column rank but nearly dependent columns. In floating-point arithmetic, why can QR be preferable to forming $X^\top X$ to solve $\min_{\boldsymbol{\beta}} \|\mathbf{y} - X\boldsymbol{\beta}\|_2^2$?

A. Forming $X^\top X$ can magnify the conditioning problem; QR avoids this step.
B. QR changes the response variable to remove all noise.
C. QR makes the original predictors statistically independent.
D. QR guarantees exact coefficients for every input matrix.

Answer

Answer: A. For full column rank, $\kappa_2(X^\top X) = \kappa_2(X)^2$. QR avoids forming the Gram matrix but cannot remove inherent sensitivity.

1g)

Bisection is applied to a continuous function on $[a, b]$. At $m = (a + b)/2$, $f(a) < 0$, $f(m) > 0$, and $f(b) > 0$. Under the standard bisection update, which half-interval retains an endpoint sign change?

A. $[m, b]$.
B. $[a, b]$ without shortening it.
C. $[a, m]$.
D. An interval centered at b and lying outside $[a, b]$.

Answer

Answer: C. The sign change is between a and m, so that half retains a root.

1h)

For $f(x) = a(x - x^*)^2/2$ with $a > 0$, gradient descent uses $x_{k+1} = x_k - \alpha f'(x_k)$, $\alpha > 0$. The errors $x_k - x^*$ alternate in sign and $|x_{k+1} - x^*| > |x_k - x^*|$. What is the most appropriate adjustment?

A. Increase the step size further.
B. Replace the gradient by zero.
C. Stop and declare the oscillation proof of multiple local minima.
D. Reduce the step size sufficiently or use a suitable line search.

Answer

Answer: D. Here $x_{k+1} - x^* = (1 - \alpha a)(x_k - x^*)$. Alternating, growing errors imply $\alpha a > 2$; a step with $0 < \alpha a < 2$ gives convergence.

1i)

When minimizing Q, simulated annealing accepts an uphill move with fixed $\Delta Q = Q_{\text{new}} - Q_{\text{current}} > 0$ with probability $\exp(-\Delta Q/T)$, where $T > 0$. What happens as T increases?

A. The acceptance probability becomes zero.
B. The acceptance probability increases.
C. The acceptance probability decreases.
D. The probability is unchanged because ΔQ is fixed.

Answer

Answer: B. At higher temperature, $-\Delta Q/T$ is less negative, so uphill moves are accepted more often.

Question 2: True or False [30]

Write **True** or **False**. Each part is worth **5 points**; no explanation is required.

2a)

A data frame contains one character column and one numeric column. After converting the entire data frame with `as.matrix()`, the column that was numeric still has numeric type.

Answer

Answer: False. The character column forces a common character type in the resulting matrix, including the formerly numeric column.

2b)

If a computed scalar $\hat{r}$ should equal zero in exact arithmetic, checking $|\hat{r}| < \varepsilon$ for a suitably chosen tolerance $\varepsilon > 0$ can be more appropriate than requiring $\hat{r} = 0$.

Answer

Answer: True. Rounding error can leave a small nonzero residual; a context-appropriate tolerance accommodates it.

2c)

For the same response vector $\mathbf{y}$, let X and $\widetilde{X}$ be full-column-rank design matrices with the same column space. In exact arithmetic, their ordinary least-squares fitted vectors satisfy $X\widehat{\boldsymbol{\beta}} = \widetilde{X}\hat{\boldsymbol{\gamma}}$.

Answer

Answer: True. Least-squares fitted values are the projection onto the column space, regardless of its basis.

2d)

Let $\Sigma = R^\top R$ be a Cholesky factorization. If $\widetilde{\Sigma} \neq \Sigma$ is another positive-definite matrix of the same size, the unchanged factor R always satisfies $\widetilde{\Sigma} = R^\top R$.

Answer

Answer: False. Since $R^\top R = \Sigma$ and $\widetilde{\Sigma} \neq \Sigma$, the unchanged R cannot factor the new matrix.

2e)

An error sequence satisfying $e_{n+1} = 2e_n^2$ with $0 < e_0 < 1/2$ converges quadratically to zero.

Answer

Answer: True. Here $e_n = (2e_0)^{2^n}/2 \to 0$, and $e_{n+1}/e_n^2 = 2$. Thus the order is two, unlike the linear and sublinear examples in the practice exam.

2f)

The standard secant method for solving $f(x) = 0$ requires an analytic formula for $f'(x)$ at every iteration.

Answer

Answer: False. Secant slopes use function values at two iterates instead of an analytic derivative.

Question 3: Conceptual [16]

Answer **3a(i)**, **3a(ii)**, and **3b(i)** separately. Use one or two sentences per part; brief formulas or R expressions may be used.

Throughout, $\|\mathbf{v}\|_2 = (\sum_j v_j^2)^{1/2}$ denotes the Euclidean norm, which evaluates the size of the vector $\mathbf{v}$.

3a) R and numerical computation [10]

3a(i) [4]

A numeric vector `x` contains missing measurements and at least one observed value. A student replaces each missing value by zero and averages all entries. Explain why this can be inappropriate, and state how to compute the mean of only the observed values. You may give an R expression or describe the calculation.

Answer

Answer: Zero replacement treats missing measurements as observed zeros and generally changes the average ([2] points: interpretation [1], effect [1]). Exclude missing entries and average only the observed values ([2] points: exclusion [1], denominator [1]); accept `mean(x, na.rm = TRUE)` for the latter [2] points.

3a(ii) [6]

For the same full-column-rank design matrix X and response $\mathbf{y}$, two numerical algorithms return approximate OLS coefficient vectors $\widehat{\boldsymbol{\beta}}_1, \widehat{\boldsymbol{\beta}}_2$ such that

$$\frac{\|\widehat{\boldsymbol{\beta}}_1 - \widehat{\boldsymbol{\beta}}_2\|_2}{\|\widehat{\boldsymbol{\beta}}_1\|_2} = 0.2, \qquad \frac{\|X\widehat{\boldsymbol{\beta}}_1 - X\widehat{\boldsymbol{\beta}}_2\|_2}{\|X\widehat{\boldsymbol{\beta}}_1\|_2} = 10^{-8}.$$

Both denominators are nonzero. Explain how nearly collinear columns of X can produce this behavior. Name one diagnostic and state qualitatively what result would indicate poor conditioning; you do not need to calculate its value.

Answer

Answer: Writing $\mathbf{d} = \widehat{\boldsymbol{\beta}}_1 - \widehat{\boldsymbol{\beta}}_2$, near dependence permits non-negligible $\mathbf{d}$ with small $X\mathbf{d}$: coefficient changes offset each other [2]. Thus the fitted values $X\widehat{\boldsymbol{\beta}}_1$ and $X\widehat{\boldsymbol{\beta}}_2$ can remain close [1]. Name a valid diagnostic [1] and interpret a concerning result [2]: a large condition number, a reciprocal condition number near zero, or a singular-value ratio near zero indicates sensitivity. Accept equivalent valid diagnostics; no numerical calculation or cutoff is required.

3b) Optimization [6]

3b(i) [6]

A numerical algorithm is used to minimize a continuously differentiable function $f : \mathbb{R} \to \mathbb{R}$ without constraints. In the current variable and objective scales, its step and derivative tolerances are $\varepsilon_x = 10^{-8}$ and $\varepsilon_d = 10^{-4}$. It stops at x_{k+1} because the step test passes, with

$$|x_{k+1} - x_k| = 10^{-10} < \varepsilon_x, \qquad |f'(x_{k+1})| = 10^{-1} > \varepsilon_d.$$

Give one plausible cause of the small update and explain how it can produce a small step. Explain why the step test alone does not establish approximate stationarity under the stated derivative tolerance.

Answer

Answer: A tiny step size is one possible cause [1]. For example, gradient descent in one dimension has $x_{k+1} - x_k = -\alpha_k f'(x_k)$, hence $|x_{k+1} - x_k| = \alpha_k |f'(x_k)|$ for $\alpha_k > 0$; a tiny α_k can yield a tiny update despite a non-negligible derivative [2]. Approximate stationarity requires the derivative to be sufficiently close to zero, here $|f'(x_{k+1})| \leq \varepsilon_d$ [2]. Accept an equivalent statement of the zero-derivative condition at a differentiable unconstrained local minimum. The reported derivative fails $|f'(x_{k+1})| \leq \varepsilon_d$, so the step test alone does not establish approximate stationarity [1]. Accept a correctly explained poor-scaling or numerical-stagnation mechanism for the first [3] points.

Answer Key and Grading Rubric

Question 1: Multiple Choice [54]

Part	1a	1b	1c	1d	1e	1f	1g	1h	1i
Answer	C	B	A	D	B	A	C	D	B

Award **6 points** for each correct answer and **0 points** otherwise.

Question 2: True or False [30]

Part	2a	2b	2c	2d	2e	2f
Answer	False	True	True	False	True	False

Award **5 points** for each correct answer and **0 points** otherwise.

Question 3: Conceptual [16]

Accept equivalent concise explanations. The expanded answers above give model responses and acceptable alternatives.

Part	Grading criteria	Points
3a(i)	Recognizes that missing values are treated as observed zeros (1) and that this changes the average (1); excludes missing entries (1) and uses the observed count (1). Accept `mean(x, na.rm = TRUE)` for the latter 2 points.	4
3a(ii)	Explains that near dependence permits offsetting coefficient changes (2) while fitted values remain similar (1); names a suitable diagnostic (1) and interprets a concerning result (2).	6
3b(i)	Gives a plausible cause (1) and explains the small update (2); states the small-derivative requirement for approximate stationarity (2) and recognizes that the reported derivative fails the stated tolerance (1).	6
Total		**16**

Past Exam 1

STAT 8670: Computational Methods in Statistics
Term: Fall 2025
Instructor: Chi-Kuang Yeh
Date: October 8, 2025
Total: 100 points

Exam Instructions

Note 1: Only a non-programmable calculator is allowed. Any other electronic devices are not allowed, and you would receive **0** for any usage of such a device during the exam.

Note 2: The maximum score is 100 points. If x is the total number of points earned, the recorded score is $\min(100, x)$.

Note 3: Please write your answers on separate papers. Do **NOT** write on this paper, even for multiple choice. Any answers you write on this sheet will **NOT** count towards your grade!

Grade distribution

Question	Content	Points	Score
1	Multiple Choice	18	
2	Conceptual	20	
3	Calculation	30	
4	Algorithms	16	
5	Convergence	16	
Total		**100**	

Question 1: Multiple Choice [18]

Each of the questions is worth [2] marks.

1a) [2]

In R, suppose

```
a = c(3, 2, 0)
b = c(1, 2, 3)
```

What is the output of `cbind(a, b)`?

A.
$$\begin{pmatrix} 3 & 1 \\ 2 & 2 \\ 0 & 3 \end{pmatrix}$$

B.
$$\begin{pmatrix} 3 & 2 & 0 \\ 1 & 2 & 3 \end{pmatrix}$$

C.
$$\begin{pmatrix} 3 & 2 \\ 0 & 1 \\ 2 & 3 \end{pmatrix}$$

D.
$$\begin{pmatrix} 1 & 3 \\ 2 & 2 \\ 3 & 0 \end{pmatrix}$$

Answer

A

1b) [2]

List is one of the most important object type in R. In R, suppose

```
x = list(a = 1:3, b = "hello")
```

What is the output of `print(x)`?

A.
```
$a
[1] 1 2 3
$b
[1] "hello"
```

B.
```
[1] 1 2 3 "hello"
```

C.
```
a b
1 2 3
"hello"
```

D.
```
$1
[1] "a" "b"
```

Answer

A

1c) [2]

What is the output of the following `R` code?

```
x <- list("a", "b", 1:4)
length(x)
```

A. 3

B. a b 1 2 3 4

C. 7

D. a b 1:4

Answer

A. 3

1d) [2]

Which of the data type allow non-homogeneous data structure to be stored?

A. list

B. vector

C. matrix

D. boolean

Answer

A. list

1e) [2]

For multivariate methods, to generate a random vector that follows a multivariate normal distribution $N_d(0, \Sigma)$ with a given covariance matrix Σ, one often uses the Cholesky decomposition. What is the role of the Cholesky factor L when $\Sigma = LL^\top$?

A. L is the eigenvector matrix

B. Multiply L by a standard normal vector gives the desired correlation structure

C. L^{-1} is used to whiten the data

D. None of the above

Answer

B. Multiply L by a standard normal vector gives the desired correlation structure

1f) [2]

In the acceptance–rejection algorithm, the proposal distribution $g(x)$ and the constant k should be chosen such that:

A. $g(x)$ is complicated to sample from, and k is as large as possible.

B. $g(x)$ is easy to sample from, and k is as small as possible while satisfying $f_X(x) \le k\, g(x)$ for all x.

C. $g(x)$ closely matches $f_X(x)$ but k can be arbitrarily large.

D. $g(x)$ is uniform on $[0, 1]$, and $k = 1$ regardless of $f_X(x)$.

Answer

B $g(x)$ is easy to sample from, and k is as small as possible while satisfying $f_X(x) \le k\, g(x)$ for all x.

1g) [2]

What is **not** a difference between theoretical results and computational reality?

A. Theoretical results assume exact arithmetic, while computers use finite precision.

B. Theoretical results may ignore algorithmic complexity, while computation must consider time and memory.

C. Theoretical results are always affected by rounding error, while computation is exact.

D. Theoretical models may assume infinite samples, while computation works with finite data.

Answer

C. Theoretical results are always affected by rounding error, while computation is exact. (This is false.)

1h) [2]

Let Z_1 and Z_2 be independent standard normal random variables, and denote a Chi Square distribution with degree of freedom ν by χ^2_ν . Which of the following statements is *correct*?

A. $Z_1 + Z_2 \sim \chi^2_2$

B. $Z_1^2 + Z_2^2 \sim \chi^2_2$

C. $Z_1 Z_2 \sim \chi^2_2$

D. $(Z_1 + Z_2)^2 \sim \chi^2_2$

Answer

B

1i) [2]

Consider estimating the integral

$$I = \int_0^1 e^{-x^2}\, dx$$

using Monte Carlo integration.

If we generate $X_1, X_2, \ldots, X_n \overset{\text{i.i.d.}}{\sim} \text{Uniform}(0,1)$, which of the following provides a consistent estimator of I?

A. $\frac{1}{n}\sum_{i=1}^{n} X_i^2$

B. $\frac{1}{n}\sum_{i=1}^{n} e^{-X_i^2}$

C. $\frac{1}{n}\sum_{i=1}^{n} e^{X_i^2}$

D. $\frac{1}{n}\sum_{i=1}^{n} e^{-X_i}$

Answer

B

Question 2: Conceptual [20]

2a) [4]

In your own words, explain why statistical computing is important. {2–3 sentences}

Answer

Any reasonable answers are fine.

2b) [6]

Identify three main reasons why computational results may differ from theoretical results. {1–3 sentences each}

Answer

Any reasonable answers are fine.

2c) [6]

Let $\Sigma \in \mathbb{R}^{r \times r}$ be a covariance matrix. Write its eigendecomposition and describe the dimensions and properties of the eigenvector and eigenvalue matrices. Explain what can be said about the signs of the eigenvalues.

Answer

A covariance matrix is symmetric and positive semidefinite, so

$$\Sigma = Q \Lambda Q^\top.$$

- $Q = [q_1, \dots, q_r] \in \mathbb{R}^{r \times r}$ contains an **orthonormal** set of eigenvectors. Thus $Q^\top Q = QQ^\top = I_r$ and $Q^{-1} = Q^\top$.
- $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_r) \in \mathbb{R}^{r \times r}$ contains the eigenvalues.
- All eigenvalues are nonnegative. They may be ordered as $\lambda_1 \geq \cdots \geq \lambda_r \geq 0$. Zero eigenvalues are allowed; all are strictly positive when Σ is positive definite.

2d) [4]

Let $X_1, X_2, \dots$ be i.i.d. random variables with $\mathbb{E}|X_1| < \infty$ and $\mathbb{E}[X_1] = \mu$. For the estimator

$$\hat{\mu}_n = \overline{X}_n = \frac{1}{n} \sum_{i=1}^{n} X_i,$$

state its limit and a mode of convergence guaranteed by these assumptions. Name the theorem that justifies your answer and state whether convergence in probability also holds.

Answer

By the **strong law of large numbers**,

$$\overline{X}_n \xrightarrow{\text{a.s.}} \mu.$$

Almost-sure convergence implies convergence in probability, so $\overline{X}_n \xrightarrow{p} \mu$ as well. The i.i.d. and finite absolute first-moment assumptions ensure that the stated strong law applies.

Question 3: Calculation [30]

3a) [6]

Suppose X has the cumulative distribution function

$$F_X(x) = \begin{cases} 0, & x < \ln(2/3), \\ 3e^x - 2, & \ln(2/3) \le x \le 0, \\ 1, & x > 0. \end{cases}$$

Derive the inverse CDF for $0 < u < 1$, and write the steps for generating an independent sample from this distribution using uniform random numbers. State the range of the generated values.

Answer

1. Solve $u = 3e^x - 2$ for x:

$$F_X^{-1}(u) = \ln\left(\frac{u+2}{3}\right), \qquad 0 < u < 1.$$

2. Generate independent $U_1, \dots, U_n \sim \text{Unif}(0, 1)$.
3. Set $X_i = \ln((U_i + 2)/3)$ for each i.

The support is $[\ln(2/3), 0]$, and the generated values lie in its interior with probability one. The CDF is continuous and increasing on that interval, taking values 0 and 1 at its endpoints, so the inverse-CDF construction gives the required distribution.

3b) [8]

Describe a simulated annealing algorithm for **minimizing** an objective function $f(x)$. Use a symmetric random-walk proposal and geometric cooling. Specify the initialization, acceptance rule for better and worse proposals, cooling schedule, and a suitable stopping rule.

Answer

1. Choose an initial state x_0, an initial temperature $T_0 > 0$, a cooling factor $0 < \alpha < 1$, a maximum iteration count K, and a minimum temperature $0 < T_{\min} < T_0$. Keep track of the best state found.
2. At iteration i, propose $x' = x_i + \varepsilon_i$, where the increment distribution is symmetric about zero, and compute $\Delta = f(x') - f(x_i)$.
   - If $\Delta \le 0$, accept the proposal: $x_{i+1} = x'$.
   - If $\Delta > 0$, accept it with probability

$$p = \exp\left(-\frac{\Delta}{T_i}\right) = \exp\left(-\frac{f(x') - f(x_i)}{T_i}\right).$$

     Draw $U \sim \text{Unif}(0,1)$ and accept when $U \le p$. If rejected, set $x_{i+1} = x_i$.
3. Update the best state if the objective improves, and cool using $T_{i+1} = \alpha T_i$.
4. Repeat until K iterations have been performed or the temperature falls below $T_{\min}$. Return the best state found.

A single rejected proposal gives $x_{i+1} = x_i$ even when the search is far from an optimum, so zero change in one iteration is not a sufficient stopping rule. This finite run provides a candidate solution; it does not guarantee the global minimum.

3c) [6]

Write the steps for simulating a random variable from the Gaussian mixture

$$0.7\, N(\mu_1, \sigma_1^2) + 0.3\, N(\mu_2, \sigma_2^2).$$

State any independence assumptions used in your construction.

Answer

1. Independently generate $X \sim N(\mu_1, \sigma_1^2)$, $Y \sim N(\mu_2, \sigma_2^2)$, and $U \sim \text{Unif}(0,1)$.
2. If $U \le 0.7$, set $W = X$; otherwise set $W = Y$.

Independence of the component selector and the component draws gives

$$\Pr(W \le w) = 0.7 \Pr(X \le w) + 0.3 \Pr(Y \le w),$$

which is the desired mixture CDF. Equivalently, first select the component using U, then generate one normal draw from that component.

3d) [10]

Let Z_1, Z_2 be independent standard normal random variables. Prove that $W = Z_1^2 + Z_2^2 \sim \chi_2^2$ using moment generating functions.

You may use the facts that, for $Z \sim N(0,1)$,

$$M_{Z^2}(t) = (1-2t)^{-1/2}, \qquad t < \tfrac{1}{2},$$

and that the MGF of a χ_ν^2 random variable is

$$M_{\chi_\nu^2}(t) = (1-2t)^{-\nu/2}, \qquad t < \tfrac{1}{2}.$$

Explain where independence is used.

Answer

Since Z_1 and Z_2 are independent, their squares are independent. For $t < 1/2$,

$$\begin{aligned} M_W(t) &= \mathbb{E}\left[e^{t(Z_1^2+Z_2^2)}\right] \\ &= \mathbb{E}\left[e^{tZ_1^2} e^{tZ_2^2}\right] \\ &= \mathbb{E}\left[e^{tZ_1^2}\right] \mathbb{E}\left[e^{tZ_2^2}\right] \\ &= (1-2t)^{-1/2}(1-2t)^{-1/2} \\ &= (1-2t)^{-1} = M_{\chi_2^2}(t). \end{aligned}$$

Independence justifies the third equality. Both MGFs exist on an open interval containing zero, so uniqueness of the MGF gives $Z_1^2 + Z_2^2 \sim \chi_2^2$.

Question 4: Algorithms [16]

We want to **find the root** of

$$f(x) = \ln(2x) - 2, \qquad x > 0.$$

Use the bracket $[1, 10]$ for bisection and the initial value $x_0 = 7$ for Newton's method. Retain full precision in intermediate calculations and report final numerical values to four decimal places.

4a) [8]

Apply the **bisection method** with $a_0 = 1$ and $b_0 = 10$. At iteration j, define $c_j = (a_{j-1} + b_{j-1})/2$, then update the bracket to $[a_j, b_j]$ so that it still contains the root. Report

$$\boldsymbol{\theta}_j = (a_j, b_j, c_j, f(a_j), f(b_j))$$

for $j = 1, 2$, and explain each bracket update.

Answer

Initially, $f(1) = \ln 2 - 2 \approx -1.3069 < 0$ and $f(10) = \ln 20 - 2 \approx 0.9957 > 0$. The function is continuous on $[1, 10]$, so the bracket contains a root.
Iteration 1: $c_1 = (1 + 10)/2 = 5.5$ and $f(c_1) = \ln 11 - 2 \approx 0.3979 > 0$. It has the same sign as $f(b_0)$, so replace the upper endpoint: $a_1 = 1$ and $b_1 = 5.5$.

$$\boxed{\boldsymbol{\theta}_1 \approx (1,\ 5.5,\ 5.5,\ -1.3069,\ 0.3979).}$$

Iteration 2: $c_2 = (1 + 5.5)/2 = 3.25$ and $f(c_2) = \ln 6.5 - 2 \approx -0.1282 < 0$. It has the same sign as $f(a_1)$, so replace the lower endpoint: $a_2 = 3.25$ and $b_2 = 5.5$.

$$\boxed{\boldsymbol{\theta}_2 \approx (3.25,\ 5.5,\ 3.25,\ -0.1282,\ 0.3979).}$$

4b) [8]

For the same root-finding problem, apply **Newton's method** with initial value $x_0 = 7$. Derive the update formula and report $\boldsymbol{\eta}_j = (x_j, f(x_j))$ for $j = 1, 2$.

Answer

Since $f'(x) = 1/x$, Newton's update is

$$x_{j+1} = x_j - \frac{f(x_j)}{f'(x_j)} = x_j[3 - \ln(2x_j)].$$

Starting with $x_0 = 7$, the two iterations are

$$\begin{aligned} x_1 &= 7[3 - \ln 14] \approx 2.526598693, \\ f(x_1) &\approx -0.379978811, \\ x_2 &= x_1[3 - \ln(2x_1)] \approx 3.486652661, \\ f(x_2) &\approx -0.057910666. \end{aligned}$$

Thus, to four decimal places,

$$\boxed{\begin{aligned} \boldsymbol{\eta}_1 &= (2.5266,\ -0.3800), \\ \boldsymbol{\eta}_2 &= (3.4867,\ -0.0579). \end{aligned}}$$

The exact root is $x^* = e^2/2 \approx 3.6945$. The iterations use the unrounded previous value.

Question 5: Convergence [16]

For each sequence, identify its limit x_∞ and its rate of convergence using the error ratio

$$\mu = \lim_{n\to\infty} \frac{|x_{n+1} - x_\infty|}{|x_n - x_\infty|}.$$

5a) [6]

For $x_n = 1 + (2/5)^n$, find x_∞ and μ, and classify the convergence rate. Show the error-ratio calculation.

Answer

Since $(2/5)^n \to 0$, the limit is $x_\infty = 1$. The error ratio is

$$\frac{|x_{n+1} - 1|}{|x_n - 1|} = \frac{(2/5)^{n+1}}{(2/5)^n} = \frac{2}{5}.$$

Thus $\mu = 2/5 = 0.4 \in (0, 1)$, so the sequence converges **linearly** to 1.

5b) [10]

A sequence converges **sublinearly** to x_∞ if it converges to that limit and its error ratio tends to 1. Construct an infinite sequence with this property, state its limit, and verify the error-ratio limit.

Answer

Take $x_n = 1/n$ for $n = 1, 2, \ldots$. Then $x_n \to 0$, so $x_\infty = 0$, and

$$\frac{|x_{n+1} - 0|}{|x_n - 0|} = \frac{1/(n+1)}{1/n} = \frac{n}{n+1} \longrightarrow 1.$$

Hence $\mu = 1$ and the convergence is **sublinear**.

Part III

Project

Instruction

Group size: 1-3 students | **Language:** R or Python | **Course grade:** 30%

Choose **one statistical method**, implement its main algorithm, test it, and explain what you learned. One well-developed, established method is enough.

Group Members

Group	Members	Topic
1	Jones, Airianna; Grant, Jada; Honne, Shravya	TBA
2	Guo, Jennifer; Pham, Duc; Park, Jieun	TBA

Project Overview

What you will produce

Use the same method for all three components. Follow the links for full requirements and rubrics.

Component	Deliverable	Weight
Report	An 8-10 page PDF explaining the method and results, plus source files	10%
Package	An installable R or Python package with documentation, an example, and tests	10%
Presentation	An oral presentation explaining the method, implementation, and findings	10%

Python option: You may build a **Python package**; see the Python requirements.

Report writing model

Use **The R Journal** as the writing model for the 8-10 page report: introduce the method and software, demonstrate a reproducible package workflow, and evaluate accuracy, computing time, and limitations. This structure applies to both **R and Python projects**. See the journal references and report structure and downloadable Quarto starter.

How to get started

1. **Choose a topic.** Use the instructor's list (TBA), or propose your own method for instructor approval.
2. **Propose your plan.** Send member names, the method, language, core algorithm, and a planned experiment. See approval details.
3. **Build and compare.** Test two settings against a known answer or reference method; explain accuracy, computing time, and limitations.

A few ideas

- **Bootstrap intervals:** how often do intervals contain the true population mean?
- **Gradient descent:** how does step size affect convergence when fitting a regression?
- **Monte Carlo integration:** how do more random draws improve accuracy?

Due on December 13, 2026: report and package. Presentation date, time, and time limit: **TBA**. All members contribute and present. No slides or other presentation files need to be submitted.

Report

STAT 8670 | Fall 2026
Due on December 13, 2026
Report: 10 points (10% of the course grade)
Group size: 1-3 students
Project total: 30 points (30% of the course grade)

What is the project?

Choose **one statistical method**, implement it in **R or Python**, and show how well it works. Your group will produce a reusable package, explain the method and results in a written report, and present the project to the class.

An established method is a suitable topic. The goal is to understand its statistical reasoning, implement its main computational steps, and evaluate the results. Your project should answer:

1. **What problem does the method solve?** Identify the inputs, outputs, and assumptions.
2. **How does the method work?** Explain the algorithm and implement its core steps.
3. **How well does it work?** Use reproducible experiments to examine accuracy, computing time, and limitations.

Component	What your group submits or delivers	Points
Report	An 8-10 page PDF explaining the method, implementation, and results, with its source files	10
Package	An installable R or Python package, documentation, a worked example, and tests	10
Presentation	An oral presentation explaining the project; no presentation files to submit	10
Total	**One coordinated project**	**30**

Each component is worth **10 points and 10% of the course grade**: report 10%, package 10%, and presentation 10%. Each page gives the requirements and point-by-point rubric for that component.

Groups and topic approval

Work individually or in a group of **2 or 3 students**. Choose one group name and list every member on the report and repository, and introduce them during the oral presentation. Each member should make a meaningful contribution and be able to explain the method and results. Groups submit one shared set of materials and receive a shared project score under the rubrics below.

You may choose a topic from the **instructor's topic list** or propose your own statistical method. **Self-selected topics require the instructor's approval.** Confirm the scope with the instructor before starting the full implementation.

> **i** Topic selection and schedule
>
> **Instructor's topic list:** TBA
> **Group and topic proposal deadline:** TBA
> **Report and package:** Due on December 13, 2026
> **Presentation date, time, and time limit:** TBA, after groups are finalized
> Send the instructor a short proposal with your group members, proposed method, choice of R or Python, core algorithm to implement, and one planned experiment or application. Explain what your group will implement and which existing libraries you plan to use.

Examples of a manageable project

These examples illustrate possible scope; confirm your choice with the instructor.

Statistical method	What you could implement	What you could investigate
Bootstrap confidence intervals	Resampling and interval construction for a mean or median	How often do the intervals cover the true value as sample size changes?
Numerical optimization	Gradient descent for a regression objective	How do step size and starting value affect convergence, accuracy, and time?
Monte Carlo integration	A basic estimator and a variance-reduction method	How does estimation error change as the number of draws increases?
A permutation test	Shuffling group labels and calculating a test statistic	Does the test maintain its intended rejection rate under a simulated null model?
A regression or dimension-reduction method	The main fitting algorithm and a prediction or transformation function	How do its results compare with a trusted reference implementation?

For example: *"We implement percentile bootstrap confidence intervals for a population mean in Python, compare coverage and interval length at two sample sizes, and explain when the method performs poorly."* A focused question like this is enough to organize all three project components.

Report requirements

Write **8-10 pages of main text**, excluding the title page, references, and appendices. Use Quarto, R Markdown, or LaTeX and submit the rendered **PDF plus its source files**. Use readable 11- or 12-point text, standard margins, and numbered sections. Place long code listings and supplementary output in an appendix or the repository.

Writing model: The R Journal

Write the report as a short **R Journal-style software article**: explain the statistical problem, introduce your implementation, show a usable workflow, and evaluate its behavior. The R Journal author guide emphasizes a broad readership, software design, practical use, and reproducible results. For this course, also explain the statistical formulation and the core algorithm your group implements, as required by the rubric.

This is a **course adaptation** of the journal's structure. Both **R and Python projects** use it. Keep the course's 8-10 page limit, readable 11- or 12-point text, and PDF-plus-source submission. A course project may implement an established method; journal acceptance, methodological novelty, and CRAN/PyPI publication are not requirements. The existing report, package, and presentation rubrics determine the grades.

Read these references before drafting:

- R Journal author guide: article organization, software reporting, and reproducibility.
- Official rjtools documentation: the journal's R Markdown authoring workflow. This is an optional tool for R users; the course also accepts Quarto and LaTeX.
- Official article-template showcase: links to source files and a rendered sample article.
- Kenny and Wolock (2025), "SimEngine: A Modular Framework for Statistical Simulations in R": a published example linking motivation, design choices, a user workflow, and discussion. Use its exposition as a model; your project can remain a small implementation of one method.

Suggested report structure

Use a descriptive title identifying the method or package, and list the group and all members. Start with an abstract of **about 150-200 words**, covering the problem, implementation, evaluation, and main result. This length is a course writing suggestion. Define technical terms for readers who have not seen your code.

The following is an illustrative **9-page plan**, including the abstract. Adjust it within the 8-10 page main-text limit; references, the title page, and appendices are excluded.

Section	What the reader should learn	Pages
Abstract	The purpose, implementation, evaluation, and main finding.	0.5
1. Introduction	The problem, intended users, existing approaches, and the scope of your package.	1

Section	What the reader should learn	Pages
2. Statistical method	The estimand, test, or optimization objective; notation, assumptions, inputs, and outputs. State constraints when relevant.	1
3. Algorithm and implementation	Core steps or pseudocode, design choices, tuning/stopping rules, numerical safeguards, and which routines you wrote or reused.	1.5
4. Package workflow and example	Installation, a minimal function call, its output, and its statistical interpretation. Use a small example a reader can reproduce.	1
5. Experiments and results	Two settings, a known answer or reference method, accuracy and timing comparisons, and an explanatory results figure.	3
6. Discussion and conclusion	What worked, computational cost, a limitation or failure case, and one useful extension.	1

For Section 5, give the simulation or data-analysis design, metric definitions, random seeds, replication counts when relevant, and software/hardware details needed to understand the timing results. Use the same experiment conditions for your implementation and the reference. Distinguish correctness checks from performance comparisons. Include at least one reproducible simulation study or data analysis with **two settings**, as required above.

After the main text, include **References**, **Software availability and reproducibility**, and **Author contributions**. Cite methods, data, packages, and borrowed or adapted code with consistent author-year references. Identify the package repository and the exact release tag or commit used for all results. Give commands for reproducing the main tables and figures. Add a brief contribution table; an individual project should say that the work was completed individually.

Number and caption figures and tables and explain their statistical meaning in the text. A few short, evaluated code examples belong in the workflow section. Keep full source listings, long logs, and additional experiments in the repository or supplementary material. Use the same submitted package version for the report and package assessment.

Download and adapt the template

Download the STAT 8670 report starter (ZIP). Unzip it to obtain `Project_Report_GroupName.qmd` and `README.md`. Replace `GroupName` and the writing prompts, add your code and citations, and render the matching PDF. The starter uses the R Journal-inspired structure with ordinary Quarto, so it works as a report outline for either an R or a Python package.

Report grading [10]

The first four criteria assess **clarity and writing** [**4**]; the last four assess **method, experiments, and interpretation** [**6**].

Criterion	Full-credit evidence	Points
R1. Organization	Required sections form a logical account of the problem, method, and findings.	1
R2. Writing and notation	Explanations are readable and technically precise; symbols are defined and used consistently.	1
R3. Figures and tables	Results are legible, labeled, captioned, and interpreted in the text.	1
R4. Scientific reporting	The report follows the length and format requirements and includes references and a contribution statement.	1
R5. Statistical method	The purpose and assumptions are correct [1]; the mathematical formulation is correct and clearly stated [1].	2
R6. Algorithm	The computational steps and important implementation choices are explained accurately.	1
R7. Experiments	Reproducible experiments cover two settings [1]; a meaningful reference or baseline supports the accuracy and timing comparisons [1].	2
R8. Interpretation	Conclusions follow from the evidence and discuss computational cost, limitations, and a possible extension.	1
Total		**10**

Score each 1-point requirement as **1** for complete and correct evidence, **0.5** for partially correct or incomplete evidence, or **0** for missing or substantially incorrect evidence. The 2-point criteria each contain two 1-point requirements, as shown above. A missing report receives **0/10**. Package and presentation marks are assessed separately using their own rubrics.

i Report submission checklist

One member submits the following on behalf of the group to the **iCollege project submission area** (folder details TBA) by **December 13, 2026**:

- `Project_Report_GroupName.pdf`.
- The matching `.qmd`, `.Rmd`, or `.tex` source, with supporting files in a ZIP if needed.
- The package repository URL and the exact release tag or commit used for the report.

Replace `GroupName` with your group's chosen name and use it consistently for all components. Include all member names. Check that the report's figures and tables can be reproduced from the submitted package and experiment code.

Next: Package requirements and grading | Presentation requirements and grading

Package

STAT 8670 | Fall 2026
Due on December 13, 2026
Total: 10 points (10% of the course grade)
Group size: 1-3 students
Language: R or Python

Build an **installable R package or Python package** that implements your group's chosen statistical method. A user should be able to install it, call a documented function, and reproduce a worked example. Use the same method and submitted version for your report and presentation.

Start with Instruction for a quick overview, and review the topic approval details before choosing your topic.

Python packages are accepted. Follow the Python requirements; R and Python packages use the same **10-point rubric**.

What your group should implement

Implement the **main algorithmic steps** of the approved method in functions your group writes. You may use standard tools for arrays, matrix operations, random-number generation, plotting, and other supporting tasks. Identify these dependencies and clearly distinguish your implementation from any existing method used for comparison.

For example, a bootstrap project should implement the resampling loop and interval construction; a gradient-descent project should implement the update rule and stopping criterion. The package should expose at least one main user-facing function with meaningful inputs and outputs.

> **i** A small, complete package is a suitable project
>
> Choose one method with a manageable scope. Make it correct, usable, documented, and reproducible. A package with one well-developed method can meet all requirements. Distribution through your GitHub repository is sufficient; publication on CRAN or PyPI is optional.

Required package contents

1. **Working implementation.** Provide the core method, sensible defaults, and outputs that match the documentation. For iterative methods, expose the stopping tolerance or iteration limit and report convergence information.

2. **Input checks.** Detect invalid dimensions, unsupported input values, and other relevant failure cases. Give useful error or warning messages.
3. **Function documentation.** Describe each public function's purpose, arguments, return values, assumptions, and a runnable example.
4. **One worked example.** Provide an R vignette or a Python tutorial/notebook that runs from start to finish. Show inputs, a function call, the output, and its statistical interpretation.
5. **Meaningful automated tests.** Cover a known-answer or reference case, an edge case, and invalid input. Set a random seed and use an appropriate numerical tolerance for stochastic or approximate calculations. Tests should check behavior and results, not just that a function runs.
6. **Reproducible experiments.** Include the scripts or notebooks used for the report, with seeds, dependencies, and instructions for generating the main figures and tables. Include usable example data or clear instructions for obtaining or generating it.
7. **README and attribution.** State the project purpose, group members, installation steps, a minimal usage example, commands for running tests and experiments, and the software versions used. Cite external code, methods, and data, and include an appropriate license.

R and Python requirements

Both languages are assessed using the **same 10-point rubric**.

Requirement	R package	Python package
Package structure	Include `DESCRIPTION`, `NAMESPACE`, and functions in `R/`.	Include `pyproject.toml` and an importable package directory.
Dependencies	Declare packages used by your implementation and examples.	Declare dependencies and the supported Python version.
Function reference	Use roxygen2 and include generated help files in `man/`.	Include docstrings describing parameters, outputs, and examples.
Worked example	Include at least one working vignette.	Include at least one runnable tutorial or notebook.
Tests	Include automated tests, for example with testthat.	Include automated tests, for example with pytest or unittest.
Installation and checks	The source package installs and `R CMD check` finishes with **no errors**. Explain any remaining warnings or notes.	The package installs in a clean virtual environment, imports successfully, and all submitted tests pass.

Record the installation and checking commands in the README. Include the resulting check or test log with the submission. Re-run the worked example using the installed package before submitting.

Package grading [10]

The first four criteria assess **functionality and implementation** [**6**]; the last four assess **documentation, examples, testing, and checks** [**4**].

Criterion	Full-credit evidence	Points
P1. Statistical correctness	The core algorithm is implemented correctly [2]; results agree with known answers or a suitable reference within justified tolerance [1].	3
P2. Code quality	Functions are readable and organized, with sensible use of computation and memory.	1
P3. Input handling	Relevant invalid inputs and failure cases are detected with useful messages.	1
P4. Usable outputs	Returned values match the documentation and provide the information needed to use and interpret the method.	1
P5. Documentation	Public functions and the README provide complete, accurate instructions and attribution.	1
P6. Worked example	A vignette or tutorial runs completely and explains a meaningful statistical use case.	1
P7. Tests	Automated tests cover a reference case, an edge case, and invalid input, with sensible tolerances.	1
P8. Installation and checks	The submitted version meets the language-specific installation/check requirements and includes the check or test log.	1
Total		**10**

Each listed requirement is scored on its stated points. Partial credit is available in **0.5-point increments** for correct but incomplete work; missing or substantially incorrect evidence receives zero for the relevant requirement. A missing package receives **0/10**. The report is assessed separately for how clearly it explains and evaluates the method.

i Package submission checklist

One member submits the following for the group to the **iCollege project submission area** (folder details TBA) by **December 13, 2026**:

- A **public GitHub repository URL** containing the complete package, documentation, tests, and experiment code.
- An **exact release tag or commit identifier** for the version to be graded. Use this same version in the report and presentation.
- `Project_Package_GroupName.zip`, a snapshot of that version, including the check or test log.

Before submission, follow your own README from a clean R session or Python environment: install the package, run the tests, run the worked example, and reproduce the report results. The tagged or identified version is the grading submission; later repository edits do not replace it automatically.

Development resources

- **R:** R Packages, second edition covers package structure, documentation, vignettes, and testing.
- **Python:** The Python Packaging User Guide explains a minimal installable package and `pyproject.toml`.
- **Python tests:** The pytest getting-started guide shows how to write and run tests.

Related: Instruction | Report requirements and grading | Presentation requirements and grading

Presentation

STAT 8670 | Fall 2026
Presentation date and time: TBA
Format: In-class oral presentation; no file submission
Time limit: TBA, after groups are finalized
Total: 10 points (10% of the course grade)
Group size: 1-3 students

Present the statistical method your group implemented in **R or Python**. Help classmates understand the problem, the main computational idea, the evidence that the method works, and a limitation. Use the same project and submitted package version as your report and package.

What to present

Organize the presentation around this sequence:

1. **Problem and motivation.** Start with a concrete statistical question and explain why someone would use your method.
2. **Method.** Define the key assumptions, inputs, and outputs. Explain the main algorithm in words or with a small worked example. Define the objective and constraints if your method involves optimization.
3. **Implementation.** Identify the main functions your group wrote and one important computational choice, such as a stopping criterion, a tuning parameter, or numerical stabilization.
4. **Package example.** Walk classmates through a small input, the function call, its output, and what the result means. A verbal explanation is sufficient; a live demonstration is optional.
5. **Results.** Explain the main comparison from your report using relevant numerical results. Discuss accuracy and computing time, including what changes between the experimental settings.
6. **Conclusion.** State the main finding, a limitation or failure case, and one possible extension. Finish with a clear take-home message and be ready for questions.

> **i** Group participation
>
> For groups of 2 or 3, **every member must speak** and explain a substantive part of the work. Plan the transitions and rehearse together within the announced time limit. Every member should understand the central method and results and be prepared to answer questions. Individual projects follow the same content requirements and rubric.

Oral presentation format

Begin by introducing the project and all group members. Explain the work at a level classmates who have taken STAT 8670 can follow, and leave time for questions within the announced time limit.

Slides are optional. No slides or other presentation files need to be submitted. The assessment is based on your oral explanation and discussion.

Presentation grading [10]

The presentation is assessed as **clarity and delivery** [**4**], **organization and communication** [**3**], and **technical depth and application** [**3**].

Criterion	Full-credit evidence	Points
S1. Clear explanation	The problem, method, and main finding are explained accurately at a level classmates can follow.	2
S2. Delivery and participation	Delivery is audible, well-paced, and within the announced time limit; every member explains a substantive part.	1
S3. Understanding and questions	The group demonstrates understanding and responds to questions with accurate reasoning.	1
S4. Organization	The talk progresses logically from motivation to method, implementation, results, and conclusion.	1
S5. Accessible language	Key terms are defined, and explanations connect the statistical ideas to the project question.	1
S6. Communication of results	The main findings and their practical meaning are highlighted, with relevant numerical evidence.	1
S7. Implementation insight	The group explains the main computational steps and an important implementation choice.	1
S8. Worked example	A verbal walkthrough explains a small input, a package call, its output, and interpretation.	1
S9. Critical discussion	The conclusion identifies a supported finding, a limitation, and a useful extension.	1
Total		**10**

Partial credit is available in **0.5-point increments** for partially demonstrated criteria. Missing or substantially incorrect evidence receives zero for the relevant criterion. A missing presentation receives **0/10**. Slides are not required to earn full credit. The report and package are graded separately.

> **i** Presentation arrangements
>
> Present in class on the announced date. Presentation order, date, time, and duration are **TBA** and will be announced after groups are finalized.

Related: Instruction | Report requirements and grading | Package requirements and grading